Warsaw University of Technology
The Faculty of Electronics and Information Technology
Institute of Computer Science

Academic year 2010/2011



BACHELOR OF SCIENCE THESIS

Stefan Karol Wójcik, Bartosz Maciej Zając

Learning Management System

Supervisor:
Roman Podraza, Ph.D.

Evaluation .....

.....
Signature of the Head
of Examination Committee



Specialization: Electrical and Computer Engineering, Computer Systems and Networks

Date of birth: 11 October 1988 r.

Date of the beginning of the studies: 1 October 2007 r.

Curriculum Vitae

I was born on 11th of October 1988 in Starachowice, Poland. After graduating from elementary and secondary school in Krynki, I continued my education in 2nd Stanisław Staszic High School in Starachowice. My class profile majored in Mathematics and Physics. In October 2007 I began my studies at Warsaw University of Technology. Between February and June 2010 I participated in EUKLA (European Korean Leadership Alliance) Student Exchange and Internship Program at Kyungpook National University in Daegu, South Korea.

.....
student's signature

Bachelor of Science Examination

Examination was held on: .....

With the result: .....

Final Result of the Studies: .....

Suggestions and Remarks of the B.Sc. Examination Committee: .....

.....



Specialization: Electrical and Computer Engineering, Computer Systems and Networks

Date of birth: 21 September 1988 r.

Date of the beginning of the studies: 1 October 2007 r.

Curriculum Vitae

I was born on 21st of September 1988 in Ostrowiec Świętokrzyski, Poland. After graduating from elementary and secondary school in Ostrowiec, I continued my education in 9th Klementyna Hoffmanowa High School in Warsaw. My class majored in Mathematics, Physics and Information Technology. In October 2007 I commenced my undergraduate studies at the Faculty of Electronics and Information Technology at Warsaw University of Technology. From September 2010 to March 2011 I was participating in the Erasmus student exchange program at Universidad Politécnica de Madrid, Spain.

.....
student's signature

Bachelor of Science Examination

Examination was held on: .....

With the result: .....

Final Result of the Studies: .....

Suggestions and Remarks of the B.Sc. Examination Committee: .....

.....

SUMMARY

The CampusNet platform is a software-oriented approach to enhance the learning experience within a high school. Mutually connected modules cover every educational, administrative and communicational aspect of school's environment and support learning processes. The project consists of three main parts, i.e.:

1. *LMS - Learning Management System - related to school management and administration,*
2. *VLE - Virtual Learning Environment - module dedicated to e-learning process,*
3. *WNI - Wireless Network Infrastructure - physical infrastructure of school's network.*

Three aforementioned modules form a coherent system, that significantly improves the functioning of any high school in terms of information exchange, academic materials' availability and access to relevant data.

Key words: *Programming, Python, Django, JavaScript, jQuery, AJAX, JSON, Web Applications, E-learning*

STRESZCZENIE

Tytuł: *System Zarządzania Nauczaniem*

System CampusNet to internetowa aplikacja wspierająca proces nauczania w szkole średniej. Wzajemnie połączone moduły wspomagają również wewnątrzszkolną komunikację, usprawniają zarządzanie wszelkimi rodzajami szkolnych danych oraz ułatwiają administrowanie placówką. Projekt został podzielony na trzy główne części:

1. *SZN – System Zarządzania Nauczaniem,*
2. *WŚE – Wirtualne Środowisko Edukacyjne,*
3. *ISB – Infrastruktura Sieci Bezprzewodowej.*

Części te tworzą spójną platformę, która skutecznie poprawia wydajność funkcjonowania instytucji edukacyjnej w najważniejszych kwestiach, takich jak wymiana informacji, dostępność materiałów dydaktycznych czy też stały dostęp do istotnych danych.

Słowa kluczowe: *Programowanie, Python, Django, JavaScript, jQuery, AJAX, JSON, Aplikacje sieciowe, E-learning*

Contents

1. Introduction . . . 1
- 1.1. Preface . . . 1
- 1.2. Objectives of the project . . . 1
- 1.3. Joint authorship . . . 2

2. Problem analysis . . . 4
- 2.1. Methodology of work . . . 4
- 2.2. System overview . . . 5
  - 2.2.1. Requirements analysis . . . 5
  - 2.2.2. Use cases . . . 9
- 2.3. Similar systems . . . 14
  - 2.3.1. Moodle . . . 14
  - 2.3.2. SchoolTool . . . 16
  - 2.3.3. Commercial systems . . . 18
- 2.4. Technology selection . . . 20
  - 2.4.1. Data Base Management System . . . 20
  - 2.4.2. Server-side technologies . . . 21
  - 2.4.3. Client-side technologies . . . 28
  - 2.4.4. Version control . . . 36
  - 2.4.5. Project management . . . 39
  - 2.4.6. Other tools . . . 41

3. Project development . . . 44
- 3.1. Database model . . . 44
  - 3.1.1. Design and integration . . . 44
  - 3.1.2. Models' description . . . 45
- 3.2. Web application . . . 51
  - 3.2.1. General description . . . 51
  - 3.2.2. Structural design and implementation . . . 51
  - 3.2.3. Application security . . . 60
  - 3.2.4. Cross-browser compatibility . . . 61
  - 3.2.5. Internationalization and localization . . . 63

4. Testing . . . 65
- 4.1. Methodology . . . 65
- 4.2. Tools . . . 66
- 4.3. Testing the application . . . 66
  - 4.3.1. Functional testing . . . 67
  - 4.3.2. Regressive testing . . . 68
  - 4.3.3. Usability testing . . . 68

5. Relevant problems and future development . . . 69
- 5.1. Future development . . . 69
  - 5.1.1. Optimization . . . 69
  - 5.1.2. Functionality extension . . . 69
  - 5.1.3. Graphical User Interface . . . 70

6. Conclusions . . . 71

6.1. General conclusions 71
6.2. Conclusions on technology selection 71
6.3. Conclusions on project development 72

A. Installation manual 73
A.1. Setup workflow 73
A.2. Apache configuration 75

B. Administrator manual 80
B.1. Main menu 80
B.2. Adding new entry 81
B.3. Form validation 82
B.4. Settings 83

C. User manual 84
C.1. Logging in 84
C.2. Event calendar 85
C.3. Timetable 88
C.4. E-register 89
C.4.1. Marks 90
C.4.2. Attendance 96
C.5. Settings 98
C.6. Community 100
C.7. Messages 102
C.8. Lessons 105
C.9. Board 107
C.10. Subject's description 108
C.11. Paricipants 110

D. Testing manual 111
D.1. Setup 111
D.2. Running functional testing 112
D.3. Running regressive testing 113

E. Database visualization 114

Bibliography 116

1. Introduction

1.1. Preface

This document is the written result of a bachelor thesis, submitted in complete fulfillment of all requirements for the Bachelor's Degree in Electrical and Computer Engineering at Warsaw University of Technology, Faculty of Electronics and Information Technology. It demonstrates work done by the authors from the beginning of October 2010 to the end of May 2011. This thesis has been developed under the supervision of Roman Podraza, Ph.D. from the Institute of Computer Science at Warsaw University of Technology. The document describes Learning Management System, a module of CampusNet platform created and developed by the authors of this bachelor's thesis. Nonetheless, as LMS is an integral part of bigger system, references to its other parts exist within this record.

1.2. Objectives of the project

The idea behind CampusNet platform and also the main objective of our thesis is to create a fully functional system supporting operation of a high school. In other words – one may call it a catch-phrase (hopefully of high quality) – this design is aimed at providing better learning experience. The team desired to introduce a large number of functionalities, that will cover every aspect of secondary school functioning.

All of aforementioned ought to be easily accessible for school community, which implies the existence of demand for versatility and availability of the system. In the presence of such needs, the decision to design an application accessible via web browser seems to be well justified. Due to the fact, that the principles of work of desired functionalities are operations on educational data, creation of a decent database arose as another objective.

Even though this particular project is a compound problem, it should be perceived by the users as intuitive and easy to approach. Assuring simple access to large amount of mutually connected sets of information and arranging them in a suitable way turn out to be another target.

Another important issue results from the fact, that CampusNet system is a subject of three separate theses, each of them concerning single module of the system. The entire task, conducted by three different groups, has to be integrated and coherent, which introduces the need of proper work distribution and coordination.

Last objective refers to the implementation of the system in a real school. Even though it is beyond the scope of our theses, we have already started this process in cooperation with Bolesław Prus High School[1] in Skierniewice, Poland, supplying it with a beta version of CampusNet platform.

[1] Official webpage: `http://www.loprus.pl/`

1.3. Joint authorship

As stated before, Learning Management System is a part of the CampusNet platform, software project developed by a group of five people, namely:

- Łukasz Grądzki – Virtual Learning Environment,
- Bartosz Staryga – Wireless Network Infrastructure,
- Piotr Więcek – Wireless Network Infrastructure,
- Stefan Wójcik – Learning Management System,
- Bartosz Zając – Learning Management System.

The project is divided into three individual modules (Fig. 1.1), each of them being a subject of separate bachelor thesis. Nonetheless, the foundation of the project has been laid as a group work and discussion on strategic decisions involved the entire team. Thus, some sections (specified within the list below) refer to the CampusNet platform in general instead of focusing on LMS only:

- methodology of work (Sec. 2.1),
- technology selection (Sec. 2.4),
- database model (Sec. 3.1),
- internationalization and localization (Sec. 3.2),
- user and installation manuals (See Appendices).

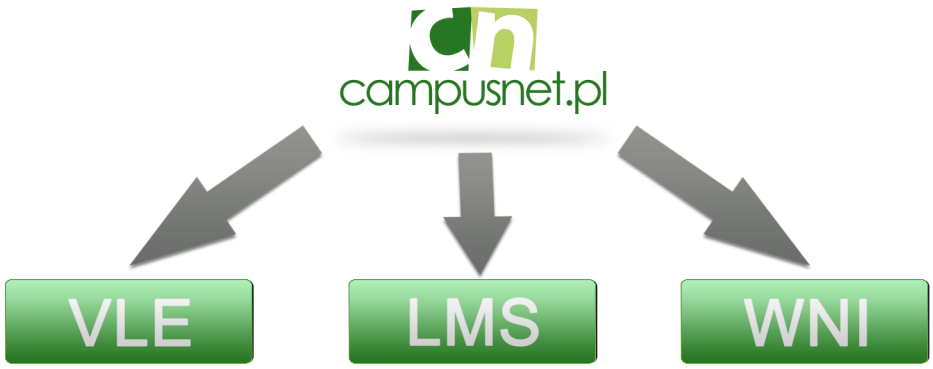


Figure 1.1. *General structure of CampusNet educational platform.*

Source code of the application exists as an integral part of both VLE and LMS modules. More detailed information will be given in further sections of this document, as well as within the code itself, in form of comments. The code related to Learning Management System may be found within the following files and directories:

- ◇ `campusnet/admin_panel/*`
- ◇ `campusnet/db_model/*`
- ◇ `campusnet/parent_panel/*`
- ◇ `campusnet/static/*` (except for several VLE files)
- ◇ `campusnet/student_panel/*`
- ◇ `campusnet/teacher_panel/*`
- ◇ `campusnet/welcome_page/*`

This document refers to the Learning Management System module, designed by Stefan Wójcik and Bartosz Zając. It was developed under the supervision of Roman Podraza, Ph.D. The remaining parts of CampusNet platform – VLE and WNI were supervised by Julian Myrcha, M.Sc.

Above all, this bachelor's thesis is a shared achievement of the authors'. Altough it should be considered as created in full collaboration, one can distinguish several sections that were written individually.

Bartosz Zając is fully responsible for:

- ◇ Section 1.1. Preface.
- ◇ Section 1.2. Objectives of the projects.
- ◇ Subsection 2.2.1. Requirements analysis.
- ◇ Section 2.3. Similar systems.
- ◇ Subsection 2.4.1. Data Base Management System.
- ◇ Subsection 2.4.4. Version control.
- ◇ Subsection 2.4.5. Project management.
- ◇ Subsection 2.4.6. Other tools.
- ◇ Section 3.1. Database model.
- ◇ Chapter 4. Testing

Stefan Wójcik was in authority of:

- ◇ Section 2.1. Methodology of work.
- ◇ Subsection 2.2.2. Use cases.
- ◇ Subsection 2.4.2. Server-side technologies.
- ◇ Subsection 2.4.3. Client-side technologies.
- ◇ Subsection 3.2.1. General description.
- ◇ Subsection 3.2.3. Application security.
- ◇ Subsection 3.2.4. Cross-browser compatibility.
- ◇ Subsection 3.2.5. Internationalization and localization.

All the other sections that are not mentioned above were completed in full cooperation of both authors and cannot be perceived as a work of individual.

2. Problem analysis

2.1. Methodology of work

As the objectives of the project have been specified, we had to think of how to proceed with the project development, so that our work is efficient and targeted at constant improvement. This is one of the key issues, mostly related to the productive management and maintaining a proper communication within a team. One must ensure that all the team members understand the goals, know how to meet them and take care of the deadlines. Thus, the development stage of our project was preceded by a structured planning process, including technology selection, assessment of the costs, analysis of the timeline, research of the current e-learning market and specification of each team member's responsibilities. All the major steps of this process are presented on the CampusNet development graph (Fig. 2.1).

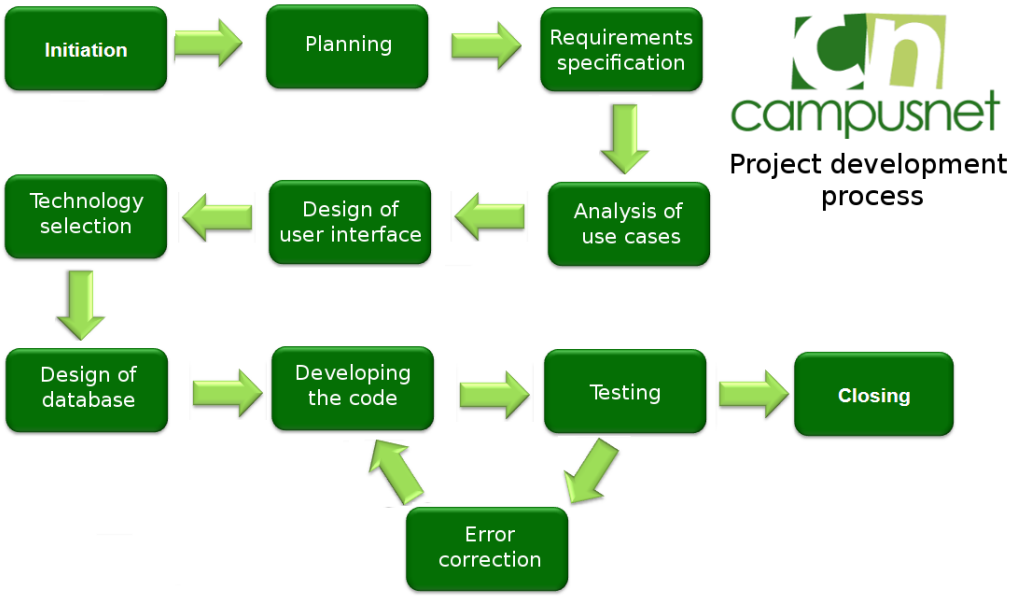


Figure 2.1. *CampusNet project development process.*

There are many applications available that may speed-up and facilitate the efficient management, thus we decided to use some of them. There were three tools that have proven to be the most helpful. These are:

- Dropbox,
- Mercurial,
- Trac.

Dropbox is a free service that significantly simplifies the file exchange. From the perspective of the end-user, it is just seen as a directory, but every time the files are copied there, it automatically synchronizes with the online version of the folder. Its more detailed description may be found in section 2.4.

Mercurial is a Source Control Management tool that greatly improves the way one can share the code in a team. It takes care of the revisions, proper merging of files and maintains the structural organization. It may be integrated with Trac, which is a Project Management Software. It allows us to manage the project in an intuitive fashion, check the history of changes, create tickets and milestones and easily access the repository's code or verify the previous revisions. The detailed description of Mercurial and Trac is presented in section 2.4.

From the methodological point of view, Dropbox has helped us with the quick exchange of documents, concept arts, images and learning materials. Mercurial provided a solid base for the code exchange, error tracking and simultaneous development of a single project, whereas Trac took care of project management in general. Having used aforementioned tools intensively, we fully believe that project development without them would be much more challenging and time-consuming.

To facilitate specification of the objectives and ensure that the team members share the common vision of project design, we decided that the proper software engineering approach would prove to be necessary. To visualize our efforts, we created a graph of consecutive steps in our project's development (Fig. 2.1).

As our bachelor project is a web application, good design of an intuitive, neat and easy-to-use graphical interface is an essential matter. Prototyping the aforementioned, one has to always ask oneself "what does the user want to do?". There is no better choice than to actually receive the answers from the user, filter and analyze them and then build an implementation upon them.

Keeping that in mind, we have visited one high school in Skierniewice, Poland, to talk about the desired functionalities, requirements and expectations. Both the school's administrative staff and the students community were pleased with what we would be able to offer them and gave us a lot of feedback about the expected implementation. More information on this topic is provided in section 2.2.

Having this information sorted out, it was easier to introduce an effective, yet efficient layout with functional arrangement of buttons, boxes, menus, etc. But before the actual coding, we decided to create a complete specification of the requirements, objectives and a set of use case diagrams to visualize our analysis of the problem along with the feedback we received from the high school teachers, students and the administrative staff.

2.2. System overview

2.2.1. Requirements analysis

There exist enormous number of mistakes that can be made during project's realization. Requirement analysis, which is considered to be the basis of each software application, seems to be an area particularly exposed to error's commitment. That is the main reason why it should be treated with extraordinary attention. Turning the words into action, we discussed and determined some assumptions

regarding implementation of aforementioned analysis. The following extract lists the most important conclusions:

- ◇ involvement of future users in the analysis,
- ◇ determination of clear objectives,
- ◇ introduction of use cases and prototypes,
- ◇ setting up priorities,
- ◇ clear statement of functional and non-functional requirements,
- ◇ giving the final decisions to programmers.

We believe that compliance with these rules allowed us to accomplish the requirements analysis part with decent results.

School implementation

As it was mentioned before, CampusNet is being implemented in Bolesław Prus High School in Skierniewice. This information possesses certain significance with regard to requirements analysis – not only it introduces new aspect to be taken into consideration, but also provides means to deal with the others.

It is basically impossible to satisfy user expectations without actually asking for their specification. Naturally, we decided to visit the target school and interview its community in sense of their needs and demands. Received feedback served as the starting point for determination of functional and non-functional requirements.

The team, with cooperation of institution's employees, examined also the existing infrastructure of school's network, particularly important for the Wireless Network Infrastructure module of our system. A detailed discussion on this topic lays beyond the scope of this thesis, however as a matter of form we will briefly describe the actual state of the network.

Estimated number of members of school's community oscillates around seven hundred people (+ parents). They are provided with about forty computers, that form the internal network. No workstations nor servers, one router and five switches, wireless connection (three access points) available only within limited areas of the building – these facts imply the need for radical infrastructure changes. Nonetheless, it is a matter of concern for WNI developers, while we would like to focus now on conclusions drawn from the discussion with future users.

Functional requirements

This paragraph describes the requirements resulting from the needs signalized by future users. It contains – in a form of table – the information about functionalities the system should ensure, along with the priority of their implementation and short description. Functional requirements were divided according to their priorities into three groups, each represented by a single table (Obligatory: Tab. 2.1, Recommended: Tab. 2.2, Optional: Tab. 2.3).

No	Name	Description	Note
1	Message sending	School's community members should be able to exchange messages.	Instant internal message system available for all CampusNet users.
2	Database managing	School's database should be completely editable.	Available only through administrator panel.
3	Timetable viewing	Users ought to be able to see their schedules.	One has an access to his/her personalized timetable.
4	Following course history	Teachers, students and their parents shall have a possibility to examine every past lesson.	Only teachers are given the ability to edit this information.
5	Mark and attendance registering	Daily attendances and obtained marks shall be accessible to users.	Edition allowed only to teachers.
6	Authorization system	Each user shall log in the system with his/her personal data.	Requires appropriate database entry.
7	Reports generation	Teachers should be able to generate student reports easily.	Student's marks and attendances are saved as a PDF file.

Table 2.1. *CampusNet obligatory functional requirements.*

No	Name	Description	Note
8	File exchange	Users shall be able to exchange files throughout the system.	Feature of VLE module.
9	Using e-learning module	Students should possess the possibility to enhance learning process using multimedia.	Feature of VLE module.
10	Wi-Fi management	Administrator shall be capable of managing the Wi-Fi network.	Feature of WNI module.
11	Events scheduling	Teachers should be able to schedule school events in the calendar.	Students and their parents are allowed only to browse the calendar.
12	Checking subject description	Brief description of given subject ought to be available to every user.	Only teachers can edit this information.
13	Publishing posts on subject board	Students and teachers should have the possibility to exchange subject-related information.	Posts are seen only by course participants (including the teacher).
14	Sending event notifications	Users shall be notified about upcoming important events.	Notifications shall be send only to people interested in given event.

Table 2.2. *CampusNet recommended functional requirements.*

No	Name	Description	Note
15	Viewing people profiles	Every user ought to be capable of seeing basic information about other community members.	Presented data depends on the type of user.
16	Settings adjustment	Users should be able to personalize their account.	All users may change their passwords or upload profile photo.
17	Browsing lists of participants	Course participants shall have the chance to find easily their fellows.	Available only to people participating in given course.
18	Sending SMS notifications	Users ought to be alerted in case of urgent incidents.	Costs generated by this feature shall be supervised by an administrator.
19	Timetable generation	School should receive support in the process of creation of timetables.	This process should facilitate the manual approach to timetable organization.

Table 2.3. *CampusNet optional functional requirements.*

Non-functional requirements

This paragraph describes the second type of requirements (Tab. 2.4), often unjustifiably depreciated by developers. We will attempt to assess these requirements with reference to the FURPS (Functionality, Usability, Reliability, Performance, Supportability) model developed at Hewlett-Packard[1]. As functional requirements were mentioned in the previous section, we will focus on the remaining ones.

No	Name	Description	Note
1	Usability	The platform should provide an intuitive and accessible interface.	Access to the most heavily used features should be as direct as possible.
2	Reliability	The system should be available every day for at least 16 hours.	Working hours depend on the school, preferably from 7 am till 11 pm, which results in 67% availability.
3	Performance	The platform ought to be capable to service the entire school community (about 700 people + parents).	The average response delayed should be maintained below 1ms.
4	Supportability	The platform should be runnable on different types of devices.	It shall support each type of device equipped with a web browser, including mobile phones, PDAs, iPads, etcetera.

Table 2.4. *CampusNet non-functional requirements.*

All tables presented within this chapter describe the requirements in general. More detailed discussion will appear along with use cases in the further sections of this thesis.

[1] See: `http://en.wikipedia.org/wiki/FURPS`

2.2.2. Use cases

One of the most important aspect in data modeling was the identification and structural recognition of people related to the school institution, later referred to as actors. We have divided them into four groups, namely administrators (even though it is mostly expected to be a secretary or a principal's assistant), teachers, parents and, obviously, students. Each actor shares some set of functionalities that might generally be useful for everyone, however there is also a particular group of features targeted strictly at the specified group.

After numerous team meetings, as well as discussions with high school teachers and students, we were able to identify their needs and present them in a form of use case diagrams. It is essential to point out that these were made at an early stage of development, but they proved to be well organized. In section 3.2, one may read how we turned those use cases into real-life applications, overcoming the obstacles and assuring an intuitive design. As VLE functionalities are out of the scope of our thesis, they have been excluded from the diagrams.

Universal set of functionalities

As the main purpose of our application is to improve the communication between various entities in school, the messaging system is of a key significance and has to be implemented for all the types of users. Diagram (Fig. 2.2) situated below this paragraph shows how the messaging may be used.

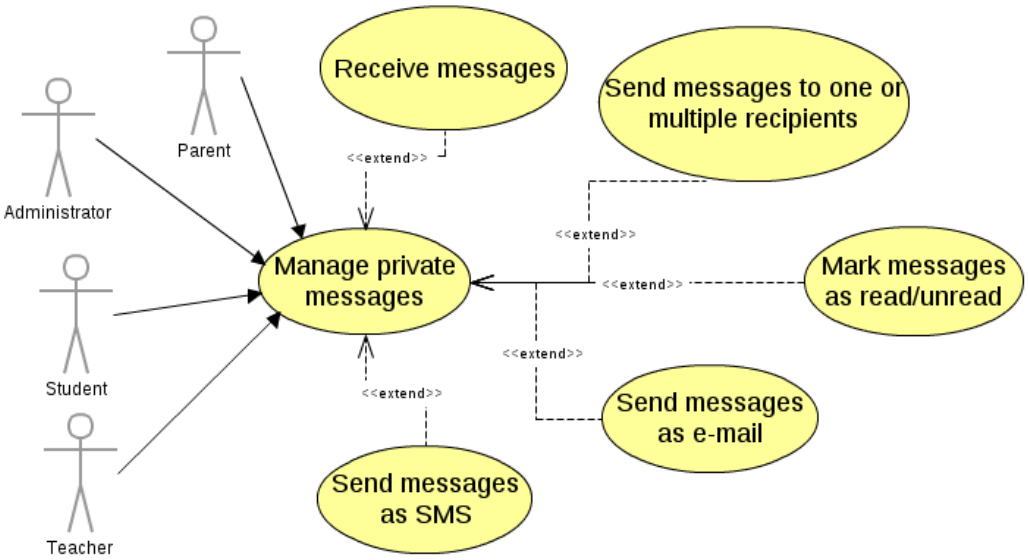


Figure 2.2. *Use cases of CampusNet messaging system.*

Considering that our system will store a significant bit of private data, we have to protect it somehow. In detail, application's security will be covered in section 3.2, but in general, it has to base on an authorization mechanism. As the use case diagram (Fig. 2.3) below shows, all the users should be able to log in, log out and maintain the session. Login function should not only authenticate the user, but also determine his type and therefore redirect him into an appropriate panel.

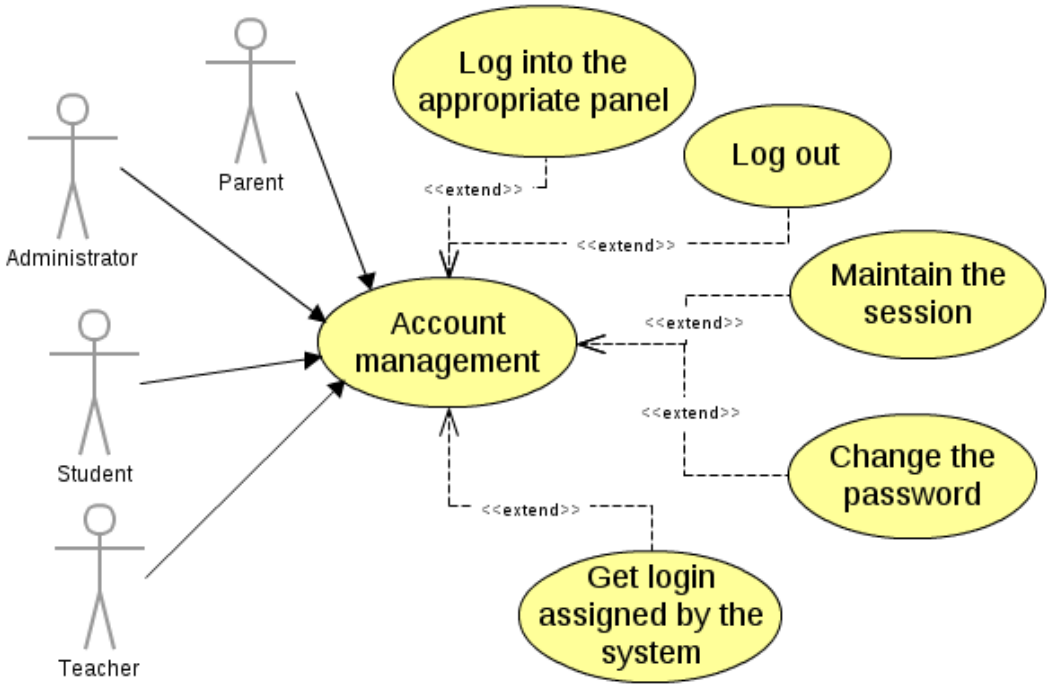


Figure 2.3. *Use cases of account management system.*

Moreover, one can distinguish between private messages and public message boards created for each and every subject. However, as the administrator is excluded from this type of information exchange, the subject's board use case will be presented consequently in the other actors' diagrams.

Administrator

The kind of system that we develop is in constant need of being managed, administered and monitored. It deals with real people and undergoes a regular flow of new information, part of which is private and should be entered only by an authorized entity. That is when the administrator comes into play. He introduces new people, subjects and classes to the database and manages their data. He also specifies the school hours and links subjects with them and with particular weekday (Fig. 2.4). The system cannot be functional unless it is provided with the initial data entered by the administrator.

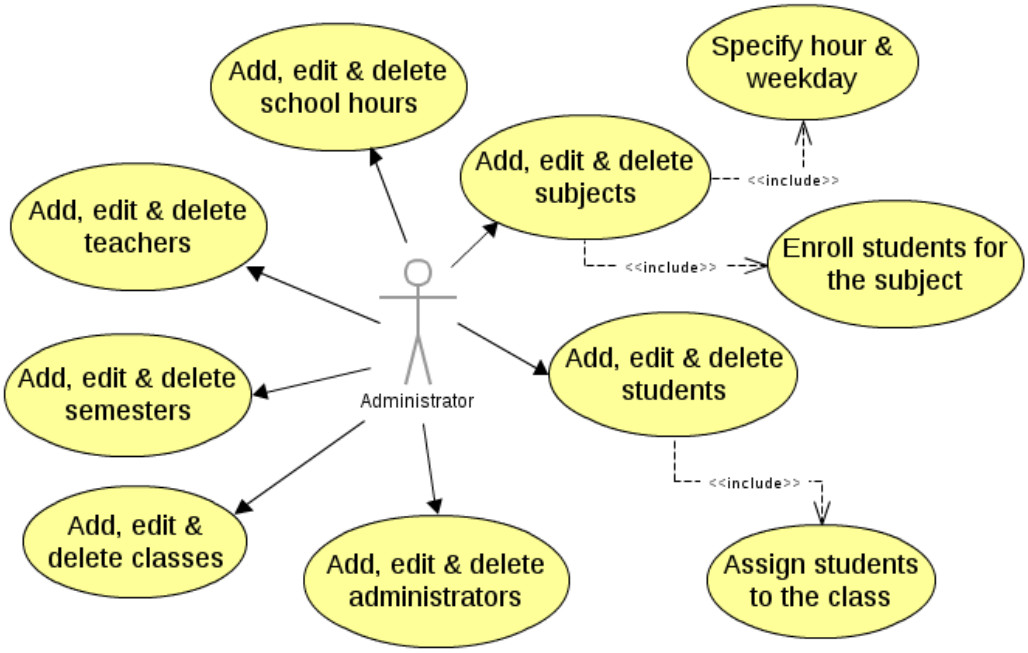


Figure 2.4. *Use case diagram for administrator.*

Student

Student is destined to be the most important and the most regular user of our system. After all, the conclusion from our research is that this type of person is in the biggest need of being provided with a better learning environment. The diagram in Fig. 2.5 shows the most essential features that students expect from LMS (again, we would like to remind that functionalities of VLE are excluded).

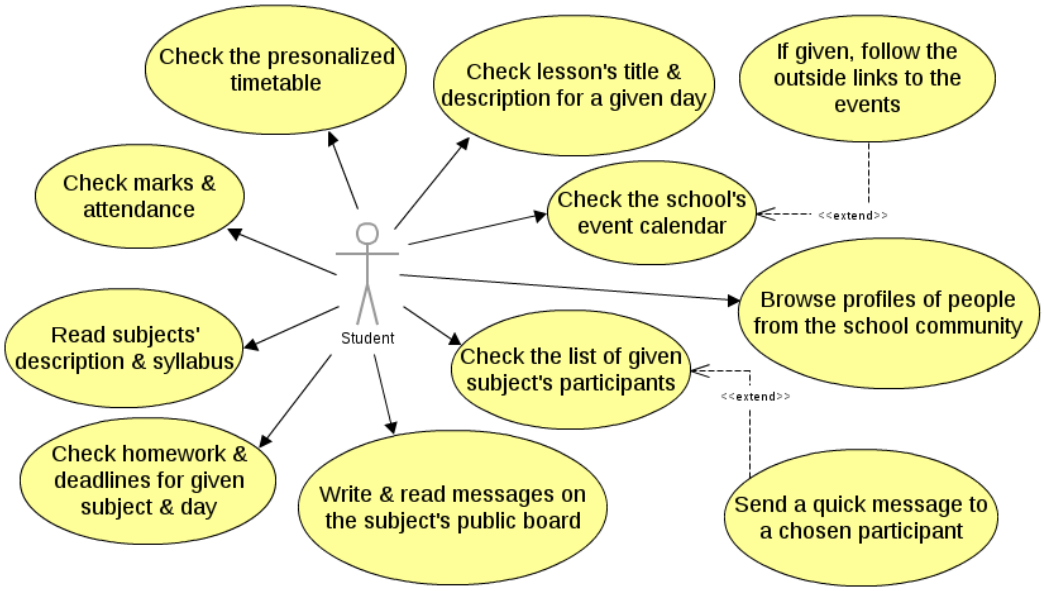


Figure 2.5. *Use case diagram for student.*

Parent

A fully functional LMS could not exist without acknowledgement of parents. This user's most essential goal is to monitor the child's progress in school and allow for easy communication with the teaching staff. Therefore, most of the functionalities designed for the parents (Fig. 2.6) are a direct resemblance of the ones aimed at students. The only difference is, parents do not see their own results, but rather their children.

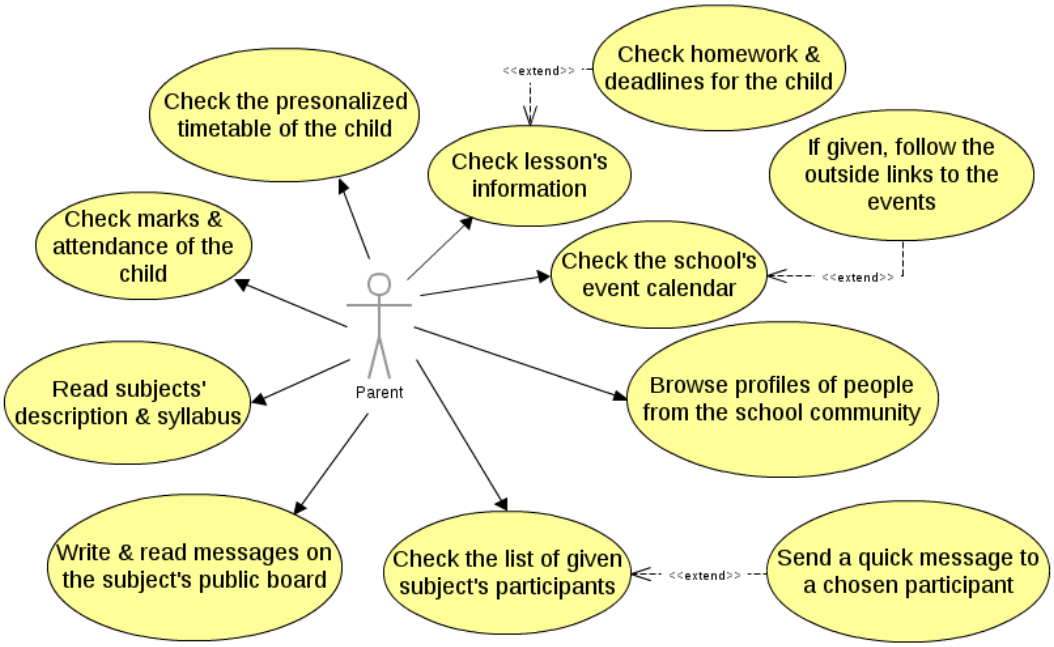


Figure 2.6. *Use case diagram for parent.*

Teacher

Teacher is the type of user that will be undoubtedly the most responsible for the substantial content and informative value of our web application. He/she is the one that fills in the information about the related subjects, creates and edits calendar events, uploads the lessons' information, fills in students' marks and checks attendance. Use case diagram below (Fig. 2.7) shows the basic options for this actor.

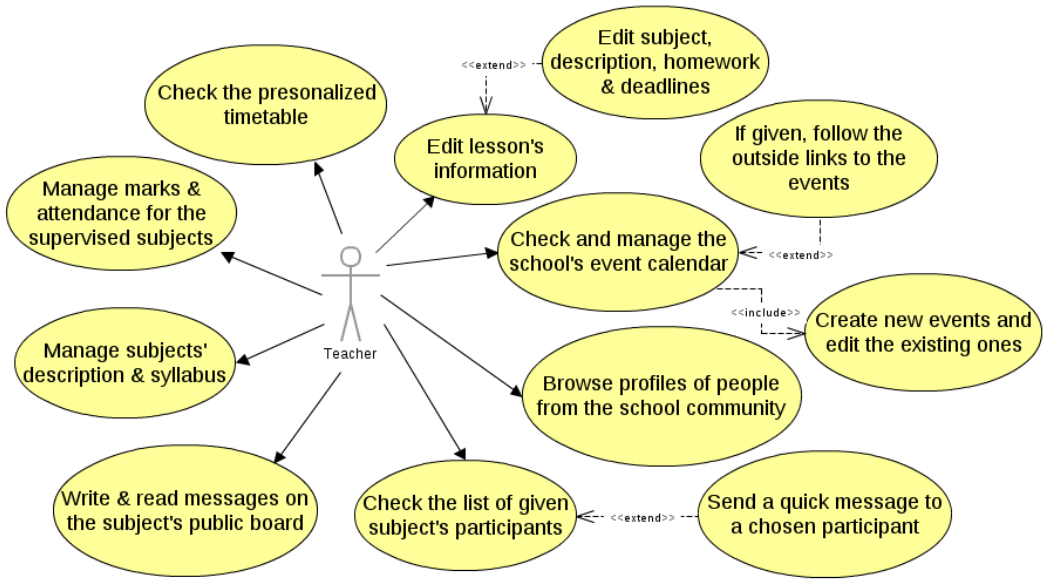


Figure 2.7. *Use case diagram for teacher.*

2.3. Similar systems

The examination, comparison and study on similar solutions is regarded as an inevitable part of each software project development. Spending appropriate amount of time on such research during the stage of project design may result in a significant contribution to its further development. We believe, that thanks to the testing of other systems, we evaded committing numerous mistakes, managed to extend the set of available features, as well as spared a lot of time normally required for fixing defects resulting from design errors. Below we present descriptions of systems we find interesting, solutions that influenced us either as an inspiration or warning.

2.3.1. Moodle



Figure 2.8. *Logo of the Moodle project.*

Modular Object-Oriented Dynamic Learning Environment[2] (first version written by Martin Dougiamas), initially released in 1999 , is an e-learning software

[2] See official webpage: `http://moodle.org/`

platform, one of the biggest initiatives in its category across the world. Extraordinary statistics prove the magnitude of the project: as of 22nd May 2011, the platform was used in 212 countries on more than 54 thousands registered sites (with USA, Spain, Brazil, UK and Germany leading the list). About 4,5 millions courses followed by nearly 43 millions users – all these numbers are truly respectable[3]. The project, created with use of such technologies as Apache, PHP, MySQL/PostgreSQL, provides course management system accessible through a web browser. The current version, which was released in the end of 2010, is labeled as 2.0. It is a cross-platform design, working on Linux, Unix, FreeBSD, MS Windows, Mac OS and NetWare operating systems.

One important remark: even though Moodle statistics seem to be overwhelming, we do not consider it as a point of reference. This software platform is destined mainly for universities, while our happens to be optimized for high schools. Nonetheless, it heavily influenced the domain of e-learning, introducing some functionalities similar to those present in our design. Thus we believe, that Moodle deserves at least a brief note in our thesis.

The platform offers wide variety of features, including discussion forum, file downloading, grading, instant messages, on-line calendar, news and announcements. Moreover, there exists a possibility to extend the system by creating plugins, that introduce new functionalities, e.g. authentication methods, enrollment methods, quizzes, etcetera.

Even though from the capabilities and popularity point of view Moodle may and should be perceived as an inspiration, it possesses its disadvantages. Those are exposed especially when it comes to visual aspect and intuitiveness of the platform. Its graphics seem to be old-fashioned (Fig. 2.9) or even ugly (Fig. 2.10), depending on certain implementation. Even if this fact has no impact on the utility of the system, it surely influences the comfort of use. Moreover, when the system is presented to a potential user, GUI[4] is the thing that determines one's first impression and may become a decisive factor when a choice from among several platforms is to be made. Besides the presentation aspect, Moodle is definitely lacking in intuitiveness. From the personal experience we have learned, that the path leading to desired functionality may require as much as several mouse clicks - way too many to describe Moodle as user-friendly.

Finally, we would like to mention the availability of the platform. Moodle remains under constant development as an open source project (under the terms of GPL[5] v2), which allow users to modify and distribute it freely. Thanks to the aforementioned cross-platform feature and very dynamic and helpful community, it may be recommended as a useful tool supporting learning process.

[3] See official webpage: `http://moodle.org/stats/`
[4] Graphical User Interface
[5] General Public License

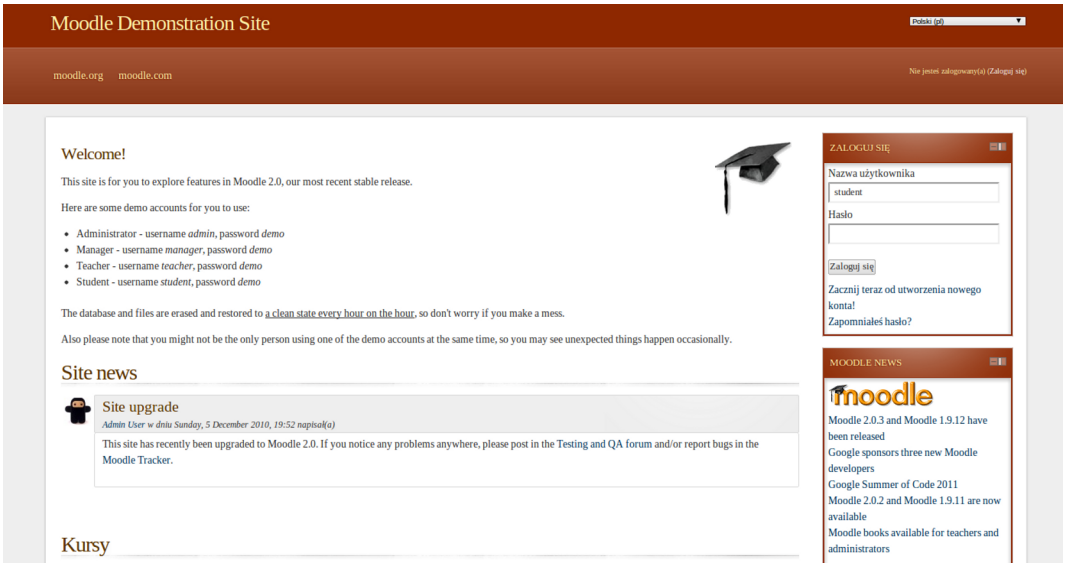


Figure 2.9. *Standard interface of Moodle-based webpage.*



Figure 2.10. *GUI of Madrid University of Technology Moodle page.*

2.3.2. SchoolTool

The second system we would like to describe is SchoolTool[6], suite of free administrative software for schools. Unlike Moodle or our CampusNet, it is not defined as Learning Management System, as the concept of e-learning was implemented only rudimentary. As states platform's official web page, the default configuration causes SchoolTool to act as a Student Information System. In other words, the system contains information related to students (attendance, marks, demographics), but does not include any learning objects or courses curriculum.

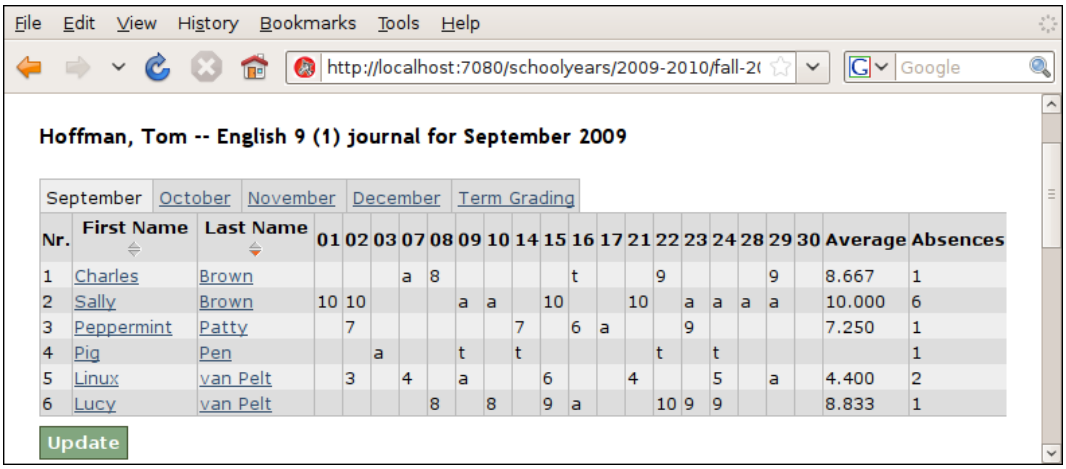
[6] See official webpage: `http://schooltool.org/`



Figure 2.11. *Logo of the SchoolTool project.*

SchoolTool, initially released in 2004, has been created with use of Zope (Z Object Publishing Environment) Object Database and Zope 3 framework. This software, written in Python, is considered to be the Python killer application, that revealed real value and potential of this programming language in the area of web applications development. The software installations run on Ubuntu packaging system; in case of non-Ubuntu environment the recommendation is to use a dedicated virtual server.

The platform provides a set of interesting features, which partially overlaps the one offered by Moodle, as well as by our design. The default set consist of functionalities supporting storage of information on demographics, calendaring, resource booking, attendance and participation journal, teacher grade book, meeting scheduling and reporting materials generation (with use of ReportLab library, the same technology we employed within e-register module).



Nr.	First Name	Last Name	01	02	03	07	08	09	10	14	15	16	17	21	22	23	24	28	29	30	Average	Absences
1	Charles	Brown				a	8					t			9				9		8.667	1
2	Sally	Brown	10	10				a	a		10			10		a	a	a	a		10.000	6
3	Peppermint	Patty		7						7		6	a			9					7.250	1
4	Pig	Pen			a			t		t					t		t					1
5	Linux	van Pelt		3		4		a			6			4			5		a		4.400	2
6	Lucy	van Pelt					8		8		9	a			10	9	9				8.833	1

Figure 2.12. *GUI the SchoolTool project.*

Let us now turn to the visual aspect of SchoolTool (Fig. 2.12). Again, it does not take advantage of any sophisticated effects, giving the impression of obsoleteness. Nonetheless, it appears that the system was meant like that – thanks to the simplicity of design it remains neat and easy to read. We are not able to describe interface intuitiveness in detail, as we did not have the opportunity to test it for a longer period of time. Still, limited number of functionalities and items in menu bars suggests easiness of use. Moreover, the platform seems to be very well documented. The SchoolTool book, a document that serves as the primary source of

user and developer documentation on the system, is organized in very transparent way, while its intelligible language encourages new user to wade through it.

Last but not least, the availability of the system. SchoolTool has been built on completely open source software stack (Ubuntu, Zope, Python), and is licensed under GPL version 2. The platform remains actively developed, mainly by teams from USA and Lithuania with fundings from Mark Shuttleworth (South African entrepreneur heavily involved in commercial support of open source projects, including e.g. Ubuntu). Other locally funded developers are responsible for additional component's contribution. Finally, the internationalization of SchoolTool is provided by volunteers from all around the world.

2.3.3. Commercial systems

In the following paragraph we would like to briefly present two systems available on Polish market. Solutions proposed by Librus and Prymus.info are very interesting from the point of view of this project and may impact its future development. Unfortunately, technical specifications of these platforms are not available to general public, as they are not open source projects. Consequently, we are going to focus on available functionalities, as well as the visual and financial aspects.

Librus



Figure 2.13. *Logo of the Librus project.*

Librus[7] is in fact a publishing house specialized in providing software for educational institutions. The company is responsible for development of two projects – AMS (Attendance Monitoring System) and SEP (School Educational Platform). In fact, these projects may be considered as equivalents for CampusNet modules – LMS and VLE, respectively. While in case of our project such division results from the fact, that the modules are subjects of different theses, Librus uses it as a marketing gimmick. The company advertises their own SEP as "the first platform that became completely integrated with the Attendance Monitoring System". Moreover, they underline the fact, that the AMS is "the first e-register that became completely integrated with the School Educational Platform". Our team would like to express deep admiration and congratulate on successful merge of two modules developed by the same company. In addition, we would like to hype our LMS as the first module integrated with VLE, and the other way round.

Nonetheless, Librus' AMS and SEP offer some interesting functionalities, e.g. possibility of scanning manually filled-in evaluation sheets or generation of a wide

[7] See official webpage: `http://www.librus.pl/`

variety of statistics, helpful in the end of each semester. In addition, its GUI represents a decent quality and the e-learning part contains plenty of useful materials. In spite of relatively high cost (2000-2500 PLN for 2-years basic license, lots of additional fees for premium features), Librus' software is probably the most common platform used in capital high schools.

Prymus

Figure 2.14. *Logo of the Prymus.info project.*

Prymus.info[8] is the last system that we would like to mention in this chapter. It is a relatively new project, that arose during last year. Similarly to Librus, Prymus developers do not present any technical information on their project. However, on the basis of its functionalities, it may be considered as the equivalent for LMS module of CampusNet, as it does not introduce any features supporting learning process. Nonetheless, provided set of e-register functions looks promising, as it contains, inter alia, automatic printout of school reports, neat and transparent message system or various statistics generation. The platform may be also an example to follow in terms of GUI (Fig. 2.15) – visible buttons, enjoyable graphics and intuitive interface. Another thing to consider is the price for using it – unfortunately, the company does not state the cost on its website, encouraging potential users to contact them personally.

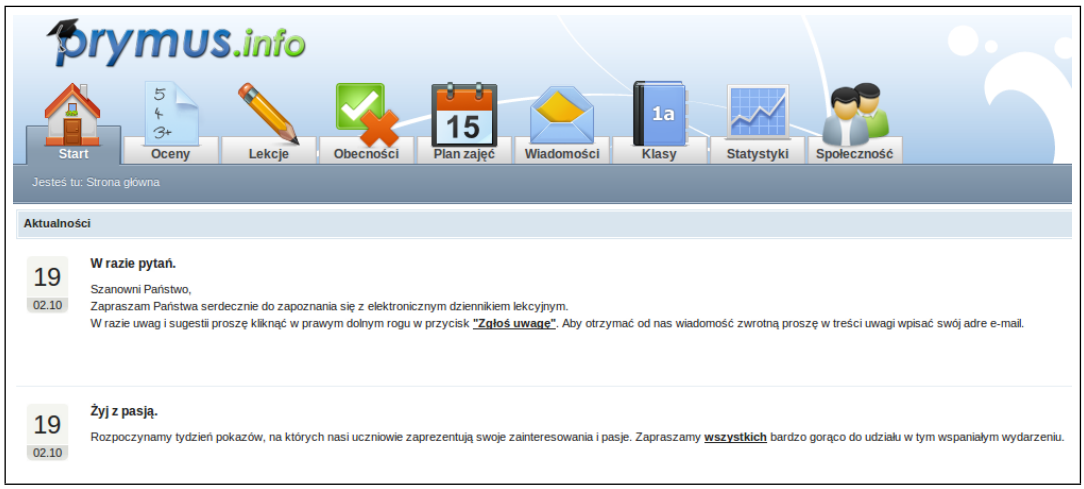


Figure 2.15. *Prymus.info GUI example.*

[8] See official webpage: `http://prymus.info/`

2.4. Technology selection

CampusNet in its whole needs a complex set of technologies to support it. In this section, one can find all the key information about our process of technology selection for the Learning Management System module. The analysis would not be complete without mentioning the alternative solutions to our problems, however the technologies we actually selected are the ones, which we describe in the most detailed way. Also, we decided to include few code snippets for several reasons, the most important of them being listed below:

- ◇ Code examples show the most important features of the selected technologies.
- ◇ They also present the way they were used in our project.
- ◇ Reader is more aware of what the selected tools can do and how they work.

2.4.1. Data Base Management System

MySQL RDBMS[9]

The choice of a Database Management System was one of the first decision made regarding our project. Even tough the list of DBM systems contains dozens of positions, we did not have many doubts while making the selection, thanks to the establishment of some certain criteria that ought to be satisfied, i.e.:

- ◇ relational model of database,
- ◇ free software license,
- ◇ active development,
- ◇ cross-platform,
- ◇ supported by Django web framework.

The requirements above significantly limited our possibilities, leading us to the final selection of MySQL Relational DBMS available under GPL (version 2). We consider this choice to be the one that provides the satisfying engine in terms of speed and reliability, along with increasing set of useful features. Team's familiarization with this system also stood as an important factor.

MySQL RDBMS by default ships without any graphical user interface allowing to manage it, hence the user has to use the command line to communicate with the software. While this works perfectly for the purpose of making queries or creating tables, we needed an additional front end tool to visualize the database.

The issue of visualization was solved by MySQL Workbench, an official graphical administration application. While it provides several interesting features, including SQL Editor tool or database administration tool, we were mostly using the ER diagramming and drag'n'drop visual modeling, which resulted in database scheme. An example of such scheme (in fact, an early-stage design of CampusNet database) may be found in Appendix E).

Django integration

Another important factor supporting the choice of MySQL RDBMS was the easiness of integration with Django web framework. Having completed the installation, it was just enough to execute the following commands:

[9] Relational Database Management System

```
1 mysql > CREATE DATABASE campusnet CHARACTER SET utf8;
2 python manage.py syncdb
```

Listing 2.1. *Creating MySQL CampusNet database*

...along with proper entries in settings.py file to set up a working database for our project. Moreover, thanks to the object-relational mapper implemented in the framework, we were able to define data models (each model maps to a single table in database) entirely in Python and use the dynamic database-access API (application programming interface) instead of standard SQL. Nonetheless, in case of need SQL may be used as well. More about Django framework, its features and importance to our design will be presented in the server-side technologies paragraph.

2.4.2. Server-side technologies

First and foremost, it is essential to clarify the terms used in this paper. Our definition of the server refers to the computer where the project is deployed at and run. The term "client" refers to the browser that loads and displays the WWW content. Technologies used for dynamic generation of the HTML web pages have an essential value for functioning of the web application. There are numerous possibilities to choose from and several key issues related to technology selection. First of all, it has to provide the channel for communication with the database, ergo it must be compatible with the DBMS chosen in the previous subsection. Secondly, it must meet the basic security needs. Last but not least, it has to be efficient both in terms of performance and for the sake of clean and fast coding. As for the beginning phase of the project, none of us has been proficient in server-side programming, thus another important factor was the ease of learning.

There are plenty technologies available on the market – open source as well as commercial – that would meet our needs for this project. For the obvious reasons, we decided to consider only the ones that we could use for free. Therefore, we quickly excluded Microsoft Active Server Pages, ASP.NET and Oracle Application Express from the group of our back-end technology candidates. Having done the thorough research, we concluded that one of the technologies mentioned below would prove the most useful to us:

- J2EE, including Java Servlets, Java Server Pages (JSP), Java Beans and Hibernate framework for persistent database connection,
- Ruby on Rails,
- Python programming language with Django web framework,
- Perl,
- PHP.

Throughout this project two main trends could be observed. The first period of development was devoted solely to J2EE solutions. As the new problems emerged and number of reported issues grew, we had to re-think our approach and finally decided to switch our interests to Python and Django. One could write more than a couple of books about every of the aforementioned technologies, thus in the following paragraphs, we will only focus on describing the most basic and essential features, revealing the reasons of our technology selection.

Moreover, except for the language, frameworks and libraries, we also needed to choose the application server, that we would deploy our project on. However, the

options were quite straightforward – J2EE solutions work well with Apache Tomcat, whereas PHP, Perl, Ruby on Rails and Django applications can be installed by introducing additional modules to Apache HTTP Server. That way or another, one can see that Apache is the undisputed leader in the field of open source application servers.

Perl

Perl, which is a shorter version of "Practical Extraction and Report Language" is a scripting language that was one of the first important technologies on the market, enhancing quick development of Web applications. With its flexible syntax, it enables the developer to write a swift and short yet effective code. Perl's syntax is also greatly optimized to increase performance in string processing, especially in applications that need to process large amounts of text. It also contains a rich base of additional plug-ins and libraries.

As for every programming language, Perl has some disadvantages as well. One of the main issues that the developers mention is the complexity of the code followed by an overload of features built into the core of the language. Some functions would be a lot more suitable, if put into separate modules or libraries. Considering that in order to develop our system, we would have to learn this language from the start, these complications could greatly diminish our efficiency.

PHP

PHP is a recursive acronym for Hypertext Preprocessor and it is the most widely used open source scripting language, adjusted to suit the needs of web applications' development. It is mainly used inside the HTML files, being nested inside special tags. The syntax is relatively clear and simple, basing highly on C and Java languages, which makes it easy to learn and analyze. However, despite the fact that one would not take much time learning the basics, using its more sophisticated features might result in being problematic. Also, the more users there are, the worse is the efficiency of PHP-based applications, as the scripts are reinterpreted every time the user requests a web page. Being future-oriented, our team decided that there are more innovative and efficient technologies, the knowledge of which will greatly benefit in the future.

Ruby on Rails

Ruby on Rails (also abbreviated as RoR) is an open source framework for quick development of web applications, written in Ruby language. Its main goal is to make the coding quick, easy and comfortable, especially by introducing the rule "convention over configuration", which states that configuring the application should be as simple as possible and the default, recommended patterns shall replace (or at least diminish the meaning of) the large variety of options and settings.

This is a relatively new technology and it still gains more and more admirers. Twitter – one of the most popular websites on the Internet – is powered by RoR. Despite this fact, we decided to postpone learning of this framework, as we had no experience whatsoever with the Ruby programming language.

Transition from J2EE to Python with Django

In the introduction to this section, it was mentioned that J2EE was our main choice during the beginning of development. It seemed to cover all the necessary functions that we needed for creating the Learning Management System. The general development schema looked as follows:

- ◇ Design the data structure and implement it in MySQL server.
- ◇ Use Hibernate for Object-relational mapping (ORM) and develop helper classes for persistent communication with the database.
- ◇ Nest JSP code into HTML pages for presenting the dynamically generated content.
- ◇ From JSP perspective, call the Hibernate functions for database management.

This approach has been sufficient for a sustainable period of time. Nevertheless, crucial problems arose as the application development was in a more and more advanced stage. We realized that even though it would be possible to create our system using previously chosen means, it will neither be quick nor straightforward process. Some of the issues related to J2EE version of our system are listed below:

- ◇ As our knowledge of Java Server Pages expanded, we realized it is out-dated and fairly low-level. There have been no recent changes and improvements to this technology.
- ◇ Configuration of MySQL/Java connector and management of the connection pool is problematic, so that maintaining a constant, uninterrupted connection to the database is a challenging process.
- ◇ Authorization, authentication and session management was found to be inconsistent, unintuitive and hard to maintain.
- ◇ JSP and Hibernate are not well documented whatsoever, what greatly impairs the learning process. Thus, getting help about the proper usage of aforementioned technologies along with the design conventions was basically impossible.
- ◇ Due to JSP being nested into the HTML pages, the presentation layer is mixed up with the application's logic.
- ◇ Queries written in Hibernate Query Language (HQL) may become really complex.
- ◇ With time, the code in general was getting less and less legible.

During that time, working on other projects, we also gained some experience in Python programming language and its additional set of modules, including Django. We realized this framework is much more efficient and coder-friendly. It was not an easy nor hasty decision to make the transition, but after detailed discussion and analysis of all the advantages and disadvantages, we decided to redesign the whole application and switch the technology we build on. There were several reasons to do so, namely:

- ◇ Django is a relatively new framework, but it has been rapidly gaining fans since its earliest release. It is considered to be a very future-oriented technology, also considering the job trends and need for efficient web developers.
- ◇ Integration with MySQL is a lot simpler than in J2EE and it allows for extremely flexible management of the data models.

- ◇ Both Django and the Python itself were found to be expertly documented, what substantially improves the learning process.
- ◇ Django incredibly facilitates the authorization process, session management and site administration, by providing tools that are complete and ready to use.
- ◇ It is advertised as the "web framework for perfectionists with deadlines"[10] and it is hard to disagree with this catch-phrase. The syntax and application model are designed so well, that it takes only a little time to develop sophisticated functions that would become very complex in JSP with Hibernate.

General overview of Django application model

In general, Django is a high-level web framework written in Python, which follows the DRY[11] principle. This implies that it encourages a concise design and promotes a regular pattern in application development.

The structure of a project developed in Django is quite strict and solid. Generally, project is a collection of one, common configuration and multiple applications put together to support a particular Web site. There can be numerous applications comprising one project, where access to the respective parts is provided by specifying unique URL[12] addresses (addressing will be further discussed in this section). Below we will describe how the Django project may be started and developed on the example of CampusNet.

Having installed the Python interpreter and Django libraries, one can immediately start a new project. This is done with one simple instruction entered to the operating system's command line:

```
1 django-admin.py startproject campusnet
```

Listing 2.2. *Starting Django's new project.*

After this command is executed, the directory `campusnet` is created with few files added automatically:

- ◇ `__init__.py`
- ◇ `manage.py`
- ◇ `settings.py`
- ◇ `urls.py`

The description of these files will be provided in the future paragraphs, but first, let us create an application. In our project, there are several applications and among them, each user's panel is a separate entity. For the sake of this section, we will show the structure of Django application's structure on an example of the administrator's panel. We create it by entering the command:

```
1 python manage.py startapp admin_panel
```

Listing 2.3. *Creating Django's new application.*

[10] See: `http://www.djangoproject.com/`
[11] Don't Repeat Yourself
[12] Uniform Resource Locator

Immediately, inside `campusnet` another directory is created with a name `admin_panel` along with given files:

- ◇ __init__.py
- ◇ models.py
- ◇ tests.py
- ◇ views.py

Now, as we are familiar with both the structure of an app as well as the structure of project's root directory, we may move to the description of automatically generated files and their purpose.

Among other options, in `settings.py` we specify the database name, user, password and the driver class that will be responsible for communication. With just few simple lines, we can link our project with the database. File `manage.py` may be considered the project's control center. It contains numerous functions necessary for running the project on a test server, synchronizing the data, creating new application prototypes, etc. File `tests.py` – as the name may suggest – is completely devoted to the project's testing and it will be described in details in chapter 4. As for the `__init__.py`, it is an integral element of every directory, indicating that it should be considered as Python package. Nevertheless, it does never need to be edited by the developer.

Now there are 3 most important files left - `models.py`, `urls.py` and `views.py`. The first one is where we can define our models. They have a direct impact on the database structure. Here is an exemplary model taken from LMS (all the models will be described in details in section 3.1):

```
1 class PrivateData(models.Model):
2     person = models.ForeignKey(Person)
3     address = models.CharField(max_length = 255, null=True, blank=True)
4     e_mail = models.EmailField(null = True, blank=True)
5     pesel = models.CharField(max_length = 11, null = True, blank=True)
6     phone_no = models.CharField(max_length = 32, null = True, blank=True)
```

Listing 2.4. *Example of a Django model - class PrivateData from LMS*

At this point, it is sufficient to say that the class `PrivateData` inherits from a Django class `models.Model` and has several fields, which types vary from character field (of various lengths) through e-mail field (Django automatically validates if a proper e-mail address was entered) to the one indicating a foreign key to another model.

After specifying the models, we can use one of the Django's most useful features – it allows for an instant synchronization with the database, by using the following command:

```
1 python manage.py syncdb
```

Listing 2.5. *Django's database synchronization.*

This operation results in generation of SQL statements that will create the tables in the specified MySQL database. There is a simple pattern for that – table's name is of a form app-name_class-name, where classes are taken from all the `models.py`

files in all the applications comprising to our project. The attributes' names in the tables are exactly the same as the names of the variables we declare in the classes.

Having our models ready, we may move to specification of URL addresses. This is the crucial stage of development, where we can link the HTTP addresses with a particular views. First, we edit `urls.py` situated in the root directory of our project. It contains the URL patterns in a form of a list. Single entry looks as following:

```
1 (r'^admin_panel/', include('admin_panel.urls'))
```

Listing 2.6. *Exemplary entry from Django's urls.py file.*

The first parameter is a regular expression used for address matching and the second represents an action that needs to be done. What Django actually does is performing an analysis. It trims off the home address and matches what is left with all the possible URL patterns and if it finds a link, the instruction given in the second parameter is executed. This is just a redirection to another `urls.py` file. It literally says "if `http://address-of-the-project/admin_panel/` is requested, check what to do next in `admin_panel/urls.py`. Thus, we may take a look at that file now and see that it contains more URL patterns. Examples are as follows:

```
1 url(r'^$', list_students, {}, 'list-students-default'),
2 url('^teachers/add/$', add_teacher, {}, 'add-teacher'),
3 url('^help/$', direct_to_template, {'template' : 'help.html'}, 'admin-help'),
```

Listing 2.7. *Examples of various entries from urls.py file.*

First of all, we have to realize that the previous operation trimmed the requested address not only off the home address, but also left all the matched parts behind. Therefore, when we consider `admin_panel.urls`, the address string we have to match would be cut from `http://address-of-the-project/admin_panel/` to just an empty string. In regular expressions, `"^"` denotes the beginning and `"$"` stands for the end of the string. Thus, from the URL patterns given above, we can see that the one in line no 1 matches. If the home address was concatenated with `teachers/add/`, it would result in matching the pattern from line 2 and therefore calling the HTML rendering function `add_teacher`, which is defined in the file `views.py` and will be described in the future paragraphs.

Moreover, we can see the given patterns are a bit extended compared to the one seen in root directory's `urls.py`. First two parameters stayed the same – number one is regular expression and number two is the instruction to be executed in case of a matching address string. The third parameter may be thought of as a dictionary containing additional options and the fourth argument is a unique name that may be used to identify given URL, but is optional in general. Additionally, there are some built-in functions that may be useful. For example, if there is no dynamical content to be generated, we can omit the HTML rendering functions from `views.py` and directly respond with an HTML template, using function `direct_to_template` and specifying the HTML file that we want to use.

The advantage of such an addressing approach is that the links specified in this form are independent from the home address where the project is deployed.

Therefore, it does not matter where we deploy it, it will always find the right addresses and process the requests appropriately.

We have already mentioned that `views.py` contains the HTML rendering functions. From the perspective of logical and functional analysis, this is the place where the most crucial part of the code exists. After the URL pattern is matched, Django processes the request by executing the view function. Inside it, we can analyze and use the data provided in the request header, retrieve information from the database, update it, save it or delete it - basically prepare everything to be later rendered into the HTML template and shown to the user. One of the functions from the administrator panel's `views.py` is presented below:

```
1  def add_teacher(request, template="add/add_teacher.html"):
2      if not request.user.is_authenticated or not request.user.is_staff:
3          return HttpResponseRedirect("/admin_panel/login/")
4      try:
5          logged_user = Person.objects.get(user = request.user)
6          logged_user = logged_user.first_name + " " + logged_user.last_name
7      except:
8          logged_user = "Unknown"
9
10     context = {
11         "logged_user" : logged_user,
12         "add_link" : "/admin_panel/teachers/add/"
13     }
14     return render_to_response(template, context,
15         context_instance=RequestContext(request))
```

Listing 2.8. *Exemplary function from Django's views.py file.*

The word `def` is a python's equivalent for a function. In lines 2 and 3, we see how easy it is to handle the user's authorization in Django. The user is associated with the session and therefore, with the HTTP request. If he/she is not authenticated or is not a member of the staff (in our database model, only administrators are represented as the staff members), the website will be redirected to show the login page. However, if the administrator is already logged in, it will retrieve the related `Person model`, saving the first and last name in a string (lines 5-6). It may happen that accessing the database will fail and raise an exception. That is why we may wrap the code in the try-except block and handle all the possible errors.

The most essential part comes at the end of the function. There are two variables that control the content and looks of the web page, namely the context and the template. Context is a dictionary - a list of key - value pairs, where a name string is considered the key and value may be any variable. Keys specify the names by which given variable can be accessed in a template, created from an HTML file, which name is hold in the variable template. Having those structures specified, we can call the function `render_to_response` (line 14), which will send an HTTP response to the user, containing the HTML with dynamical data (submitted in the context) rendered inside it.

There is an independent set of tags devoted only to Django templates. This approach has been chosen to separate the syntax of the control layer from the syntax of the presentation layer. In general, the template looks like a typical HTML

page, except for two types of tags presented below:

```
1 <div id="user_info">
2     {%trans "Logged in as"%}: <br>
3     {%if logged_user %}
4     {{logged_user}}
5     {% endif %}
6     <input type="button" id="logout_btn" value="{%trans "Log out"%}"/>
7 </div>
```

Listing 2.9. *Example of Django's template file.*

The first type of Django template tag is shown e.g. in lines 2 and 3. Curly brace and percent sign denote an instruction or a logical operation. A variable (exactly the same as it was specified in view's context) inside double braces will be converted to a string format and pasted into the rendered HTML. Function `trans` seen in lines 2 and 6 is from the internationalization and localization module that will be covered in section 3.2.

All of the mechanisms described in this section comprise one complete, structured and yet flexible architecture. Not only this solution facilitates the development process, but also creates opportunities for constant improvement and highly increases the readability of the code. Above all, it introduces desired independence of the related layers. We may change the URL structure of our project without changing the views. The same applies to the HTML templates - we can change them (without any need to rewrite the rest of the project) or even implement many different designs and let the user choose how he/she wants to have the data presented. It is all consistent with the Model-View-Controller (MVC) architectural pattern, even though Django creators find it more suitable to call their architecture Model-Template-View (MTV).

2.4.3. Client-side technologies

Along with the rapid development and constant expansion of the Internet and technologies associated with it, comes the immediate progress in web browsers' advancement. Nowadays, a browser is much more than just a tool for interpreting and visualizing HTML documents. With huge amount of add-ons, plug-ins and built-in functionalities, it has become a platform for listening to music, watching videos, playing games and – most importantly for our case – a base for usage of interactive web applications with advanced and flexible interface. Thanks to the sophisticated design and client-side technologies, today's web applications may offer a level of interactivity that would have never been possible with typical static site interface.

We could completely omit the client-side scripting without compromising the logic, but in order to build an efficient application with intuitive and easy user interface, responding to various events, it is obligatory to perform some operations without involvement of the server. That is why we decided to enhance our code with the wide variety of functionalities offered by JavaScript (abbreviated as JS) and its related libraries and plug-ins. In our project, the most common goals that we can achieve with its use are:

- ◇ Tweaking the visual aspect of the project by introducing color variations, animations, transparency control, etc.
- ◇ Handling all sorts of events bound to the whole web pages or their particular elements.
- ◇ Updating some part of the page content without the need to reload the whole website.
- ◇ Client-side form validation.

JavaScript

It is a lightweight object-oriented scripting language created by Netscape and Sun Microsystems for developing Internet applications that allows enriching the plain HTML web page with additional functionalities. One of its main advantages is that there are no external libraries or programs that need to be installed in order to have it working. Nowadays, all the leading web browsers are able to interpret it and execute its instructions. Because JavaScript code can run locally in a user's browser (rather than on a remote server), the browser can respond to user actions quickly, making an application more responsive. Its syntax is similar to the one we got used to in C programming, but it also has adapted some naming conventions from Java. Nevertheless, it should never be mistaken with that programming language.

JavaScript functions may consist just of standard operations and working on variables, as in every programming language, but usually its primary use – and the most important feature from our perspective - is to develop functions that are either embedded in or included from HTML pages and perform manipulations in the DOM structure. DOM stands for Document Object Model and it is a cross-platform, language independent convention for representing and interacting with objects in HTML or XML documents. Through this structure, JavaScript functions have access to particular elements in the page document's tree.

Our point is not to present a detailed description of the language itself, as there are many books and research papers devoted to that particular subject, but we would rather present the features that have proven the most useful for developing our project. In reality, there are very few functions in our web application that use just the plain JavaScript. For most of the time, it has been supported by a powerful and concise library that significantly speeds up the development process. Its name is jQuery.

jQuery

Plain JavaScript may be fully functional, but is fairly low-level and needs many lines of code even for relatively simple operations. jQuery is advertised as "write less, do more"[13] library and there is a good reason for that. It significantly reduces the complexity and simplifies the way one develops the JS scripts. It is extensively used throughout our whole project. In the next paragraph some of its most important features will be discussed.

First of all, let us specify the ways one can include JS scripts in the HTML document. Sample code shown below presents a typical structure of the HTML document with jQuery script included in the header.

[13] See: `http://jquery.com/`

```
1  <html>
2    <head>
3      <title>Title</title>
4      <script type="text/javascript" src="path-to-jquery/jquery.js"></script>
5      <script type="text/javasript">
6        $(function (){
7          // put your code here
8        });
9      </script>
10   </head>
11   <body> page content </body>
12 </html>
```

Listing 2.10. *Exemplary use of JS script in the HTML document.*

As it was mentioned before, the JavaScript code may be either included from a web page, or embedded in it. Both of these approaches are presented above. In line 4, we include the external script, which actually is the jQuery library and then, in line 5 we define our own script. The code in line 6 may be interpreted as "just after the whole web page is loaded, execute the code written between the curly braces". An extended but more intuitive version of this can be written as:

```
1 $(document).ready(function() { ... });
```

Listing 2.11. *Exemplary use of $(document).ready() function.*

Now all that is left is the content of the function.

Before we can proceed to manipulating any of the DOM elements, we have to specify which one of them we want to edit in particular. We can do this with help of the jQuery selectors. There are many possible options to single out an element and they are well described in the jQuery documentation, but the most important ones are listed below:

- ◇ Selection by id - $("#some-id")
- ◇ Selection by class - $(".some-class")
- ◇ Selection by element's type - $("elemtype")

It is up to the HTML developer to provide some ids or classes to the key elements that are expected to be accessed by the JS. As the id should be unique, the related selector always returns at most one element. On the other hand, there may be many elements of the same class or type and in this case, selector will be given a list of objects and then there may be some functions bound to them and they will be executed for each and every element.

One of the most common and important features of JS with jQuery is an easy binding of event handlers to the DOM elements. The syntax is really intuitive (also much shorter than in plain JS) and with few simple lines we may introduce a higher level of interactivity to our applications. The example below is extracted from `static/js/main.js` script, which is a set of common functions for the parent, student and teacher panels.

```
1  $(document).ready(function() {
2      $(".prevent_link").click(function(event) {
3          event.preventDefault();
4          $(this).next().slideToggle();
5      });
6
7      $("a").click(function(event) {
8          if($(this).attr("id") != "out" &&
9       $(this).attr("class") != "prevent_link") {
10             event.preventDefault();
11             reloadAddr($(this).attr("href"));
12         }
13     });
14 });
```

Listing 2.12. *Example of binding of even handlers.*

It is a neat example showing how the selectors work (selection by class, by type and by the next sibling is presented), how easy it is to bind web page elements with event handlers and how to apply simple animations on them. The first event handler is presented in lines 2 – 5. If the user clicks an element of class `prevent_link`, then the anonymous function will be called with an event parameter. Then this variable is used to prevent the default behavior (e.g. reloading the page) of the element and instead, animate its next sibling by sliding it up or down. This is actually used in CampusNet panels for customizing the subject menu. When the user clicks on a subject link, the related sub-menu will immediately collapse or unfold, depending on its previous state.

An interesting piece of generic code is written in lines 7 – 13. Whenever any link (HTML tag `a`) is clicked in any of the panels, the script checks if its id is not `out` (redirection to an external website) or its class' name is not `prevent_link`. If both of the above are true, it prevents the default behavior of the web browser, which would be to reload the whole page and go to the address specified in the link's href attribute. Instead, it reloads just one part of the page by calling the function `reloadAddr` with the parameter being the address from which it gets the updated content. Another useful function was used here – calling the `attr` function, we can extract the value of any attribute of any element.

Obviously, there are numerous events that may be caught and processed, but this chapter's goal is just introducing the reader to the technologies used and present their general use in the application. More about jQuery event handlers and animations can be read in this library's documentation [14].

AJAX

AJAX, an abbreviation for Asynchronous JavaScript and XML, is a group of technologies and concepts for developing web applications, in which the communication between the user and the server does not necessarily involve reloading the whole web page. This allows for a more dynamical interaction between the client and the server. Moreover, as it may be asynchronous, the user can still perform some operations and use the application in the current state, while the data transfer from and to the remote machine happens independently in the background.

There are few components that comprise the whole group of AJAX technologies:

[14] jQuery documentation available at `http://docs.jquery.com/Main_Page`

- ◇ HTML, CSS and DOM – for selection of the HTML elements that are bound to visualize the changes that happen after the AJAX response is acquired and their presentation.
- ◇ XML as the default format for the data exchange. Its use is not mandatory - it may be replaced by JSON objects or even plain text.
- ◇ JSON (JavaScript Object Notation), being another supported standard for information interchange, that is mostly used for serializing and transmitting structured data over the Internet.
- ◇ XMLHttpRequest – it is a class designed for the web browsers' supported scripting languages (i.e. JavaScript), that allows for sending the requests to the WWW server with use of HTTP protocol, even after the web page is fully loaded.
- ◇ JavaScript – used for the integration of the aforementioned elements.

With use of the above, we are able to build innovative websites with a level of interaction that is still rare in the World Wide Web. Since AJAX model was thoroughly described and documented, some world-leading companies acknowledged it and have expressed their interests in its usage. Nowadays, Google and Facebook are the most famous corporations that use AJAX throughout vast majority of their applications.

Even though the whole concept of asynchronous data exchange may seem complicated, there are some high-level libraries that make the use of AJAX clear and easy to understand. One of them is jQuery, therefore we decided to use it to our advantage.

Throughout our whole project, there are many instances of AJAX implementation using JavaScript with jQuery. As one of the simplest examples, we may consider the login mechanism for the CampusNet administrator panel:

```
1  $(document).ready(function() {
2    $("#login").focus();
3
4    $("#login_form").submit(function(){
5      $(".err_msg").slideUp();
6      $(".login_error").removeClass("login_error");
7      var valid = true;
8      // check if inputs are filled
9      if($("#login").val().length < 1 || $("#password").val().length < 1 ) {
10       $("#login").addClass("login_error");
11       $("#password").addClass("login_error");
12       $("#error1").slideDown();
13       return false;
14     }
15     // check log and pass
16     $.ajax({
17       type: "POST",
18       url: "/admin_panel/authorize/",
19       async: false,
20       data: {
21         login : $("#login").val(),
22         password : $("#password").val(),
23       },
24       success: function(data) {
25         if(data.indexOf("NO") != -1) {
26           $("#error2").slideDown();
```

```
27            valid = false;
28          }
29        },
30        error: function() {
31          alert("{%trans "An error occured"%}");
32          valid = false;
33        }
34      });
35      return valid;
36    });
37  });
```

Listing 2.13. *Use of AJAX for login validation.*

There is another jQuery feature that has not been introduced before, namely the event triggering (one can see an example of it in line 2). If we write a function inside the event handler method's parentheses, it means that we attach this handler to a given element. However, if we leave the method without any parameters, it will trigger the event. Therefore, after line 2 is executed, the cursor will be focused in the text field with id `login`.

Between the lines 4 – 14, for login form's validation is performed. This is one of the crucial elements of our web application. Not only the server validates the form by checking if all the necessary fields have been filled correctly, but the client browser does the same by itself. This approach makes the application more responsive and helps to lower the traffic – no data will be exchanged until we are sure it is valid.

Nevertheless, from the point of this section, the most important part of the code is presented in lines 16 – 36. It shows how the AJAX call can be made by using the jQuery built-in function. It has a JSON-like structure containing several attributes:

- ◇ `type` – it can either be `GET` or `POST` and it corresponds to the way we send the HTTP request. If the first type is chosen, the data we send to the server is presented as a part of the address and therefore visible. Otherwise, if we want to hide the information we exchange (i.e. login and password), we can use the `POST` method and the data will be hidden inside the HTTP header.
- ◇ `url` – the address that the AJAX call should be made to. In our application, it always corresponds to some URL that is specified in the Django's `urls.py`.
- ◇ `async` – this boolean attribute specifies whether the call should be asynchronous or not. In case of logging to the system, a synchronous approach is more suitable. The web page is still not reloaded, but the user cannot click on anything until the response from the server is received.
- ◇ `data` – this is a dictionary of parameters' names that we would like to send to the server along with their values. In the example above, we send the contents of fields related to the login and the password.
- ◇ `success` – corresponds to the function that should be executed when we receive the proper response from the server. It takes one parameter called data, which is the string containing the whole content of the response. Our authorization function returns `NO` if the login and password do not match any administrator and – in such case – the error message smoothly slides in to the screen.
- ◇ `error` – specifies the function, which aim is to react if the HTTP response was not received or if there was a connection failure in general.

The function `$.ajax` is jQuery's most typical way to communicate with the server in the background . Our whole design of parent, teacher and student panels is based on its usage.

Plug-ins

Being able to understand, alter and apply some external code to our own needs is an essential skill in programming. Re-usability should always be one of the priorities when developing a project. Thus, not to "reinvent the wheel", we decided to make use of multiple open source jQuery plug-ins, that significantly lessen the coding time as well as improve our project's functionality. All of them had to be individually configured and some required alterations in their code. Nevertheless, all of the chosen ones proved to be very useful and introduced a new level of advancement, both functional and visual. All the pictures presented below come from our project and show how particular plug-ins have been implemented in our project.

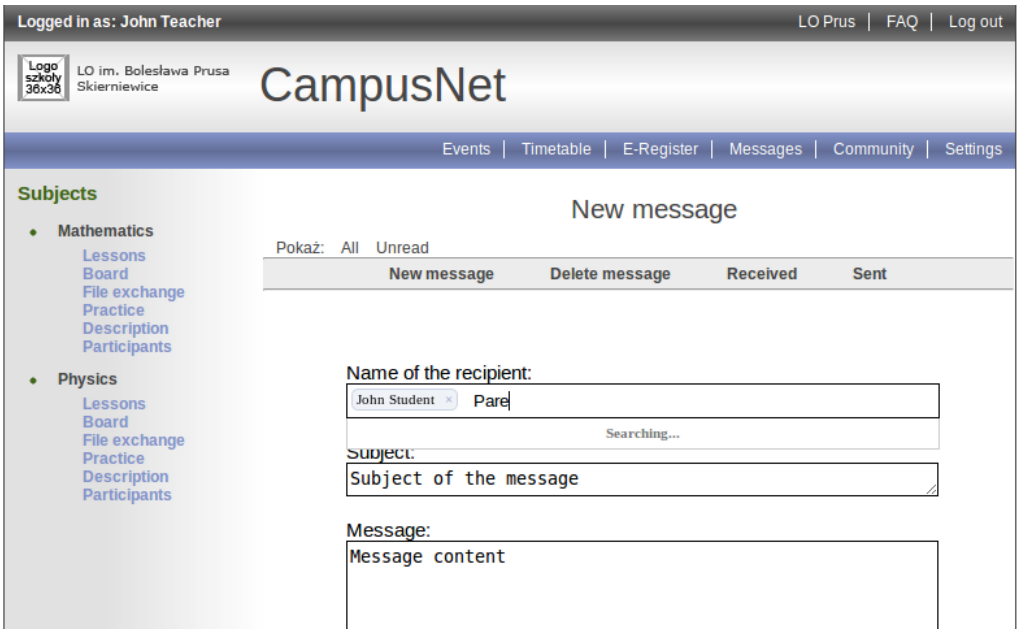


Figure 2.16. *Implementation of Tokeninput plug-in.*

Tokeninput (Fig. 2.16) is a jQuery plug-in which looks like a simple text field embedded in a web page, but offers a lot more. Being connected to Django's backend and supplied with JSON objects, it works as an auto-fill advisor. In our project, it is used in the messaging system for suggesting the users, whom we want to send a message to. After the user types in three initial letters of the first or last name, it communicates with the server in the background and returns a list of possible recipients. It also allows for selecting multiple people, what greatly simplifies the way the messages are addressed.

Another plug-in that we incorporated into our project is jQuery Datepicker (Fig. 2.17). It is a well-designed calendar that may be integrated with any text-field and help with choosing an exact date in a specified format. It is used in multiple places in our project, whenever a date field is needed. Its visual side is well designed

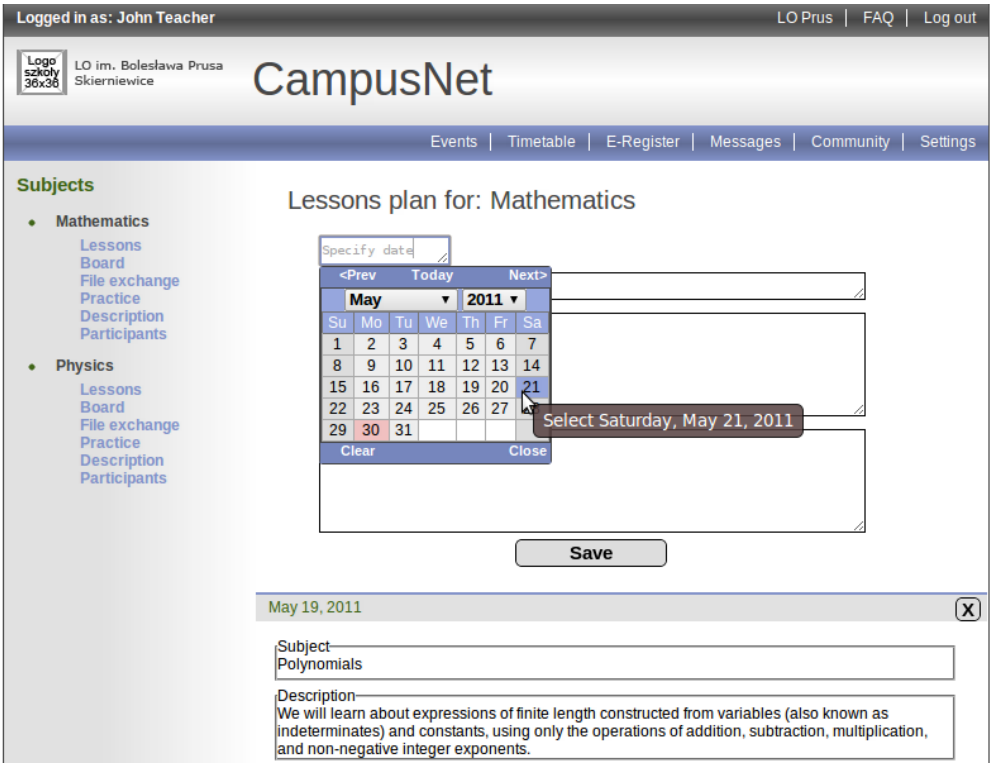


Figure 2.17. *Implementation of Datepicker plug-in.*

and customizable and it comes with many animation possibilities. It also supports several languages, which makes it a suitable tool, considering our will of creating a multi-lingual application.

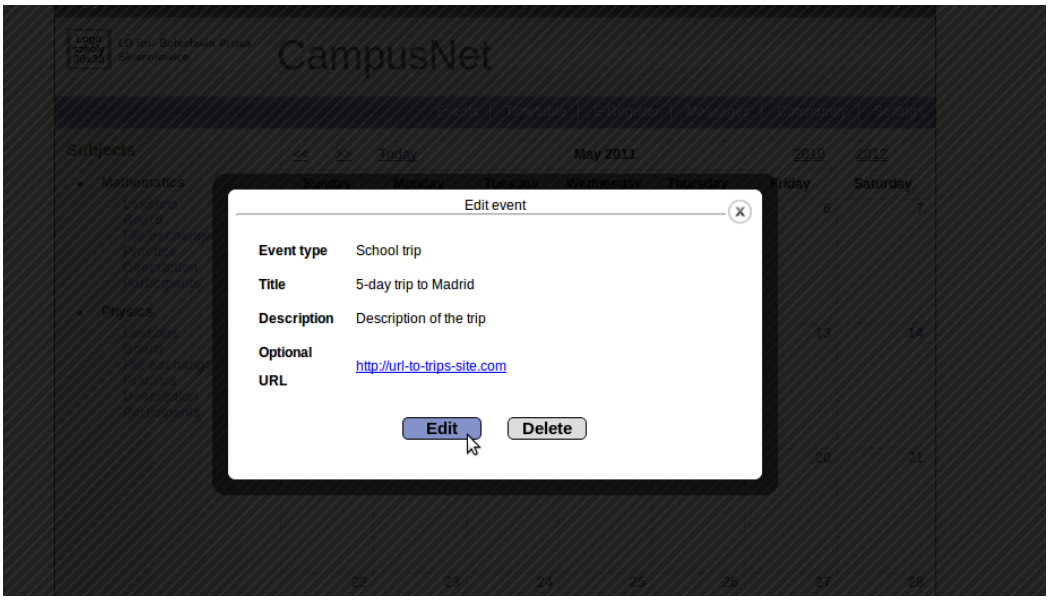


Figure 2.18. *Implementation of Colorbox plug-in.*

Colorbox (Fig. 2.18) is a light-weight, customizable plug-in which can be used for many purposes, including creation of the picture galleries, slideshows and loading new web pages as an independent pop-up box, leaving the previous web content in the background. In CampusNet project, the latter functionality is mostly used. It effectively supports the event calendar in LMS and the practice module in VLE.

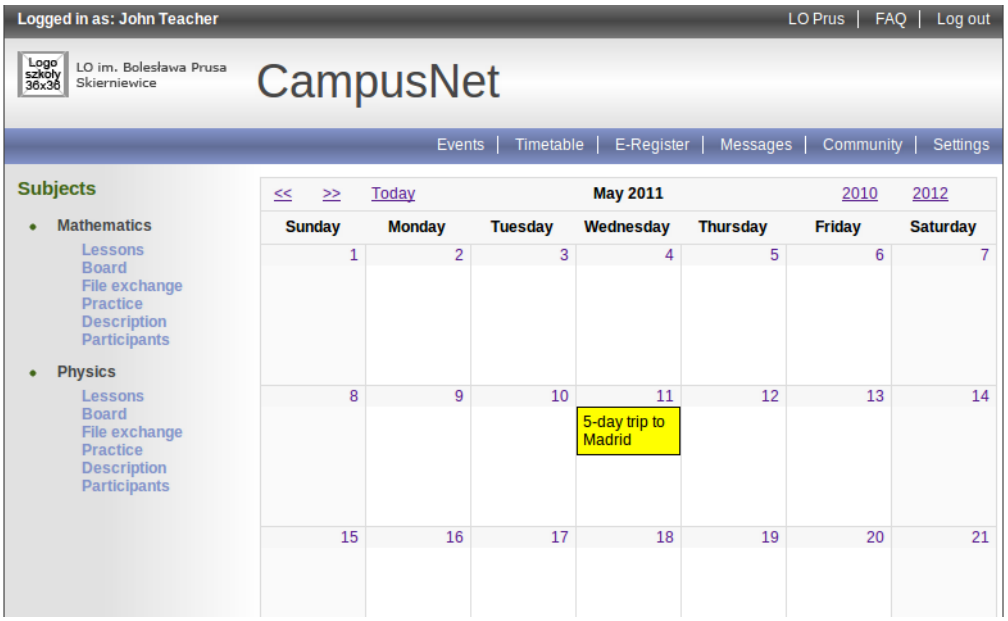


Figure 2.19. *Implementation of jMonthCalendar plug-in.*

The most sophisticated of all the plug-ins used in our project is called jMonthCalendar (Fig. 2.19). It shows all the days in a given month and supports managing the events – the ones related to the school institution in our case. It can be integrated with Django backend and communicate through JSON interface to provide an intuitive and visually attractive environment for management of events. In our project, it was also the one that needed the most alterations in the source code. We enhanced it with the flexibility to handle several event types and distinguish between them. Moreover, we integrated it with the Colorbox plug-in so that it shows pop-up boxes to create and edit events.

2.4.4. Version control

The fact that our project involves more than one person working on the source code on his own computer may be a sufficient reason to think about VCS[15]. Moreover, one of the very first observations noticed shortly after the beginning of project development was, that during the significant amount of time more than two members of our team are modifying the same file. Handling those changes without help of suitable software would be (at least) extremely complicated, not to mention the time-consumption aspect.

[15] Version Control System, also called RCS – Revision Control System or SCM – Source Control Manager.

VCS in general

The immensity of area of revision control management implies its importance to collective work. Each version control software possesses a wide set of properties, making them distinct from each other. Even though those programs differ also on technical solutions, they provide very similar features.

So as to select one particular system, we had to determine most significant features for our case. Thus we decided to look for a VCS that satisfies our requirements in terms of type of repository model, license and supported platforms. The result was narrowed down to a few solutions and finally we selected Mercurial, taking the user experience of one of team members as a decisive factor.

The very exhaustive comparison of revision control technologies including more than twenty five systems is available on the Wikipedia pages[16].

Mercurial source control manager

Above all, Mercurial is a distributed system, which means that each team member working on the project possesses a local copy of the repository and development history. This solution overcomes the traditional client-server model in many fields, e.g. it allows us to work effectively without an access to the network or central server. Moreover, each local copy works as a backup in case of data loss; the disadvantage of single point of failure is also excluded. Finally, majority of the operations are faster, as they don't need network connection. Mercurial also satisfies our second requirement: it has a free software license (under the terms of GPL - General Public License), as well as the third one: each of the following platforms – Mac OS X, Windows and Unix-like operating systems – are supported. It is written in Python (with a little addition of portable C, in order to increase performance) and uses HTTP as a transport protocol; it also remains actively developed under the supervision of its creator, Matt Mackall[17].

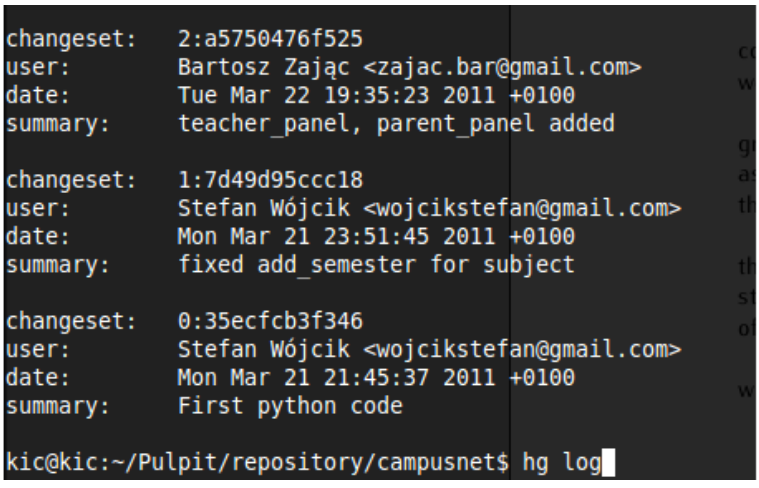


Figure 2.20. *Sample Mercurial's log output.*

[16] See: http://en.wikipedia.org/wiki/Comparison_of_revision_control_software
[17] See: http://mercurial.selenic.com/sponsors/

Setup

As stated before, Mercurial is a distributed source control management tool. This fact implies, that the commonly used client-server approach has no application here, which is also reflected in the process of installation – there exists no distinction between client's and server's version of the software.

Standard setup of Mercurial (Ubuntu OS) consists of the following steps:

◇ installation of Mercurial packages

```
1 sudo apt-get install mercurial
```

Listing 2.14. *Ubuntu's command starting the installation of Mercurial.*

◇ creation of a new repository in desired folder

```
1 hg init
```

Listing 2.15. *Mercurial's command creating new repository.*

◇ customization of the `.hg/hgrc` file, i.e. addition of the following lines

```
1 [paths]
2 default = ssh://root@ip_address//home/cn/rep
3 default-push = ssh://root@ip_address//home/cn/rep
4
5 [ui]
6 username = FirstName LastName <name@email>
```

Listing 2.16. *Editing the .hg/hgrc file.*

The default pull and push URLs make corresponding commands shorter, while username allows to identify the author of certain action. After the initial setup, Mercurial is ready to be used.

Usual workflow

Despite the fact, that Mercurial provides large number of usable commands, the typical workflow consists of only three or four of them. The first sequence:

```
1 hg log [-r tip]
2 hg pull
3 hg update
```

Listing 2.17. *Sequence of commands allowing to update the project.*

...is normally used to receive the current version of the repository. The first line causes the information about changes to be displayed, optional attribute may restrict log to the last performed change. The second command downloads the data from repository without updating user files. To do so, it is necessary to use the third and last command. After performing this sequence of commands, the user should possess an up-to-date version of the repository.

Even though automatic file merging in Mercurial works fine, in some cases program is not able to handle this task. Instead it runs GVim (Graphical Vim, text editor) with three panels, displaying three different versions of the problem file (local, from repository and suggested merge) and asking user to make the final correction. Thanks to the graphical interface and convenient highlighting desired result is easily obtainable.

One may ask what happens, if the file merging operation fails before its completion. The answer is simple – almost every VCS possesses a feature called atomic commit. The idea behind it is very simple: if an atomic commit cannot be completed, then all of the changes done since the beginning of the operation are reversed. Thanks to this solution the system is always left in consistent state.

The second workflow used frequently is connected with deploying the local version of repository:

```
1 hg status
2 hg commit –m 'Short message about changes'
3 hg push
```

Listing 2.18. *Sending updated local version of the project to the repository.*

The first command displays the status of local repository and should be performed before any commit. Mercurial shall be informed about addition of new files or deletion of old ones in order to be able to track the project properly. Hence, some additional `add` or `remove` commands may be required before a commit. The second command prepares local data to be distributed and comes along with additional parameter representing description of performed changes. Finally, with the third command, local data is being pushed to remote repositories.

In spite of the fact that Mercurial possesses its own web interface, there is no need to use it for our purposes. While simple repository updates can be easily done with command line, we use Trac (Integrated SCM and Project Management tool) for more sophisticated development issues. Not accidentally, these two tools are perfectly compatible and complement each other.

2.4.5. Project management

The idea of using project management software arose together with increasing number of code lines of our design. After committing draft versions of consecutive subparts of the project (e.g. message exchange system, calendar etc.), we were giving the opportunity to test them to other team members. Consequently, we needed an efficient way of alerting noticed errors and improvement suggestions. The Trac Ticket System not only satisfied this need, but also enhanced our methodology of work with additional interesting solutions.

Trac

This time there was no need to browse through dozens of available software, as the choice was very easy. We were familiar with the fact that Trac exists, and that it can provide all necessary features along with a handy web interface to Mercurial.

Trac possesses a bunch of useful features for group project development, including timeline, roadmap, ticket system and source browser. What is also very important, Trac is available under the terms of modified BSD (Berkeley Software Distribution) license, which means it is an open source software. Trac may be also classified as Wiki software and -what is quite interesting- it uses one of its instances to manage the development of the entire project itself. This project management software was written in Python programming language, which has the motivational meaning to our team. By using Trac we follow the example of such projects as Django, jQuery or SourceForge.net.



Figure 2.21. *Trac instance for CampusNet platform.*

Trac features

Trac offers several interesting features, that may help to develop the project. In case of our design, especially last two subsystems play an important role:

- TracWiki – the built-in Wiki, which can be used as a great source of information about the project, including documentation, manual, tutorials etc.
- TracTimeline – enables users to see the project from a historic point of view,
- TracRss – supports content syndication using Really Simple Syndication XML format,
- The Version Control Subsystem – consisting of TracBrowser, TracChangeset, TracRevisionLog,
- The Ticket Subsystem – consisting of TracTickets, TracReports, TracQuery and TracRoadmap.

TracChangeset, part of the Version Control Subsystem, turned out to be an extremely useful tool. Even though, as we already mentioned, Mercurial's merging tool works fine, countless number of times happened, that some part of the code was lost during consecutive commits. Thanks to TracChangeset, a functionality for visualizing changes to files (so called 'diffs'), tracking such losses becomes extremely simple, and the amount of time spared may be counted in hours.

The second essential Trac feature facilitating the development of our project is certainly the Ticket Subsystem. As mentioned before, we learned that group programming work requires a place to alert encountered errors, display current work assignment and the plan for future progress. First two needs may be satisfied by TracTickets, while the organization of forthcoming work can be handled with TracRoadmap.



Figure 2.22. *Logo of the Komodo code editor.*

2.4.6. Other tools

Komodo Edit

When it comes to a development of project comprising several thousands of lines, the importance of code editor choice cannot be underestimated. A proper selection of such tool may significantly improve the efficiency of work, saving a programmer significant amount of invaluable (especially when the deadline is approaching) time. Bad decision however may drastically decrease one's performance, turning the coding process into terrible experience.

Despite the fact that every team member possesses his own preferences regarding code editors, we decided to unify the methodology of work and approach to test an unfamiliar program. From the list of fast, open-source editors supporting Python, Javascript, HTML and CSS we have chosen to try Komodo Edit.

Komodo's auto-complete, syntax and error highlighting features proved to be very useful for CampusNet programmers. Complementary HTML toolkit provided similar functionalities for HTML documents edition. Light weight, stability, intuitiveness and smart short key system turned out to be big advantages of this software, allowing us to comfortably get through the coding stage of our project.

LaTeX



Figure 2.23. *Logo of the Latex editor.*

One last decision regarding software had to be made when it came to the selection of document editor. Basically, we had to decide whether to use a well-known WYSIWYG (What You See Is What You Get) editor such as Microsoft Word and OpenOffice Writer, or learn something new due to selection of a WYSIWYM (What You See Is What You Mean) software. The final choice was to follow the ambitious path - partly to extend the knowledge, partly to avoid waste of time needed to correct Word's auto-correction - and use LaTeX document preparation system.

Despite the necessity of laborious study of new markup language, LaTeX proved to be a very efficient tool that facilitated handling of this document. Thanks to application of standard templates, the task of maintaining coherence of written work did not cause any troubles. Sensible automation of generating indexes, listings, references, numerations, tables and bibliographies turned out to be extremely useful, taking into account size of the document.

Dropbox



Figure 2.24. *Dropbox project logo.*

Despite the implementation of all technologies mentioned before, the team was still lacking in one very important functionality: ability to exchange files in a quick and efficient way. This need arose due to the increasing number of various files containing diagrams, tutorials, schemes, partial documentation etcetera, that we wanted to share among each other. Thus a decision has been made: to search for a software, that allows to store aforementioned files in a single place, permitting all team members to access them from any place at any time. Again, brief research led to employment of Dropbox, a web-based file hosting service.

Dropbox equipped us with a set of interesting features, including:

- 2 GB free of charge storage limit,
- simple version control system,
- tracking changes history,
- recovery of deleted files.

These capabilities occurred to be particularly useful during the early stage of project development, when new, potentially helpful materials were discovered frequently. Later on, along with termination of research and beginning of actual coding work, Dropbox lost its significance, ousted by Trac and Mercurial.

ReportLab



Figure 2.25. *Logo of the ReportLab project.*

As a matter of form, we would like to briefly describe the aforementioned toolkit. ReportLab is an open source project, available under the terms of BSD (Berkeley Software Distribution) license. It has been written in Python (as the majority of tools we describe in this thesis) and – as a cross-platform design - works on various operating systems, including MS Windows, Mac OS, Solaris, Linux, AIX[18], FreeBSD and others.

From among various applications of this software (creating platform-independent vector graphics, it also forms the foundation of RML – Report Markup Language – XML-style language for describing document layouts), we made use of PDF generation ability, giving all teachers using CampusNet platform the possibility of automatic creation of students' mark reports.

[18] Advanced Interactive eXecutive

3. Project development

3.1. Database model

3.1.1. Design and integration

The vast majority of web applications are in fact database-driven and their performance depends heavily on the quality of database design. Our LMS is not an exception – as it is destined for high schools, it has to maintain and structure all data in a transparent and efficient way. In terms of database design one of the fundamental issues was to determine which real-life objects and dependencies related to high school education should be represented in the database. The final list of data models is presented below.

Models related to high school community:

- person,
- employee,
- teacher,
- student.

Models related to education system:

- class,
- class profile,
- lesson,
- subject,
- event,
- mark,
- mark column,
- semester mark,
- attendance.

Models related to communication between people:

- post,
- message sender,
- message recipient.

Models for information storing:

- private data,
- school info,
- hour grid.

We attempted to maintain the database as transparent as possible, avoiding unnecessary duplication of data whenever possible. Naturally this approach complicated the coding part of the project in some parts, but we were aware that it was a trade-off between code complexity and tidy database design.

We believe that the list above is sufficient for the LMS to represent the working model of operation of a high school. However, Campusnet project includes also Virtual Learning Environment, which introduces additional data models. As it is a subject of another bachelor thesis, we would like only to list them, without describing their functions in forthcoming paragraphs:

Models used in VLE:

- folder material,
- folder material group,
- memo notebook,
- memo entry,
- memo stat,
- memo language,
- memo word,
- memo definition,
- memo formula,
- memo quiz,
- memo choice.

3.1.2. Models' description

Having stressed all database models, we would like to proceed with their specification. The following paragraph list data models with all their fields and short descriptions, giving the explanation to every field if its name does not speak for itself.

```
1 class Person(models.Model):
2     first_name = models.CharField(max_length = 32)
3     middle_name = models.CharField(max_length = 32, null=True, blank=True)
4     last_name = models.CharField(max_length = 64)
5     user = models.ForeignKey(User)
```

Listing 3.1. *Model representing a Person in the database.*

Model representing every single person existing in the database. First three fields are obvious, while user is a foreign key to Django `User` class, a part of built-in authentication system. Each of remaining people-related models contain a foreign key to this class.

```
1 class Employee(models.Model):
2     person = models.ForeignKey(Person)
3     nip = models.CharField(max_length = 10, null = True, blank = True)
4     emp_date = models.DateField('data zatrudnienia', null = True, blank = True)
```

Listing 3.2. *Model representing an Employee in the database.*

This class enables us to represent a school's employee in the system, extending the previous model by some additional data such as tax identification number or date of hiring the person.

```
1 class Student(models.Model):
2     person = models.ForeignKey(Person, related_name = "personstudent")
3     identification_no = models.CharField(max_length = 10)
4     class_fk = models.ForeignKey(Class, null=True, blank=True)
5     parent1 = models.ForeignKey(Person, related_name = "parent1")
6     parent2 = models.ForeignKey(Person, null = True, blank=True, related_name =
7       "parent2")
```

Listing 3.3. *Model representing a Student in the database.*

A model with the highest number of instances (from among people-related models) in the database, describing a regular student. This class contains four foreign keys, three of which are related to class `Person` (one to own data and two remaining to parents' data). The last foreign key connects student with his/her class. Every student possesses his own identification number.

```
1 class Teacher(models.Model):
2     person = models.ForeignKey(Person)
3     title = models.CharField(max_length = 16, null = True)
4     specialization = models.CharField(max_length = 64, null = True)
```

Listing 3.4. *Model representing a Teacher in the database.*

The last class representing a human being in the database. Similarly to `Student` and `Employee` classes, it possesses a foreign key to `Person` model. It extends mentioned model by additional `title` and `specialization` fields.

```
1 class PrivateData(models.Model):
2     person = models.ForeignKey(Person)
3     address = models.CharField(max_length = 255, null=True, blank=True)
4     e_mail = models.EmailField(null = True, blank=True)
5     pesel = models.CharField(max_length = 11, null = True, blank=True)
6     phone_no = models.CharField(max_length = 32, null = True, blank=True)
```

Listing 3.5. *Model representing Private Data in the database.*

`PrivateData` class serves as an extension to `Person` class. This model is responsible for storing private information concerning each user of the system, including his/her address, e-mail address, PESEL and phone number.

```
1 class HourGrid(models.Model):
2     hour_order = models.SmallIntegerField(max_length=2)
3     start_time = models.TimeField('start time')
4     end_time = models.TimeField('end time')
```

Listing 3.6. *Model representing Hour Grid in the database.*

Model ensuring proper placement of a subject (class `Subject` possesses a foreign key to it) in the schedule. `hour_order` field stands for the event sequence number on given day, start and end time fields describe the hours of the beginning and termination of the class, respectively. This model was introduced for the sake of effectivity. From experience, we know that school hours may sometimes change throughout the semester. In such case, if each subject contained just start time and end time fields, the administrator would be forced to change the data for all the subjects separately, one by one. Having a generic hour grid allows us to change the data in one place and it will be reflected in the whole system.

```
1 class ClassProfile(models.Model):
2     name = models.CharField(max_length = 64)
```

Listing 3.7. *Model representing a Class Profile in the database.*

Very brief and self-explanatory model, representing the bias of an arbitrary class. Each `Class` object possesses a foreign key to this model.

```
1 class Semester(models.Model):
2     season = models.CharField(max_length = 8, choices = SEMSEASONCHOICE)
3     year = models.IntegerField(max_length=4)
```

Listing 3.8. *Model representing a Semester in the database.*

Simple class representing a period of study as season (winter/summer) and year. Some other classes use this model as a foreign key.

```
1 class Class(models.Model):
2     edu_level = models.IntegerField()
3     identifier = models.CharField(max_length = 3)
4     profile = models.ForeignKey(ClassProfile)
5     classroom = models.CharField(max_length = 5)
6     teacher = models.ForeignKey(Teacher)
7     semester = models.ForeignKey(Semester)
```

Listing 3.9. *Model representing a Class in the database.*

Model describing single class in a school. `classroom` field represents the number of home room for given class, `edu_level` stands for the year of education and `identifier` is usually a single letter. This class contains three foreign keys (`profile`, `teacher` and `semester`) to classes mentioned before.

```
1 class Subject(models.Model):
2     name = models.CharField(max_length=64)
3     description = models.TextField()
4     semester = models.ForeignKey(Semester)
5     teacher = models.ForeignKey(Teacher)
6     students = models.ManyToManyField('Student', null=True, blank=True)
7     hour = models.ForeignKey(HourGrid)
8     weekday = models.SmallIntegerField(max_length = 1)
9     classroom = models.CharField(max_length = 5, null = True, blank = True)
```

Listing 3.10. *Model representing a Subject in the database.*

Class representing a subject being taught in a high school. Beside foreign keys to previously described models, it contains the `name` field (the name should preferably include the class identifier to ensure easy distinction), `description` field editable by every teacher, as well as `weekday` and `classroom` fields. Finally, there exists one many-to-many relation, listing all students signed up for the course.

```
1 class Event(models.Model):
2     date = models.DateField('event_date')
3     title = models.CharField(max_length = 127)
4     description = models.TextField()
5     type = models.CharField(max_length = 15, choices = EVENTTYPECHOICE)
6     url = models.URLField(null = True, blank = True)
7
8 EVENTTYPECHOICE = (
9     ('school_event','school_event'),
10    ('exam','exam'),
11    ('student_event', 'student_event'),
12    ('school_trip','school_trip'),
13 )
```

Listing 3.11. *Model representing an Event in the database.*

This model illustrates an event, that may take place during school's year. First three fields are self-explanatory, the `type` field allows us to choose from possible event types listed in `EVENTTYPECHOICE` and url serves as optional link to a web page providing more information.

```
1 class Lesson(models.Model):
2     date = models.DateField('lesson_date')
3     title = models.CharField(max_length = 255)
4     description = models.TextField()
5     homework = models.TextField()
6     subject = models.ForeignKey(Subject)
7     people = models.ManyToManyField('Person', null=True, blank=True)
```

Listing 3.12. *Model representing a Lesson in the database.*

This class describes a single lesson occurring during given day. It possesses fields storing lesson's `title`, `description` and `homework` given to students, indicated by many-to-many field. Foreign key to `Subject` class states subject, to which the lesson is related.

```
1 class SemesterMark(models.Model):
2     student = models.ForeignKey(Student, related_name = "semester_mark_student")
3     value = models.CharField(max_length = 14, choices = SEMMARKCHOICE)
4     column = models.ForeignKey(MarkColumn)
```

Listing 3.13. *Model representing a Semester Mark in the database.*

Model representing a semester mark, received by `student` pointed by foreign key at the end of semester. The second foreign key (`column`), relates the mark to a certain column in virtual register. The `value` field is self-explanatory.

```
1 class Mark(models.Model):
2     student = models.ForeignKey(Student, related_name = "mark_student")
3     value = models.CharField(max_length = 2)
4     column = models.ForeignKey(MarkColumn)
```

Listing 3.14. *Model representing a Mark in the database.*

Model representing a partial mark, similar to the previous one. We decided to introduce this class so as to have two different types of marks separated, as values they can store are different.

```
1 class MarkColumn(models.Model):
2     name = models.CharField(max_length = 32)
3     weight = models.SmallIntegerField(max_length = 2)
4     subject = models.ForeignKey(Subject, related_name = "mark_subject")
5     semester_mark = models.BooleanField(default=False)
```

Listing 3.15. *Model representing a Mark Column in the database.*

Class describing a single column entry (e.g. class test, homework,) to be filled with marks in virtual register. `weight` field allows to balance the importance of each column. Foreign key to `Subject` class has an obvious meaning, while boolean field `semester_mark` helps to determine, whether this column contains partial or semester marks.

```
1 class Attendance(models.Model):
2     date = models.DateField()
3     lesson_no = models.SmallIntegerField(max_length = 1)
4     subject = models.ForeignKey(Subject)
5     student = models.ForeignKey(Student)
6     value = models.CharField(max_length = 2)
```

Listing 3.16. *Model representing an Attendance in the database.*

This model illustrates student's attendance at certain lesson. `date` and `lesson_no` fields uniquely determine related lesson, while `subject` and `student` foreign keys indicate other involved models. `value` field, according to its name, stores the essential information about student's presence/absence.

```
1  class Post(models.Model):
2      author = models.ForeignKey(Person, related_name = "post_author")
3      content = models.CharField(max_length = 255)
4      time = models.DateTimeField('Data dodania')
5      subject = models.ForeignKey(Subject, related_name = "post_subject")
```

Listing 3.17. *Model representing a Post in the database.*

Class describing entries on subject's board. `author`, `content` and `time` fields are used to store relevant information that is displayed in mentioned section.

```
1  class MessageSender(models.Model):
2      sender = models.ForeignKey(Person, related_name = "s_sender")
3      time = models.DateTimeField("Data wyslania")
4      topic = models.CharField(max_length = 255)
5      content = models.TextField()
6      recipient = models.ForeignKey(Person, related_name = "s_recipient")
7      read = models.BooleanField(default = False)
8
9  class MessageRecipient(models.Model):
10     sender = models.ForeignKey(Person, related_name = "r_sender")
11     time = models.DateTimeField("Data wyslania")
12     topic = models.CharField(max_length = 255)
13     content = models.TextField()
14     recipient = models.ForeignKey(Person, related_name = "r_recipient")
15     read = models.BooleanField(default = False)
```

Listing 3.18. *Models representing Messages (sent and received) in the database.*

Two models implemented into the message exchange system. The duality results from the fact, that each message is created twice, one copy belongs to the sender, while the other one to the `recipient`. `sender`, `time`, `topic`, `content` and `recipient` fields are self-explanatory, while the boolean `read` field indicates, whether the message has already been read by the owner.

```
1  class SchoolInfo(models.Model):
2      name_short = models.CharField(max_length = 64)
3      name_long = models.CharField(max_length = 255)
4      city = models.CharField(max_length = 32)
5      address = models.CharField(max_length = 255)
6      postal_code = models.CharField(max_length = 7)
7      phone = models.CharField(max_length = 16)
8      fax = models.CharField(max_length = 16)
9      website = models.CharField(max_length = 255)
10     e_mail = models.EmailField()
```

Listing 3.19. *Model representing a School Information in the database.*

The last class implemented in LMS part of the project serves to store the basic information about the high school itself. Included fields allow to keep two versions of school's name, its full address along with telephone/fax numbers, as well as the information about website and e-mail address.

3.2. Web application

3.2.1. General description

Having the data base model finished, the process of web application's development could start. The general idea behind it was to turn the objectives, use cases and requirements into code. Several steps had to be taken in order to do so:

- ◇ Design of the user interface and its structure,
- ◇ Development of URLs and views to support the logic and addressing,
- ◇ Development of HTML templates and CSS styles to support the design,
- ◇ Development of JavaScript code for form validation, animations, asynchronous communication with the server and event handling in general.

Obviously, these steps were interchangeable on some level and required many iterations before we could be satisfied with the final product. In further sections, the reader can find information about the project's implementation, description of the user interface from the perspective of offered functionalities and notes on security issues, as well as discussion on different web browsers' compatibility.

As a matter of fact, the whole web application integrates modules – LMS and VLE. As the latter one is out of the scope of this thesis, further sections will not focus on the functionalities it provides.

3.2.2. Structural design and implementation

Directory structure

Web application definitely needs to be consistent in the matter of structure, not only because it makes the development easier, but also because it is deployed on a server, thus, all the resources have to be available under the expected address. Moreover, a proper directory structure helps with the separation of the data of different types. We dealt with that issue by designing such structural organization (root is considered to be the main CampusNet folder):

- ◇ `/admin_panel/`, `/parent_panel/`, `/student_panel/`, `/teacher_panel/` - main directories for the administration, parent, student and teacher panels, respectively. They contain the files specifying the related views and URL addresses.
- ◇ `/admin_panel/templates/`, `/parent_panel/templates/`, `/student_panel/templates/`, `/teacher_panel/templates/` - contain HTML templates for all the web pages associated with given panels.
- ◇ `/apache/` - holds the files that specify the settings for deploying the whole project on Apache HTTP server.
- ◇ `/dbmodel/` - it is a directory of an artificial Django app that was created to hold all the database models used in LMS in an isolated space. It also contains some generic functions related to users' authorization.
- ◇ `/dbmodel/fixtures/` - contains database's initial data in the JSON format. Whenever we perform the Django – MySQL synchronization, the objects specified here are added to the database (unless some objects with the same ids already existed),

- ◇ `/static/` - the main directory for all the external resources that are loaded when one of application's web pages is loaded.
- ◇ `/static/css/` - an isolated place for all the Cascade Style Sheets used in the project.
- ◇ `/static/js/` - holds all the JavaScript files used in the project.
- ◇ `/static/profile_photos/` - it is a container for all the profile photos that CampusNet users can upload.
- ◇ `/welcome_page/` - directory for the URL addresses and functions related to the welcome page of the application (the authorization page).
- ◇ root folder contains the settings, general URLs (supporting further redirection to panel-specific address interpreters) and the management file for the whole project.
- ◇ `/folder/`, `/memorizer/`, `/static/files/` and `/wifi_panel/` - separate directories for VLE and WNI modules.

This organization of the directories and files has proven to be both useful and intuitive.

Administration panel

Before users can access their account-related content, first they need to be introduced to the school's data base. Most of the data related to a user is private, therefore it can only be accessed by him/her or the administrator, who is authorized to see and manage it. This separation of rights introduces the need to develop an isolated control center for all the data. Theoretically, it could be done by using the MySQL shell and modifying different tables' content by typing in the instructions, however it would definitely prove to be very problematic, as it would need an operator skilled in SQL language. Therefore, we developed a separate administration panel, used only for an intuitive management of the data base. It serves a completely different purpose than the other panels, thus its design is not consistent with them, but has rather been implemented in a way that will make the data administration as clear as possible.

The panel consists of several key areas. Most important is the menu (Fig. B.1, Area 1), where the user can choose which type of data he/she wants to add, edit or delete. The layout of the buttons is driven by the order in which the administrator usually adds new data. After clicking one of the menu items, the related overview of the existing objects appears in the content area (Fig. B.1, Area 2). The excessive rows are split into pages and can be accessed by the navigation menu (Fig. B.1, Area 6). The data may be sorted by several attributes, what is achieved by clicking on one of the buttons from the sorting area (Fig. B.1, Area 3). Moreover, each of the objects from the content area can be clicked, which triggers the application to show the edit view. The user can also add new objects or delete the existing ones by pointing to the buttons in the data manipulation area (Fig. B.1, Area 4). Finally, he/she can log out of the panel or edit his/her settings by choosing appropriate button in the settings area (Fig. B.1, Area 5).

This structural design has been shown to the high school staff and recognized as both visually attractive and easy-to-learn, thus our goal was met.

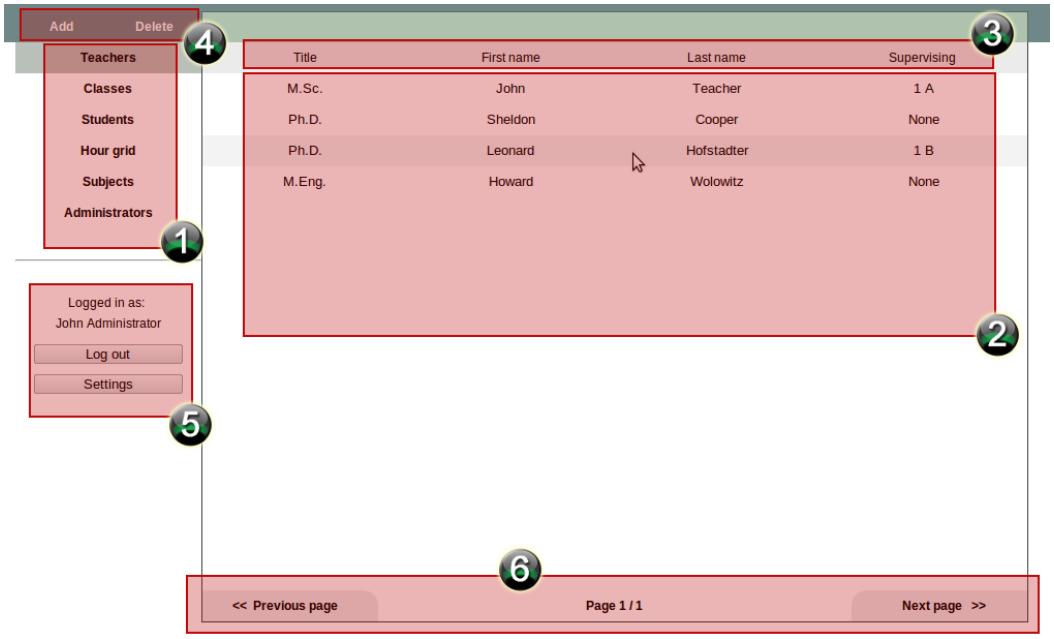


Figure 3.1. *Administrator panel layout.*

Other panels

The previous section explained the role of administrator panel, a very important part of our project. Finally, we reach the place where we would like to present the core of Learning Management System - the list and description of all project's features and functionalities seen from the point of view of different users – students, teachers and parents. Moreover, we will point out the differences between different panels and justify them.

In general, there exist five different panels within the project: administrator panel, WIFI management panel (which lies beyond the scope of this thesis, as it is a part of Wireless Network Infrastructure) and three others, as we decided to organize the user side of LMS as three separate entities (Fig. 3.2):

- ◇ parent panel
- ◇ student panel
- ◇ teacher panel

One may think about the rightness of dividing the user access into three different panels, as it does not incorporate any new elements into the set of project's functionalities. Instead, this approach significantly enlarges the amount of programming work, simultaneously increasing program's complexity and weight. Nonetheless, few minutes devoted to rational reasoning prove the correctness of such attitude. In fact, it is just enough to recall own high school experience. Let us take a regular student as an example: he/she is (at least in theory) interested only in his/her own grades, on the contrary to the teacher, which prefers to see all grades of the class he is currently teaching. Also parent needs may be considered as an instance proving the point: they do not require access to Virtual Learning Environment or subject's board. Nonetheless, as any other person using the system, parent should be able to communicate via the message system. These are

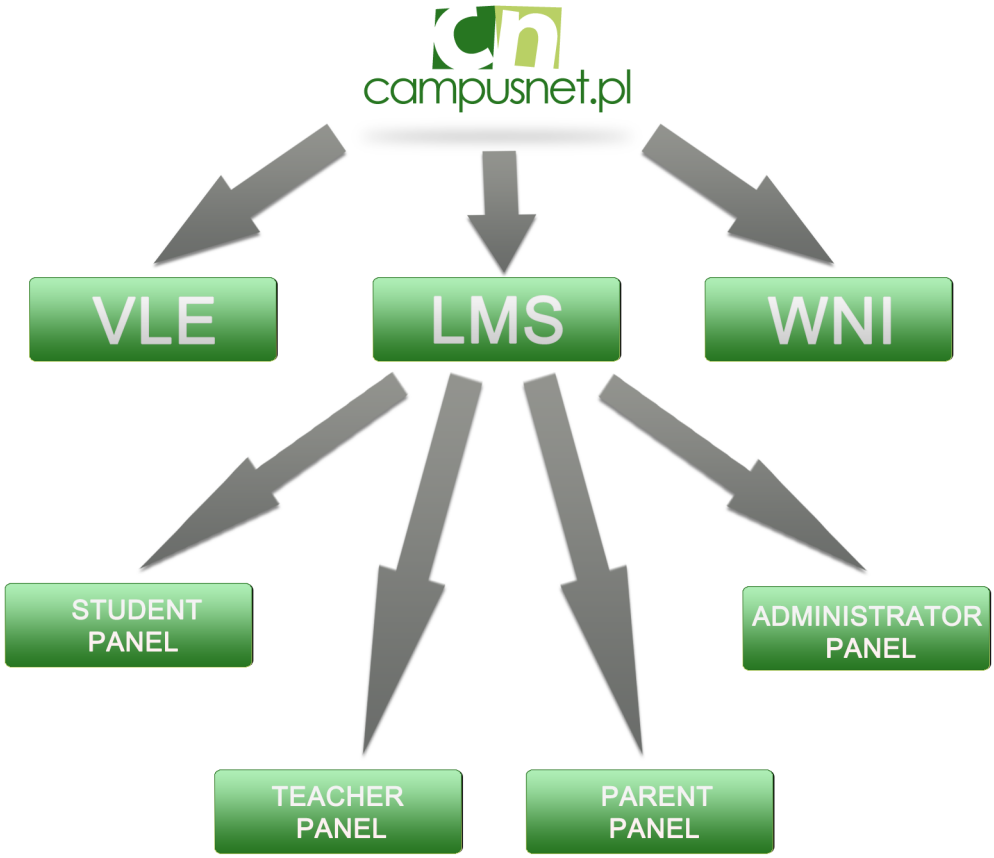


Figure 3.2. *General structure of CampusNet and LMS.*

only two examples out of many, which led us to the conclusion, that distinguishing among three types of user is not only helpful, but even necessary.

We would like to start with the list of available features, followed by a description of every item with respect to the type of user. The features are categorized according to the menus, which allow to access them.

authorization menu:

- about CampusNet,
- help,
- administrator panel,

top menu:

- high school,
- FAQ,
- logout,

subjects menu:

- lessons,
- board,

- ◇ file exchange,
- ◇ practice,
- ◇ description,
- ◇ participants,

personal menu:

- ◇ events,
- ◇ timetable,
- ◇ e-register,
- ◇ messages,
- ◇ community,
- ◇ settings.

Authorization menu



Figure 3.3. *CampusNet welcome page with highlighted authorization menu.*

Due to the fact, that CampusNet welcome page (Fig. 3.3) serves also for the authorization purposes, this menu is available for any Internet user visiting it. Consequently, it does not contain any private information related to the high school, offering some overview of the system instead.

About CampusNet hyper link provides general information about our system, encouraging the reader to visit the website of the project.

Help option supplies user with a simple authorization instructions, as well as the CampusNet staff contact methods.

Finally, the administrator link redirects user to the welcome page of administrator panel, described deeply in section 3.2.

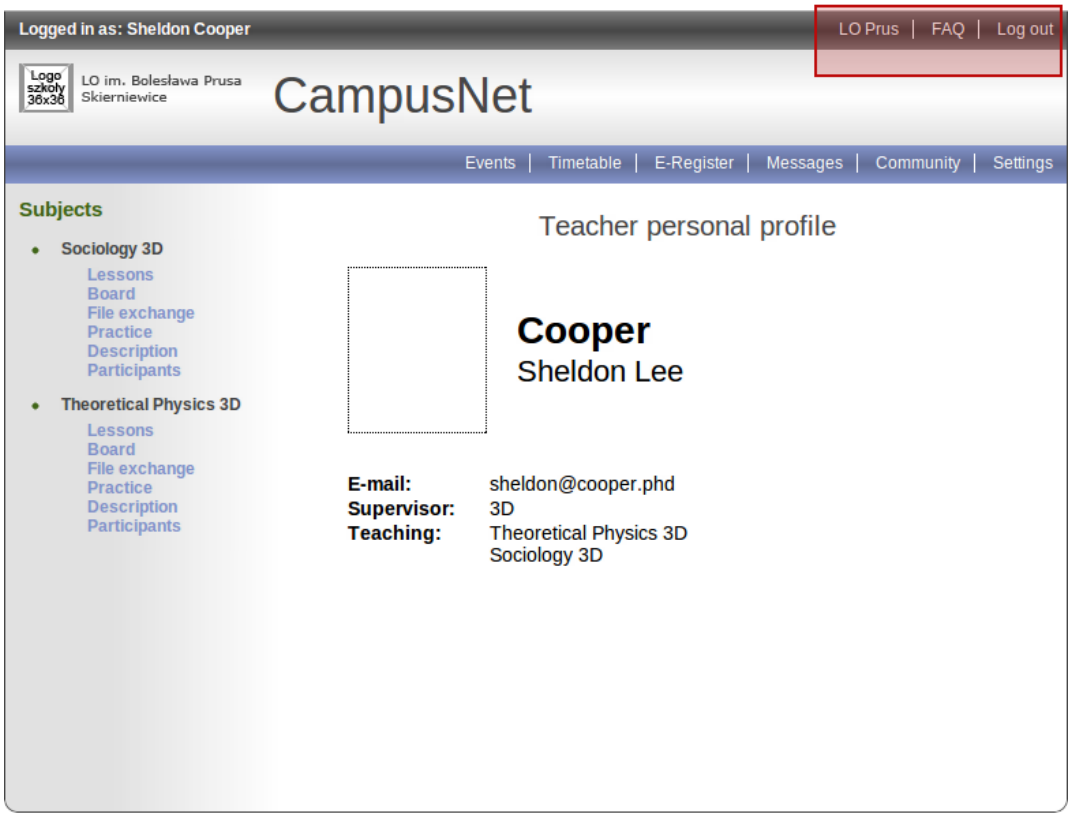


Figure 3.4. *Teacher panel with highlighted top menu.*

Top menu

Very simple menu (highlighted in Fig. 3.4) containing three items. The first hyperlink redirects user to the official web page of the school. The second item is a direct link to Frequently Asked Questions section, while the third one allows user to log out (currently the automatic logout option is not enabled in order to avoid situations, when the user is being logged out during e.g. redaction of a long message. However it can be easily changed with use of Django built-in function.). This menu remains unchanged regardless of the type of user.

Personal menu

The personal menu (highlighted in Fig. 3.5), conversely to authorization and top menus, provides different functionalities for each type of user. While the list of position of the menu remain constant, actions available to perform after selecting one of the list's element vary.

Let us begin with the first position in the menu – Events. Following this hyperlink causes a monthly calendar to be displayed on the screen. The calendar is common for the entire community and illustrates lifetime of the high school, keeping track of all school events, such as excursions, final exams, sport initiatives, etcetera. While student and parents are allowed only to browse events bound to certain days, a teacher is able to add new and delete/modify existing events. In

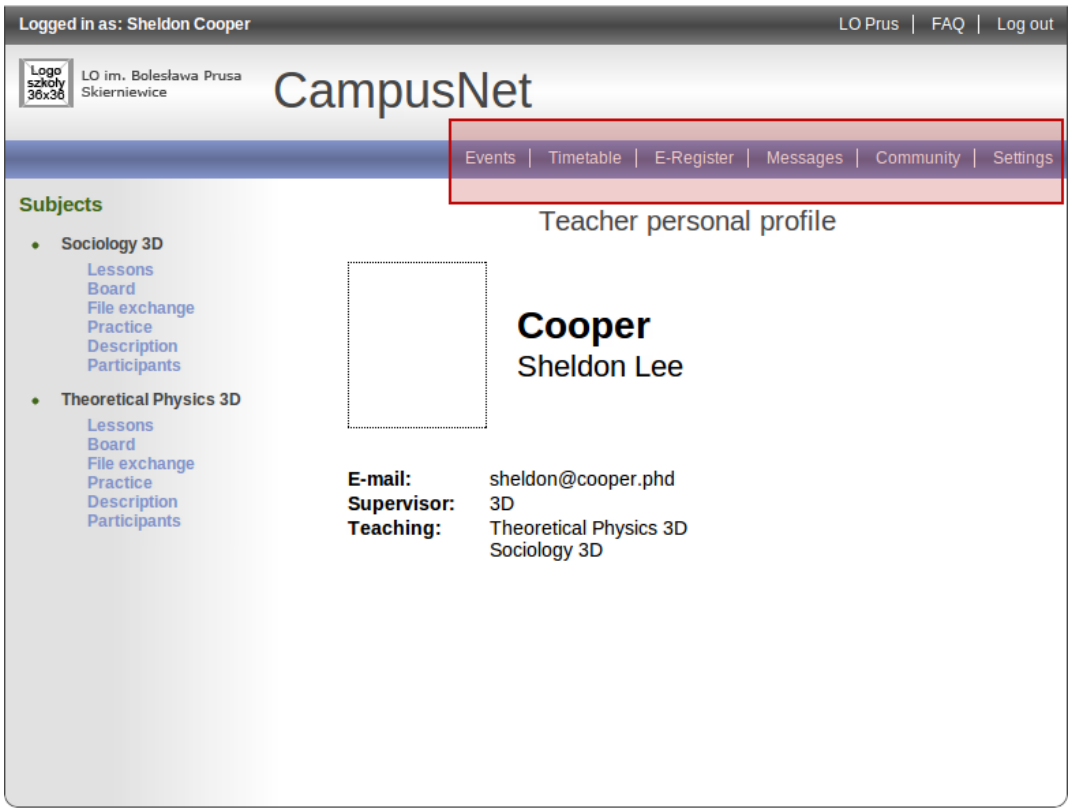


Figure 3.5. *Teacher panel with highlighted personal menu.*

further development we are going to implement a class-oriented version of calendar (and give a privilege to modify it to its president), so as to present class internal affairs, such as tests and deadlines for homeworks, in a transparent way. Adding this option to general school calendar would make it overcrowded, reducing its legibility.

Timetable - the second item in the personal menu - is far more personalized. Selection of this option displays an individual schedule, adapted for each user. In case of student, the schedule is generated on the basis of courses he/she is following. Parents by default are allowed to see the timetable of their children. When it comes to the teachers, they are able to see all tutored classes. None of three user types is allowed to modify the schedule, as in vast majority of schools timetable is issued by a superior – e.g. (vice) headmaster, or just any other person capable of organizing it in a satisfactory way. Edition is possible only from the administrator panel.

E-register functionality was introduced in order to support (and hopefully replace in the future) the traditional printed registers. It is divided into two subparts: one of them is related to marks, while the other is associated with attendances. Students are allowed only to see the extract from the register, i.e. their own grades and attendance performance with reference to courses they are attending. Similarly to the personal schedule, parents may follow the information regarding their children. As can be expected, teachers received more privileges here. For every class being tutored by them, they possess a separate instance of e-register. They

are permitted to add, edit and delete columns representing various classroom activities that undergo evaluation. The same operations are allowed regarding partial and semester marks. In case of need, students' grades may be saved as PDF file and printed out. Last but not least, school professors can monitor the attendance of every student thanks to modifiable attendance fields in the second subpart of virtual register.

The next position in personal menu redirects user to the interior message system implemented in CampusNet. At current stage of program development, the functionality of this system is similar for every user type. The module separates received and sent messages, displaying only one sort of them according to user's wish. The idea of read/unread message is also implemented - the status changes automatically after following the link to a single message. Naturally, users can create new messages and send them to multiple recipients represented as a token-style list. Message deletion may be performed simultaneously on more than one element. Even tough the module guarantees effective communication within high school community, it requires some future extension. Certainly we are going to implement some restrictions for students' recipient lists. We would also like to increase the number of available features, i.e. introduce message threads, group recipients or manual read/unread setting.

The penultimate functionality of personal menu grants a possibility to browse through the school's community information. Every person with his/her private data present in database, possesses own individual profile accessible via community module. The information displayed depends on the type of user, e.g. we believe that there exist no need for students to see parents telephone numbers, while such data may be useful for class supervisor desiring to contact them. Community module also provides space for profile photographies.

Finally, we would like to describe the last option available from the personal menu – settings module. It offers every user an insight into his/her profile, possibility to check data correctness and supplement it with missing information (indirectly – through a message to administrative staff). One may also change system password or upload personal photography. This module is similar for every user, there exist only minor differences in displayed data.

Subject menu

Now we would like to proceed to the description of last menu (highlighted in Fig. 3.6), connected with subjects. Conversely to previously mentioned menus, this one has usually more than one instance for each user. The number is equal to the number of courses the student is attending (again, parents see courses of their children), or in case of professors – tutoring.

Let us begin with the first position on the list, the lessons module. The idea behind it is to keep track of all lessons during school semester (stored information refers to the date, general topic, brief description and given homework). As can be expected, from the point of view of users the most important events are those that recently occurred, hence they appear at the top of the list. Students (and their parents) are allowed to view every entry related to the courses they follow. Teachers, who are responsible for maintaining the information up to date, are also allowed to add new events and edit them.

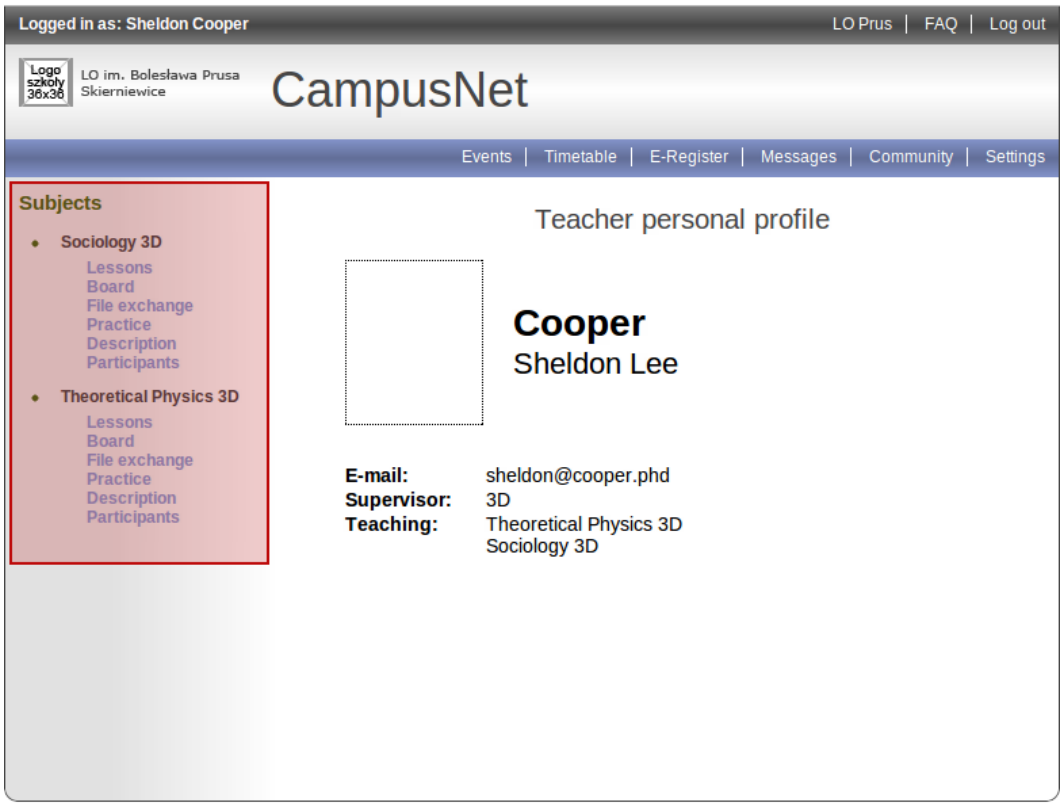


Figure 3.6. *Teacher panel with highlighted subject menu.*

The second functionality available from subject menu is message board. Thanks to this feature, every person attending given subject (usually one teacher and a group of students) is given the opportunity to quickly exchange information with other participants. The board is meant to house people's doubts and questions, along with their solutions. As of the present moment, both students and professors possess similar rights, i.e. the are allowed to post messages onto the board and delete existing ones (only one's own). Parents have no access to this section, as we believe there exist no need for such solution. Posts are sorted in ascending order, according to date of creation.

Two successive features are connected with VLE and we are not going to describe them, as they are the subject of other bachelor thesis. However it is worth to mention the functionalities they provide: the first hyperlink leads to file exchange system, while the second one redirects user to the e-learning module. Similarly to message board, these features are unavailable to parents.

The penultimate link causes subject's description to be displayed. From the point of view of students and parents, this is just a simple HTML page containing brief information on given subject. Professors are also allowed (and encouraged) to modify the description section.

Last but not least, the participants list. This feature remains unchanged for every type of user, including students, teachers and parents. It is a simple list of all people attending the subject (in fact, the same group of people is allowed to use respective message board) including their profile photographies, names and

e-mail addresses, generated on the basis of database entries. The functionality was designed to ensure easy communication between the participants. Even though parents are not directly connected with the subject, they are still able to access the list of participants. Therefore, they can see their children's classmates and teachers and contact them, if such need exists.

3.2.3. Application security

Handling large amount of private data obviously introduces the necessity to maintain a high level of security. This broad term can be split into few more specific aspects. From the perspective of CampusNet project, these include:

- ◇ Recognition of data accessibility and authorization rights – given the feedback from high school's administration staff and having briefly discussed the issue with a lawyer specialized in the IT field, we designed the application panels to show only the data that should be available for the given user. Only the administrator has permissions to see others' private information. All the remaining types of the users are only provided with the data concerning themselves. Also, everyone can see other's contact information, however a person may refuse to include it in the database. For obvious reasons, parents are able to see their children's marks, attendance and subject data, but for instance, messages remain private. Obviously, the database connection itself is secured by a password and no one can access it otherwise than through the CampusNet platform.
- ◇ Validation of the input forms – every field that requires the user to provide some data needs to be checked for errors. One can distinguish two main reasons for invalid input – first being a simple mistake, whereas the other one is aimed at compromising the application on purpose. Regardless of the reason, we designed the forms to immediately give user the feedback about inappropriate input (or its lack whatsoever in case of obligatory fields), usually by highlighting them and showing short, yet descriptive message. The form validation is provided both on the client side (for more dynamic response, including animations) and on the server side.
- ◇ Logging – operations such as giving marks to the students, updating their attendance, etc. deal with the most crucial and vulnerable data. In such cases, the fact that the modifications can only be made by authorized entities is not enough. There is always a threat of someone's password being stolen or that the user forgot to log out and another person used his/her account. To solve these issues, a logging system is necessary. Even though it is not yet introduced, we plan to implement it (see future plans in Chap. 5) for the marks and attendances, so that any modification being made is immediately saved in a log, along with information about the date of occurrence and the user responsible for it.
- ◇ Invulnerability to attacks – one has to always consider the possibility of being exposed to the hacking attempts. Fortunately, Django framework offers many protection mechanisms against the most common threats. For instance, as we were able to completely omit the native SQL queries, there is no possibility to perform the SQL injection attack, which is based on altering the HTTP request data and providing destructive code to be a part of the query executed on the server. Also, Django takes care of the group of

attacks known as Cross-site Request Forgery (CSRF). The idea behind it is to exploit the website already after the user is authenticated, thus skipping the authorization phase. This attack is prevented, because to every web form, Django automatically adds a hidden input field with a special session key. Moreover, one of the most common attacks is related to cross-site scripting. Theoretically, the user could enter a JavaScript code in one of the forms and have it interpreted by the browser, causing for instance user's private data to be sent to the hacker. Fortunately, if not specified otherwise, when the data is embedded in Django's HTML template, it is transformed into an escape sequence, meaning that all the tag-specific characters are changed into entity references.

- ◇ Data enciphering – without introducing such mechanism, it would be possible to capture the packets sent between the client and the server and extract private data from them. We have dealt with this issue by implementing a Secure Shell Layer (SSL) in Apache HTTP server (See Sec. A.2).

To sum up, we can say that we have created an integral and secure system that is resistant to the common hacking strategies and does not give the user a chance to make an error-triggering mistake, regardless whether it is made on purpose or accidentally.

3.2.4. Cross-browser compatibility

Currently, one can distinguish five leading applications for surfing the Internet. They are listed below along with the average percentage of the users [1]:

- ◇ Mozilla Firefox – 42.58,
- ◇ Internet Explorer – 25.80,
- ◇ Google Chrome – 24.63,
- ◇ Apple Safari – 4.05,
- ◇ Opera – 2.52.

To render and display the web content, a browser needs to use some layout engine. There are four major engines that support the visual presentation of the WWW pages for the aforementioned browsers:

- ◇ Gecko – Mozilla Firefox,
- ◇ Presto – Opera,
- ◇ Trident – Internet Explorer,
- ◇ WebKit – Google Chrome and Apple Safari.

By nature, our system is designed to be used by a wide variety of users with possibly big diversity among the Internet browsers they prefer. The necessity to provide the same conditions for each person connected to the school implies that our design has to be consistent regardless of the browser and its layout engine. Unfortunately, these applications differ slightly in the interpretation of HTML, CSS and JavaScript. There is an organization called W3C (World Wide Web Consortium) that is fully committed to develop universal web standards, but still not all of the browsers comply to their suggestions in every aspect. Also, it is not so easy to find

[1] Statistics taken from `http://www.w3schools.com/browsers/browsers_stats.asp`

hype-free assessments of the major browsers' CSS and JavaScript capabilities, as well as their adherence to the W3C standards.

One can distinguish two main types of compatibility issues. These are (ordered by their descending priority):

- Functional – providing that the system is equally informative and returns the same essential data for the related use case.
- Visual – assuring that the website looks exactly the same, thus its appearance is browser-independent.

Fortunately, functional incompatibility problems are far less likely than the ones from the other category. In fact, the issue whether the web developer should struggle to maintain a complete visual compatibility is debatable. The more sophisticated the design is, the more difficult it is to present it in the exactly same way with use of different layout engines. There are claims that one should rather focus on tweaking the appearance consistency throughout a single browser and accept the fact that there will be slight differences when one compares the website on multiple browsers. Nevertheless, we decided to try and make CampusNet compatible in both of aforementioned fields, as long as it does not prove to be too problematic. For instance, some advanced features of web styling (i.e. CCS 3.0) are not supported by all the browsers (even though nowadays it specifically applies to the older versions of Internet Explorer). Thus, to get the identical results on the non-supporting ones, we would have to come up with a set of images that depict the same representation. Using pictures instead of style coding would prove to be time-consuming, especially considering that we are not graphic designers. Moreover, it consumes more bandwidth and produces more traffic on the Internet, but it still would not influence the legibility of the data presented on the website, therefore we decided to omit these features in some of the browsers.

Using our application, the best results one can observe in Google Chrome and Apple Safari. In Mozilla Firefox and Opera, some user interface's elements may look a little unique, however it does not compromise the full functionality of the website. As for now, the visually consistent design and full manageability of our project in not yet implemented for Internet Explorer versions lower than 9.0. This is mostly due to different interpretation of the CSS box model (especially considering `padding` attribute and `relative` / `absolute` positioning) and lack of CSS 3.0 features, such as:

- opacity / transparency,
- rounding corners,
- box shadow,
- text shadow,
- gradients.

Nowadays, there is a strong trend that advises against developing websites in Internet Explorer first and states that sites should be developed in "modern" web-browsers, with standards first and then be tweaked for buggy versions of Microsoft's browsers. We have also adapted to such behavior and therefore, Internet Explorer adaptation of our project is listed as a future assignment (one can find more about future plans in chapter 5).

3.2.5. Internationalization and localization

Internationalization refers to the process of designing programs for the potential use of any locale. This includes marking text (such as UI elements and error messages) for future translation, abstracting the display of dates and times so that different local standards may be observed, providing support for differing time zones, and generally making sure that the code contains no assumptions about the location of its users. One can often see "internationalization" abbreviated I18N (The "18" refers to the number of letters omitted between the initial "I" and the terminal "N.").

Localization refers to the process of actually translating an internationalized program for use in a particular locale. It can sometimes be abbreviated as L10N.[2]

As we would like to extend our project to reach the greater audience, the need for internationalization and localization is inevitable. Fortunately, Django framework comes with a very powerful set of tools that support the aforementioned. There are several steps that one must follow in order to create a multi-lingual version of a website:

- ◇ Tag all the locale-sensitive strings for translation – this is typically done in two ways, dependent on the place that we introduce it in. If the text we would like to translate is in the Django views, we use the method `ugettext` that – due to the extent of its usage – is often labeled as a single sign - "_" [3]. On the other hand, it is also possible to mark strings for translation in the HTML templates. First thing to do in such case is to load the i18n module for the web page. It is done by placing `{%load i18n%}` in the HTML code. Having done this, we are able to use function `trans` in such way: `{%trans "text that will be translated"%}`. Django's translation machinery uses `gettext` function[4] via the `standardgettext` module that comes with Python.
- ◇ Generate language files – after all the locale-dependent strings have been marked, the language need to be made. There is a special module incorporated in the standard Django library that scans the files in a given project, recognizes all the strings marked for translation and produces a language file for the chosen locale. It contains the groups of text that have to be translated and placeholders, where the translations should be put.
- ◇ Fill and compile language files – for the sake of efficiency, being filled with translations, the language files should be compiled into a binary form that significantly increases the speed of dynamic translation.

After all of the above steps are completed, the application can already be presented in the translated version. In general, language the website is presented in depends on the default settings of a web browser. Nevertheless, the logical solution would be to additionally provide the user with the possibility to choose the version he/she wants to see. To do thusly, we need to add

[2] Definitions taken from: `http://djangobook.com/en/2.0/chapter19/`

[3] in such situations, Django's labeling mechanism comes in handy. During the import of the function, we can specify its alias, i.e. `from django.utils.translation import ugettext as _`

[4] See official webpage: `http://www.gnu.org/software/gettext/`

the i18n URL dispatcher to our project's standard address schemas and develop a sort of web form that sends the language code[5] with a `POST` method to `address-of-the-project/i18n/setlang/`.

At this stage of development, CampusNet is destined to be used in English. Nevertheless, it is also possible to examine the Polish version of the system, by choosing this language from the authorization panel[6]. Even though the translation itself is not yet fully implemented, the engineering process has been done. All the essential strings are marked for localization and the message files have been generated. What is left, is to complete the translation strings in those files, but this vague activity is out of the scope of this thesis.

[5] Language codes: `http://www.i18nguy.com/unicode/language-identifiers.html`
[6] It is done by clicking Polish flag situated in the top-right corner of the panel.

4. Testing

Software testing may be defined as an activity targeted at evaluation of certain program's capabilities and determination, whether it does fulfill assumed standards. Testing shall not be confused with debugging, as it covers remarkably wider area. This issue happens to be a crucial one when it comes to software quality assurance and an inevitable part of each project development. However, due to the fact, that usually it is being performed during final stages of work, developers tend to disregard its significance. This trend may be explained easily: along with approaching deadlines, people are often forced to resign some previously established assumptions; in this case testing part seems to be the most tempting solution. As can be expected, it is a bad choice, that can (and most probably will) become a cause of future complications. So as to avoid aforementioned, we designed, implemented and performed a series of tests, results of which are presented in the following paragraphs (for installation clues, see: App. D).

4.1. Methodology

There exist various approaches to the problem of software testing. Techniques implemented in order to solve this task may be divided in several ways, taking into account the character of testing, its range, aim or entity conducting it. Below we list some exemplary divisions:

Division with respect to insight into the code:

- ◇ structural (white-box) tests,
- ◇ functional (black-box) tests,

Division with respect to automation of work:

- ◇ manual tests,
- ◇ automatic tests,

Division with respect to the method of conducting:

- ◇ unit tests,
- ◇ integration tests,
- ◇ system tests,

Division with respect to purpose of the test:

- ◇ acceptance tests,
- ◇ functional tests,
- ◇ regression tests,

Other commonly used divisions:

- ◇ installation and configuration tests,
- ◇ alfa and beta tests,
- ◇ usability tests,
- ◇ post-failure tests.

We approached to select the most appropriate methods from among different types of software testing. Thus the decision was made to examine our project with use of the following techniques of testing:

- ◇ functional testing – determining whether software works accordingly to its specification,
- ◇ regressive testing – checking, if recently added functionality does not accidentally affect any other functionality,
- ◇ usability testing – test performed to estimate how fast can new users master the use of software etc.

Each testing type was supposed to be performed both automatically and manually. This decision forced us to employ some additional tools, i.e. Selenium testing framework and Django built-in testing module. Below we present brief description of mentioned technologies.

4.2. Tools

Selenium is a portable framework for software testing of web applications. Its major feature is a record/playback functionality, that allows to perform automatic tests without learning a test scripting language.

Django built-in testing module offers two way to define a test: as a doctest (by locating test in function's docstring, which are actually Python feature) or as a unit test. We decided to use the second option, results of which are presented in further paragraphs.

4.3. Testing the application

Having solid theoretical basis on testing issues, we may now proceed to the presentation of actual tests employed to the platform. As stated before, CampusNet was subjected to functional, regressive and usability tests, results of which (along with process of conducting) are briefly described below.

We are aware of the fact, that our system requires even more tests and those already performed are not sufficient to let us consider the testing phase as closed. Nonetheless, at this stage, the most important thing was to familiarize with related technologies and to try them out. Having achieved this goal, we can easily extend the testing just by following already learned patterns. Future plans (see Chap. 5) assume further trials within this area.

4.3.1. Functional testing

This type of testing is performed so as to confirm, that the software follows the expected routine specified in the documentation. Functional tests may be performed manually, but we decided to automatize the process using aforementioned Selenium framework. The procedure for test's creation usually looks like the following:

1. Performing the test manually, recording it with use of Selenium record functionality.
2. Automatic translation from Selenium language to Python code.
3. Manual enhancement of the Python code.
4. Placement of the code into desired file (in our case the code is stored within the `.../functional_tests/tests.py` file).

Having completed these steps, one is able to perform the functional testing automatically, provided the Selenium tool is installed on his/her machine. We designed three different tests for the LMS module, while two other were produced for VLE. The LMS routines are presented below:

Event test:
1. Login.
2. Follow the events hyperlink from personal menu.
3. Add new event.
4. Reload page.
5. Delete recently added event.

E-register test:
1. Login.
2. Follow the e-register hyperlink from personal menu.
3. Follow the attendance hyperlink.
4. Mark all students as present.
5. Reload page.
6. Mark all students as absent.

Subject's description test:
1. Login.
2. Follow the description link from subject menu.
3. Edit the description.
4. Reload page.
5. Check if the recently added description is present.
6. Edit it again, restoring the previous version.

One important note: user shall be aware of the fact that this testing method performs operations on actual instance of the database (it does not introduce a provisional database as in case of regressive testing). Consequently, one should pay attention to potential data lost. Another implication resulting from this fact is the need of feeding initial data to the database.

4.3.2. Regressive testing

One of the most significant kind of testing in terms of assuring decent quality of software. Regressive testing is aimed at determining, whether newly added function does influence (in a negative way) already existing ones. Thanks to this approach, such undesirable interaction may be immediately tracked and eliminated.

With use of Django test-execution framework we designed and conducted a set of unit tests, validating performance of several functions. Tests for each module are stored in `tests.py` files corresponding to each panel; they examine the following features of CampusNet platform:

Teacher panel:

- addition of new lesson in course curriculum,
- deletion of a lesson,
- edition of subject's description,

Student panel:

- addition of a new post to subject's board,
- deletion of a post,
- sending a new message,
- deletion of a message,

Parent panel:

- logging into the system,
- loading the welcome page.

The tests mentioned above are uncomplicated, which is determined by the definition of unit testing. It is worth mentioning, that in this case the actual database remains unchanged. For the time of test execution a provisional database is created and fed with initial data defined within tests' files. After the termination of this process, all temporary objects are completely destroyed.

4.3.3. Usability testing

This type of testing happens to differ slightly from regressive and functional ones, as it cannot be performed automatically. On the contrary, it demands involvement of users, that will examine the system from their point of view and support developers with their thoughts regarding for example platform's usability or interface intuitiveness.

When it comes to web application design, end-users' feedback cannot be overestimated. Thanks to the cooperation of a group of teachers, which will use our platform in the future, we were able to conduct also this kind of testing. Usability testing actually began when the first version of CampusNet was placed on the remote server, giving the teachers possibility to test it and share opinions. This allowed us to introduce desired changes from the very beginning of project's development.

5. Relevant problems and future development

5.1. Future development

Despite the fact, that CampusNet platform can already successfully support operation of a high school, the team has decided to continue project's development after submitting bachelor theses. We are aware, that some existing solutions employed in the system may be optimized, as well as we have some fresh ideas to be implemented in the future. These issues form the main subject of the following section – a discussion on forthcoming CampusNet modifications.

5.1.1. Optimization

Let us commence with possible improvements of living elements. As it was mentioned in Sec. 3.2, some visual issues are likely to appear while accessing the system through various browsers. Therefore, one of the first matter to be attended is the cross-browser compatibility. Even though it remains on a decent level at current stage of project development, we would like to approach to perfection, maximizing users' comfort. Thus, elimination of slight graphic ambiguities and assurance Internet Explorer compatibility are listed in our future development outline.

In our opinion, also the message exchange module may benefit from minor improvements. Surely a possibility of sending group messages with a single click (e.g. teacher selecting class 1C as a recipient) would be a useful supplement, as the necessity of adding every recipient separately may appear inconvenient to users.

Moreover, the team is considering implementation of an extra relation between attendances and lessons. An obvious solution is to add a many-to-many field to `Lesson` model, however it would imply an obligation to create a `Lesson` (specifying its subject etc.) before checking students presence, which happens to be an undesirable effect. This issue ought to be solved in near future.

Finally, we are going to extend the set of available languages. At this moment, CampusNet is available in English and prepared for Polish localization. Nonetheless, this may be easily changed – as the addition of another language is trivial (thanks to Django) from the programmer's point of view, only several hours of translator's work is needed so as to accomplish the task.

5.1.2. Functionality extension

Besides the aspirations for optimization of existing elements, the team possesses also some thoughts on new features that are to be implemented. Part of them was mentioned in requirements analysis (Sec. 2.2), prioritized as optional.

The aforementioned list contains - among others – automatic timetable generation, an interesting feature supporting process of creation of timetables. It should be pointed out, that designing such algorithm will not be trivial, at least if it is supposed to be efficient. Due to the fact, that it requires a great number of aspects (e.g. availability of teachers, their personal preferences, balancing the workload etc.) to be taken into consideration, its implementation may consume a large amount of time.

SMS notification of upcoming events is another issue, on the topic of which the team had some discussions. We believe, that such module will be highly useful for the entire school's community, still we prefer to wait with its development until CampusNet is successfully introduced in a real high school.

Moreover, there is one drawback that results from using AJAX extensively in the teacher, student and parent panels. Changing website's content without reloading the whole page is efficient and improves the appearance, but proves to be problematic in terms of navigation. In our application, web browser's "back" button does not function as one would consider intuitive. In general, one could state it is useless. This behavior may be considered inappropriate, even though it is easy to get used to our alternative approach to navigation. Nevertheless, there is a solution to this problem (involving use of invisible HTML `iframe` tags) and we will implement it in the future version.

Other anticipated supplementary functions are listed below:

- ◇ logging system – for the sake of security, as described in subsection 3.2.3.
- ◇ uploading photographies – current version of the system is already prepared for this feature,
- ◇ uploading homeworks & projects – efficient exchange of school tasks; submitting with obedience to automatically checked deadlines,
- ◇ animated statistics – visualization of students' performance available also to their teachers and parents,
- ◇ Android application – VLE module is to be extended to work properly on mobile devices,
- ◇ extended testing - using techniques mentioned in Chap. 4.

5.1.3. Graphical User Interface

Even though we are satisfied with the visual aspect of actual CampusNet version, we believe that there exist always a space for improvement. Thanks to the professional advice from a member of usability research company, we received some feedback concerning system's layout.

Finally, the team desires to give the users the capability of adding a personal touch to their accounts. Consequently, the concept of modifiable web-skins will be implemented within next versions of the system.

6. Conclusions

6.1. General conclusions

The first and most important conclusion refers to the objectives of the project, previously described in section 1.2. It may be stated that the team managed to satisfy all of them – we have created a completely functional system supporting operation of a high school. Moreover, all functional requirements marked in section 2.2 as mandatory or recommended (except for the event notification, however this can be easily achieved with use of message system) were successfully implemented. Also three out of five optional requirements are available for CampusNet users.

Likewise, non-functional requirements were treated with equal attention. Every user may easily access the system via Chrome and Safari web browsers, without obligation to perform any additional installation processes. Implementation of SSL guarantees data security, similarly to other methods mentioned in section 3.2. MySQL database allows us to store relatively large amount of data, maintaining it integrated, confidential and available.

Finally, integration of the LMS and VLE modules was conducted successfully, coherently joining every subpart of the CampusNet platform.

6.2. Conclusions on technology selection

The technology selection part appeared to be particularly important, as choices made at this stage were essential in terms of future work. For example, making the proper decision regarding framework may spare a lot of time, while wrong selection could turn coding part into a real torture. Frankly speaking, that is what partially happened in our case – we started to code the system with one framework, but as our knowledge was expanding, we realized that there exist more efficient solutions. This led us to carrying out another analysis, which eventually resulted in framework's change. Even though such transition required some effort, the outcome finally proved the correctness of our decision.

Another conclusion drawn from project's development refers to the Python itself. We were surprised by the intelligibility, versatility and usefulness of this programming language. Another impressive Python's characteristic is its legibility. Due to the fact, that block structures in this language are defined by indentation, one is forced to organize his/her code in a decent way. Finally, it seems that our positive thoughts on Python are shared by a number of respectable companies. Among the list of software written in this language, we may mention Trac, Mercurial, Dropbox, BitTorrent (original client) and even some video games, such as *Civilization IV* or *Battlefield 2*.

As the Learning Management System is in fact the subject of a bachelor thesis, it has to be documented in a written form. In order to accomplish task, we decided to

use Latex, a WYSIWYM editor, which proved to be very useful for such application. In our opinion it handles long documents (i.e. consisting of dozens of pages) better than the popular editors such as MS Word or Open Office. Even though it requires some effort to master the program, benefits it provides are worth their price.

6.3. Conclusions on project development

Regardless of the type of school, designing a system aimed at providing support for this institution happens to be a very compound task. Its complexity results directly from the fact, that a real-life school environment consists of enormous number of mutually connected elements, including human beings (teachers, students, parents, administrative staff), material objects (facilities, rooms, equipment), items existing due to the structure of educational system (attendances, grades, lessons, timetables) or events connected with school life (trips, sport events, exams).

All issues mentioned above influence also the database design. So as to guarantee software support for the institution, all of its main elements and the relations between them have to be reflected in the database. A proper design of aforementioned seems to be a compulsory step to be done before even commencing the coding part. Nonetheless, as we have learned, it is almost impossible to complete the project without correcting the database design. Even tough we devoted a significant amount of time to plan it properly, some minor fixes that arose during project's development were just inevitable.

Web development is an amazingly wide area, in which there exists plenty of free-licensed operating solutions. According to the rule stating that 'there is no point in reinventing the wheel', we did not hesitated to take advantage of existing open-source plugins. Using (sometimes with some modifications) such solutions as Tokeninput, Datepicker, Colorbox or jMonthCalendar expanded our system with additional features, simultaneously saving us a lot of time and effort, that might have been utilized in a more efficient way.

Another conclusion results from the fact, that we were participating in a group project. Consequently, the team was obliged to familiarize with a certain kind of software that facilitates collective work. This is undoubtedly an invaluable experience, that will surely bear interest in our professional careers. Moreover, working in a decent team with good spirit ensured mutual mobilization and great satisfaction after termination of the project.

A. Installation manual

While for project developers issues related to usage of their own design are obvious, other people may - and most probably will - encounter some obstacles on their way to familiarize with the system. Each design, even the one equipped with extremely intuitive interface, requires user manual that allows everyone to study about program's capabilities, learn where to find certain functionalities and how to use implemented features. Below we present effects of our work, dedicated to creation of a complete reference to system's functionalities for CampusNet users.

A.1. Setup workflow

This process depends heavily on the demands of person performing it. The CampusNet project does not require any setup for common users; the only thing they ought to be equipped with is Google Chrome or Safari (and Firefox/Opera in the future) web browser and Internet connection.

Nonetheless, we expect that some people will be interested in employing a local version of the platform (e.g. the reviewer of this thesis), which requires a set of certain tools to be installed on a local computer. In order to facilitate this task, the usual workflow of installation process has been divided into several steps described below.

The whole procedure was performed on Ubuntu OS (Maverick Meerkat, release 10.10, x86 32-bit version). Usage of this operating system is strongly recommended, as the setup process will match exactly the one presented in the manual.

Step 1

- ◇ software: Ubuntu OS,
- ◇ version: release 10.10, i386 (x86, 32-bit version),
- ◇ from: Polish Ubuntu Forums,
- ◇ url: http://ubuntu.pl/pobierz.php

The first step is to employ Ubuntu OS on local computer. It may be downloaded as an ISO file from the aforementioned web page. There is no obligation to install it on the machine, as Ubuntu can be run from USB stick or live CD (in case of optical drive, however, all changes are lost after reboot, which proves uselessness of that approach). More detailed tutorial on this step is available on the website.

Step 2

- ◇ software: Python interpreter,
- ◇ version: 2.6.6,
- ◇ from: existing within default Ubuntu installation,

Not much to add in this place, as Python interpreter works correctly right after loading the system.

Step 3

- ◇ software: Django framework,
- ◇ version: 1.2.5,
- ◇ from: project official website,
- ◇ url: http://www.djangoproject.com/download/1.2.5/tarball/

As can be expected, Django web framework is required to establish the local version of CampusNet platform. To setup this tool successfully, it is necessary to unpack the archive file, proceed to newly created directory and execute in terminal the following command:

```
1 sudo python setup.py install
```

Listing A.1. *Command allowing Django installation.*

Step 4

- ◇ software: SetupTools (Python Distutils Enhancements),
- ◇ version: 0.6.14,
- ◇ from: Synaptic Package Manager,

Obligatory set of extensions, available from the level of Synaptic Package Manager.

Step 5

- ◇ software: MySQL,
- ◇ version: 5.1.49 (server),
- ◇ from: Linux repositories,

Installation of MySQL database is also necessary. It may be obtained easily with usage of terminal by executing two commands:

```
1 sudo apt-get install mysql-client
2 sudo apt-get install mysql-server
```

Listing A.2. *MySQL client & server version installation.*

During the setup of server-side version, user will be prompted to set the root password (let us refer to it as `root_pwd`) for database. The same password will be used also for other purposes.

Step 6

- ◇ software: MySQLdb,
- ◇ version: 1.2.2,
- ◇ from: Synaptic Package Manager,

MySQLdb is a Python interface to the popular MySQL database server. After the installation, it is obligatory to create a MySQL database. For this reason, one should execute within terminal the following command:

```
1 mysql -u root -p
```

Listing A.3. *Running MySQL.*

The user will be prompted to enter `root_pwd` here. Then, within MySQL shell, one should create a new database and grant all privileges to his/her account by entering:

```
1 mysql > CREATE DATABASE campusnet CHARACTER SET 'utf8';
2 mysql > GRANT ALL ON campusnet.* TO campusnet@'localhost'
3     IDENTIFIED BY 'root_pwd';
4 mysql > exit;
```

Listing A.4. *Creating new MySQL database.*

The next step is Django's `settings.py` file edition. In order to ensure coherence with database user information, the value of `PASSWORD` field in `DATABASE` section should contain `root_pwd`.

Finally, having completed aforementioned, one shall synchronize the database by executing appropriate command in the terminal:

```
1 python manage.py syncdb
```

Listing A.5. *Django database synchronization.*

Because of the fact, that the synchronization is performed for the first time, user will be prompted to create a superuser account. From now on, after the execution of runserver command (presented below), our application will be available under `localhost:8000/` address.

```
1 python manage.py runserver
```

Listing A.6. *Running Django development server.*

Step 7

- ◇ software: ReportLab Toolkit,
- ◇ version: 2.4 (i386),
- ◇ from: Linux repositories,

The ReportLab Toolkit happens to be last piece of software required for proper functioning of CampusNet platform. This need has been induced by implementing PDF document creation in our project. So as to install the tool, it is necessary to execute the following command in terminal:

```
1 sudo apt-get install python-reportlab python-reportlab-accel python-renderpm
```

Listing A.7. *ReportLab Toolkit installation.*

After completion of the sequence of steps mentioned above, one receives a fully-functional local instance of CampusNet platform. At this stage of project development, we recommend to use Google Chrome browser in order to ensure the proper display of the content. Nonetheless, one of the first future improvement will be to introduce a full cross-browser compatibility.

A.2. Apache configuration

While for the sake of project development Django built-in development server handles the task perfectly, the actual working instance of CampusNet requires

also HTTP server. This time we did not have any doubts regarding technology selection – the decision to use popular open source Apache server was made unanimously. This paragraph contains detailed description of its installation, configuration, integration with Django (steps 1-4) and securing it with Secure Shells Layer protocol (steps 5-7).

Step 1

- ◇ software: Apache HTTP server,
- ◇ version: 2.2,
- ◇ from: Apache Project website,
- ◇ url: http://httpd.apache.org/

The first step on the way to accomplish the setup is to download and install Apache HTTP server. After downloading the `tar.gz` file, the following sequence of commands allows to finish this part of configuration:

```
1 tar xvf httpd-2.2.17.tar.gz
2 cd httpd-2.2.17
3 ./configure
4 make
5 make install
```

Listing A.8. *Apache HTTP server installation.*

Important note: it may occur that the `./configure` commands needs to be run with some options (e.g. due to the fact, that some tools such as `apxs` may be placed in a non-default location). In our case, it was necessary to execute `./configure` with two additional parameters, namely `--with-apxs=apxs_filepath` and `--with-python=python_filepath`. The server may be started by executing:

```
1 ...apache2/bin/apachectl start
```

Listing A.9. *Starting Apache server.*

Step 2

- ◇ software: mod_wsgi adapter,
- ◇ version: 3.3,
- ◇ from: Debian packages website,
- ◇ url: http://packages.debian.org/

The `mod_wsgi` adapter is a module that enabled us to store Python based web application within Apache server. Initially, there existed some doubts regarding choice of the module (we were also considering employment of `mod_python`), but soon it turned out that the second one is not under active development anymore, which solved the selection problem. The adapter's setup is analogical to the one described in the previous step, including `./configure` optional parameters:

```
1 tar xvf mod_wsgi-3.3.tar.gz
2 cd mod_wsgi-3.3
3 make
4 make installation
```

Listing A.10. *mod_wsgi adapter installation.*

Now, newly installed module should be also loaded into Apache:

```
# LOAD mod_wsgi
LoadModule wsgi_module modules/mod_wsgi.so
```

Listing A.11. *Loading mod_wsgi into Apache.*

Step 3

At this point, the installation of necessary tools is ready, the only remaining issue is the appropriate configuration. This step should begin with copying CampusNet files to the server, for instance into `/home/user/django_project/` folder. The next task is to create apache directory inside project folder and add `django.wsgi` script:

```
cd /home/user/django_project/
mkdir apache
touch django.wsgi
```

Listing A.12. *Adding Django.wsgi script.*

Step 4

This point concerns edition of `django.wsgi` and `httpd.conf` files that ought to be done by adding following lines:

In `django.wsgi`:

```
import os, sys
sys.path.append('/home/user')
sys.path.append('/home/user/django_project')
os.environ['DJANGO_SETTINGS_MODULE'] = 'django_project.settings'
import django.core.handlers.wsgi
application = django.core.handlers.wsgi.WSGIHandler()
```

Listing A.13. *Editing django.wsgi file.*

In `httpd.conf`:

```
# js, css, image django folder
Alias /site_media/ /home/user/django_project/static/

# django administration panel styles
Alias /media/ <django_filepath>/django/contrib/admin/media/

<Directory /home/user/django_project/static>
  Order deny,allow
  Allow from all
</Directory>
<Directory <django_filepath>/django/contrib/admin/media/>
  Order deny,allow
  Allow from all
</Directory>

# wsgi script addition
WSGIScriptAlias / /home/user/django_project/apache/django.wsgi

# changing the destination catalog for unpacking archived modules as Apache does
# not have access to the default /var/ one
WSGIPythonEggs /tmp/
```

```
22
23 <Directory /home/user/django_project/apache>
24   Order deny,allow
25   Allow from all
26 </Directory>
```

Listing A.14. *Editing httpd.conf file.*

Having completed the aforementioned procedures, one is now able to either run Apache server unprotected or configure the security measures. In the former case, one can skip steps 5 - 7 and immediately jump to step 8.

Step 5

Future steps (including this one) will only focus on protecting Apache with Secure Sockets Layer (SSL) protocol. Steps 5-7 are optional, however they are highly recommended due to security reasons and to maintain proper information privacy level.

The configuration of the HTTPS protocol requires generation and installation of the SSL keys. The keys have to be generated using OpenSSL program, thus first step is to install it:

```
1 sudo apt-get install openssl
```

Listing A.15. *Installation of OpenSSL.*

Step 6

It is obligatory to produce the keys that are used to secure the site. These keys are used when encrypting and decrypting the traffic in the secure connection. A 1024-bit private key can be generated using command:

```
1 openssl genrsa -out campusnet.key 1024
```

Listing A.16. *Generation 1024-bit private key using OpenSSL.*

A certificate is a validated public key. It is used by the secure Web server to prove its identity to the remote Web browser (this is the 'validated' part), and then it is used by the remote browser to encrypt the traffic to the Web server (this is the 'public key' part). The certificate is validated by signing it. In case of the CampusNet configuration, the certificate will be self-signed. A self-signed SSL certificate can be created using command:

```
1 openssl req -new -key campusnet.key -x509 -out campusnet.crt
```

Listing A.17. *Creating self-signed certificate using OpenSSL.*

The above process would ask for the basic properties of the certificate, such as country code, province name and the administrator e-mail. Now, the key and the certificate need to be placed in proper directories inside the Apache 2 configuration directory:

```
1 mkdir /usr/local/apache2/conf/ssl.key
2 mv campusnet.key /usr/local/apache2/conf/ssl.key/
3 mkdir /usr/local/apache2/conf/ssl.crt
4 mv campusnet.crt /usr/local/apache2/conf/ssl.crt/
```

Listing A.18. *Moving SSL certificates to appropriate directories of Apache.*

Step 7

Last stage of securing Apache server is to add SSL directives to its configuration in `httpd.conf`. It assumes, that the server host name of the CampusNet installation is `campusnet.com`.

```
#### SSL CONFIGURATION BEGINNING
NameVirtualHost *:443
Listen 443

<VirtualHost *:443>
ServerAdmin admin@campusnet.com
DocumentRoot /home/user/django_project/
ServerName campusnet.com
SSLEngine on
SSLCertificateFile /usr/local/apache2/conf/ssl.crt/campusnet.crt
SSLCertificateKeyFile /usr/local/apache2/conf/ssl.key/campusnet.key
</VirtualHost>

<Directory /home/user/django_project/>
SSLRequireSSL
</Directory>

<VirtualHost *:80>
ServerAdmin admin@campusnet.com
DocumentRoot /home/user/django_project/
ServerName campusnet.com
RedirectMatch permanent (/.*) https://campusnet.com$1
</VirtualHost>
#### SSL CONFIGURATION END
```

Listing A.19. *SSL configuration for Apache HTTP Server.*

Step 8

The Campusnet application cannot operate properly until the Apache server is not started. Therefore, the last step is the following:

```
cd /usr/local/apache2/bin/
apachectl start
```

Listing A.20. *Starting Apache server.*

Also, if one has started the server before, one should rather use this command:

```
apachectl restart
```

Listing A.21. *Restarting Apache server.*

As the result, there should be running an Apache server with the CampusNet platform (also if steps 4-7 were performed, it would accept only secure HTTPS connections).

B. Administrator manual

This appendix is aimed at providing vital feedback for CampusNet administrators, explaining how to use efficiently the administrator panel. It lists the functionalities available from within aforementioned panel, along with brief descriptions and useful hints.

B.1. Main menu

The administrator panel authorization page is loaded after clicking the `Administrator` hyperlink on CampusNet welcome page (see Fig. C.1 in Sec. C.1). After inputting correct login information, one is redirected to panel's main page (with `Students` tab selected by default). Below we list the most important elements of this page (Fig. B.1):

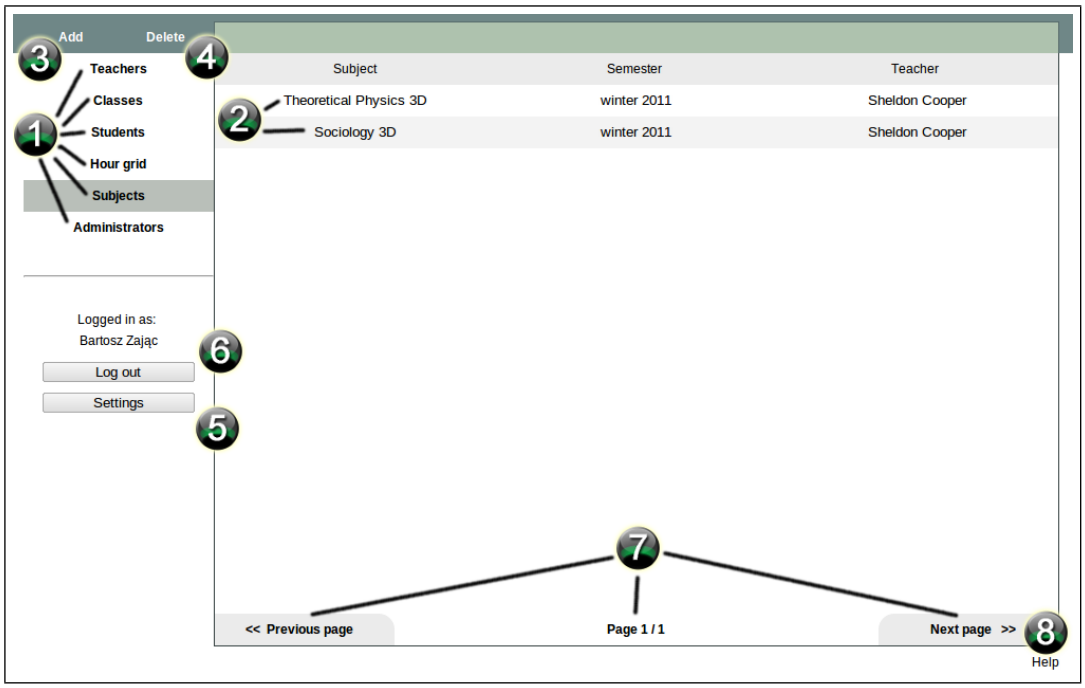


Figure B.1. *Main page of the administrator panel.*

- (1) Submenu allowing to choose certain type of data to be modified. The names here are self-explanatory; Fig. B.1 presents exemplary tab (`Subjects`) being selected.

- ◇ (2) Within this area, a list of certain items is displayed. Depending on the currently opened tab, the number of columns, as well as their labels, may differ.
- ◇ (3) `Add` action button, causing appropriate form (e.g. Fig. B.2) to be displayed. Again, the structure of the form may differ, dependent on the tab.
- ◇ (4) `Delete` action button featuring the removal of certain elements.
- ◇ (5) `Settings` button, redirecting the user to settings module of administrator panel (see Fig. B.4).
- ◇ (6) `Log out` button.
- ◇ (7) Pagination menu, used for the sake of page browsing in case of large number of database entries.
- ◇ (8) `Help` hyperlink – inactive at this stage of project development.

B.2. Adding new entry

Hitting the `Add` button causes an appropriate form, related to the currently open tab, to be displayed on the screen. After filling it in with required information (necessary fields (1) are marked with a red asterisk) and clicking on the `Save` button (4), the database is being feed with new entry.

As the exemplary addition form is presented for `Subject` tab, it contains two additional elements – the list of all students (2) eligible to be signed up for given course and the list of already inscribed people (3).

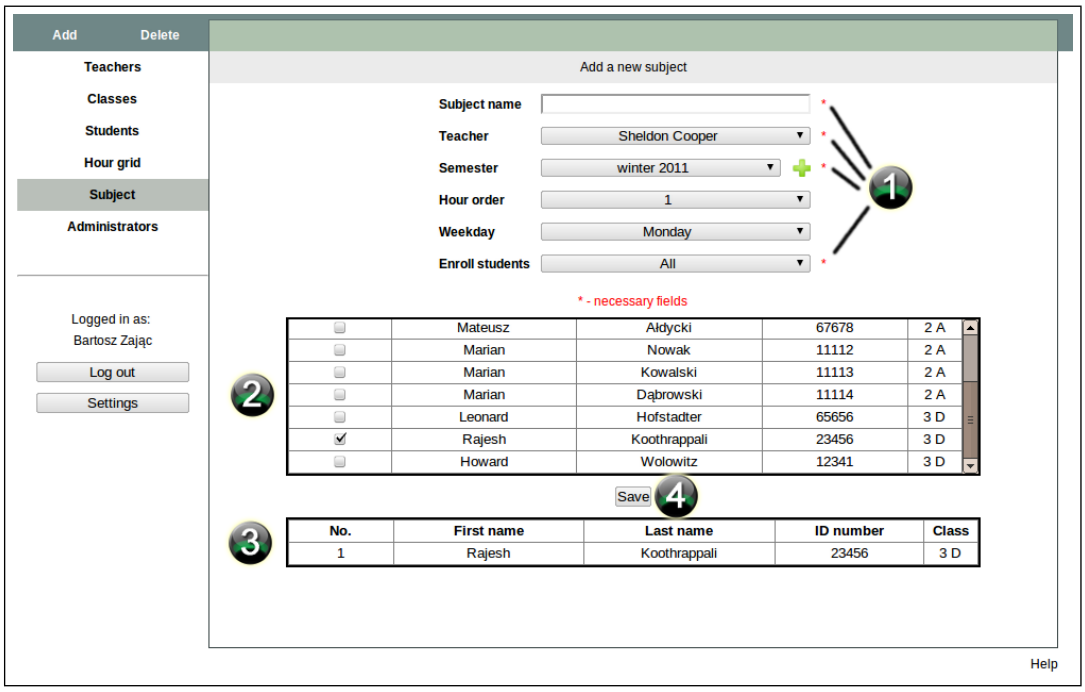


Figure B.2. *Form allowing creation of a new subject.*

B.3. Form validation

During the process of adding new entries, administrator panel takes advantage of form validation (Fig. B.3), so as to guarantee maintaining a coherent and correct database. This process may be observed after clicking on the `Save` button, while the input form remains incomplete (in this case, one necessary field (1) seems to be proper, but the other (2) is empty). In such case, a suitable message is printed on the screen (3), informing about the necessity of introducing some corrections.

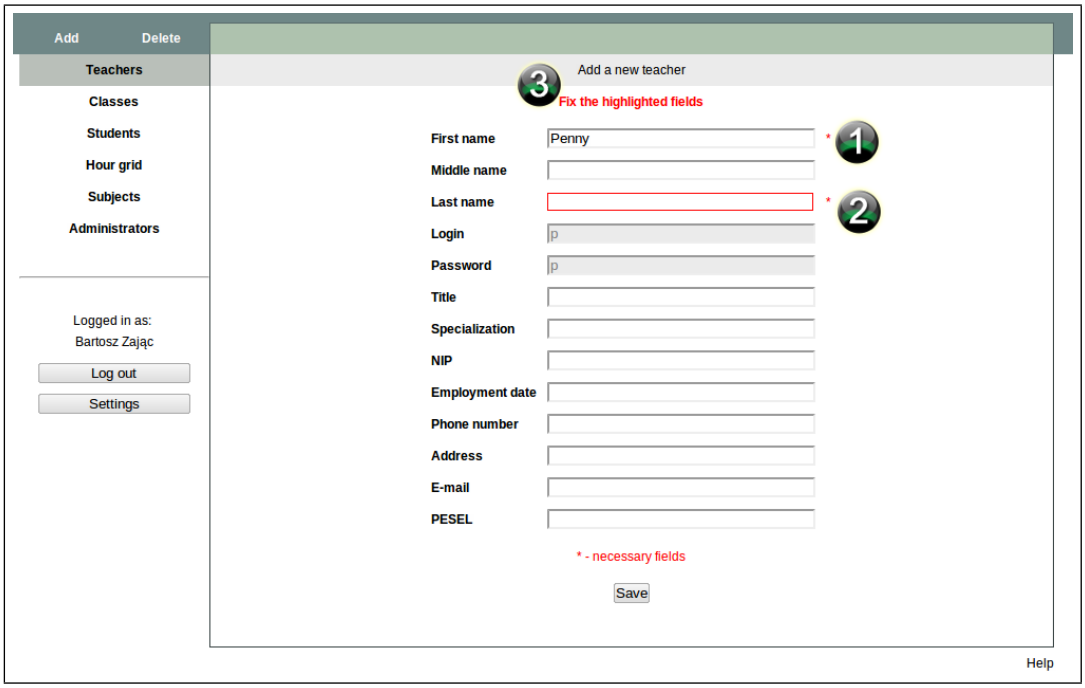


Figure B.3. *Example of form validation.*

B.4. Settings

The administrator, after clicking on the `Settings` button (see Fig. B.1), receives an access to the data (2) about him/herself (1). Thanks to the employment of `Change password` button (3), one is also able to personalize his/her login information.

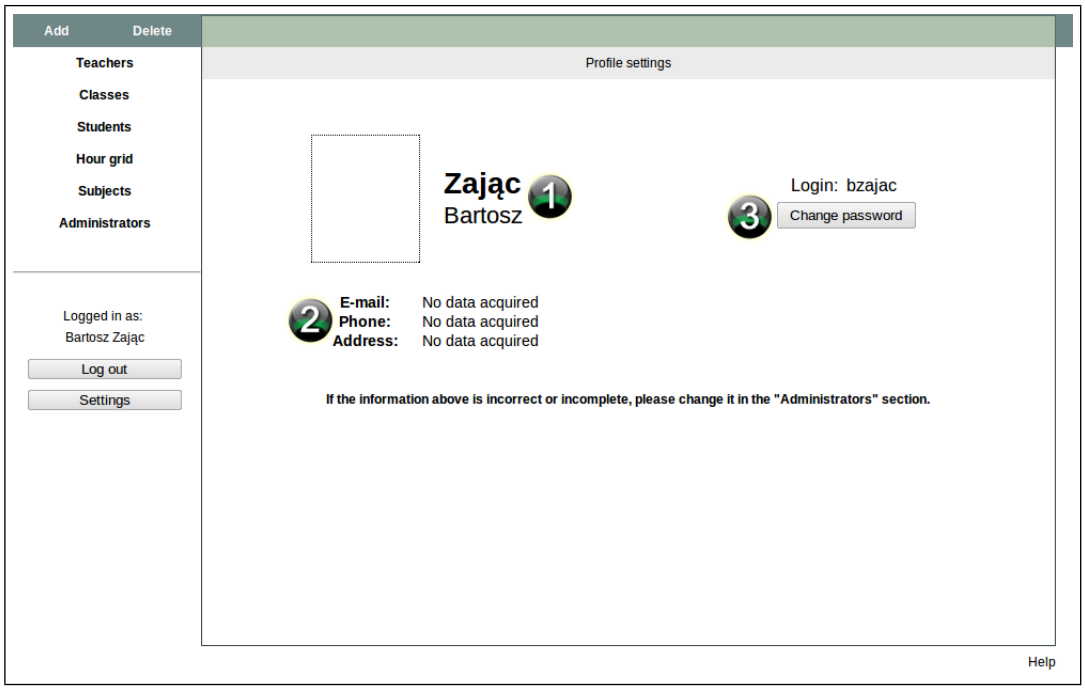


Figure B.4. *Administrator's settings module.*

C. User manual

Below we present an approach to create a complete instruction manual for CampusNet users. It lists all platform's functionalities along with detailed guide on how to perform desired operations. Let us begin with the authorization panel.

C.1. Logging in

Overview

The welcome page of CampusNet serves also as an authorization panel. So as to gain access to the system, it is necessary to enter personal username and password into corresponding fields - (1) and (2), respectively - and click onto the login button (3). From within this page, one is also allowed to enter the administration panel (4). Finally, thanks to the localization menu (5), one is also able to choose language in which the system will be displayed.



Figure C.1. CampusNet platform Welcome Page.

Incorrect login

In case of incorrect login data, suitable information is printed on the screen (1).

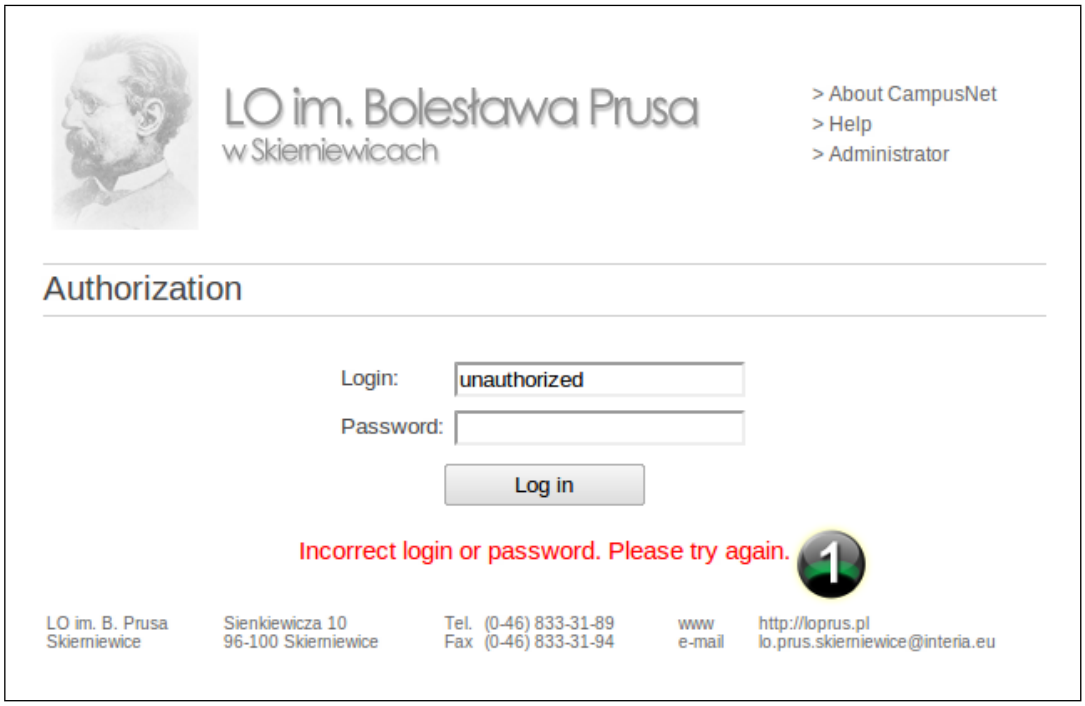


Figure C.2. Error message due to incorrect login/password.

Successful login

If such error message is printed after inputting correct data, one should contact the administrator, who can check the existence of proper entries in the database. Successful operation shall redirect user to the welcome page of adequate panel.

C.2. Event calendar

Overview

The edition of this feature is available exclusively to teachers, both students and parents are permitted only to browse it. Professors are allowed to add, edit and delete different events.

By default, the calendar will be centered on current month. Today's date is marked with use of different cell color (1). Hyperlink `Today` (2) displays the month, to which belongs the actual date. Links `<<` (3) and `>>` (4) may be used to switch to previous or next month, correspondingly, while `20xx` (5) and `20xx` (6) work similarly, but they allow user to switch between years instead of months.

An event is represented by a colored rectangle (7), (8) etc. and a text information within it. To add a new one on certain day, just click on the field representing desired date in the calendar, for example (9).

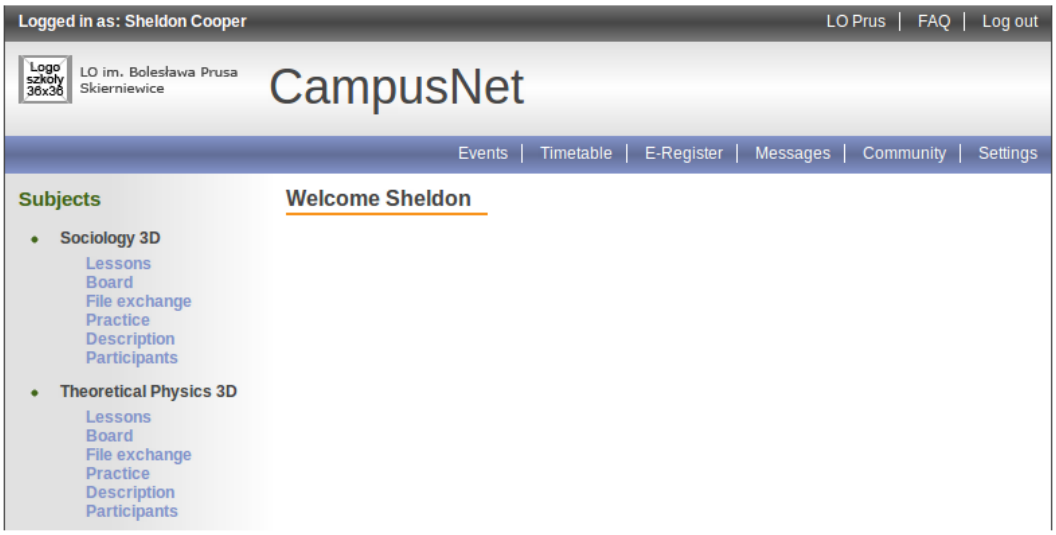


Figure C.3. User panel's page to which he/she is redirected after successful login.

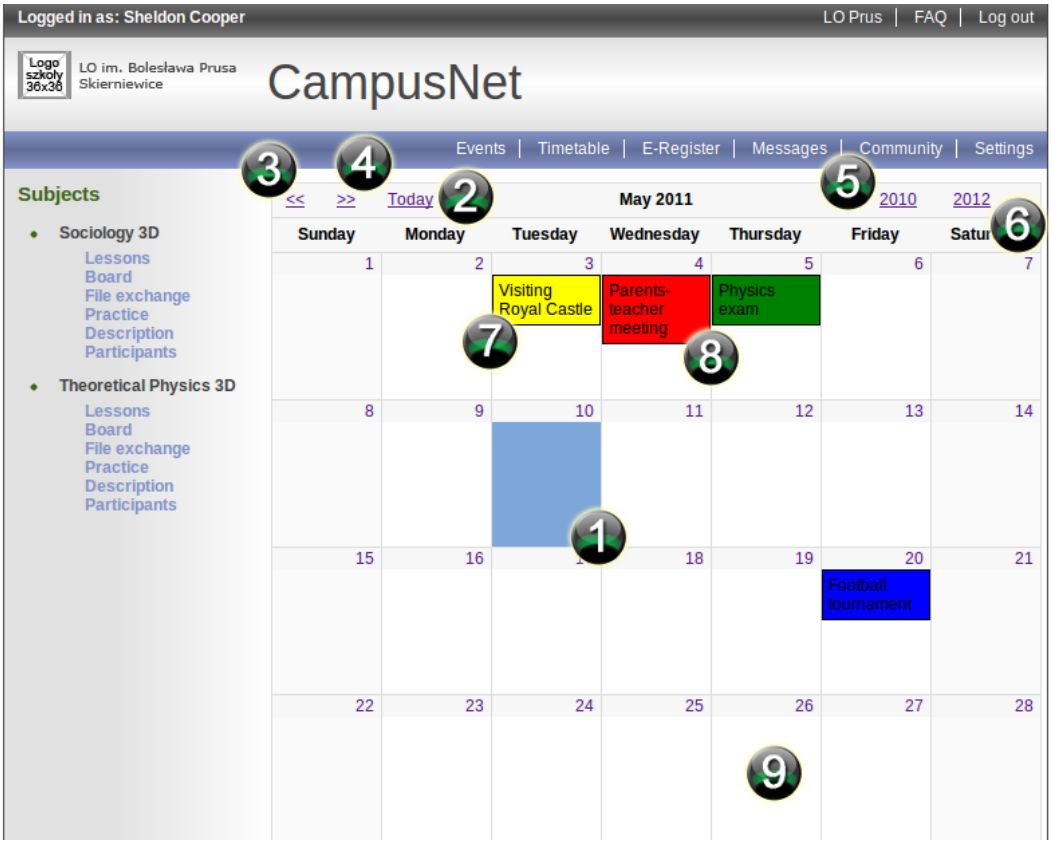


Figure C.4. Event calendar with four events of different type.

Adding event

Clicking on a date's field will cause a pop-up box to appear with four input fields (Event type (1), Title (2), Description (3), Optional URL (4)) allowing to define the

event. Two of them (Event type and Title) are obligatory, while filling the remaining ones is up to the user. So as to confirm the creation, one has to click onto `Save` button (5).

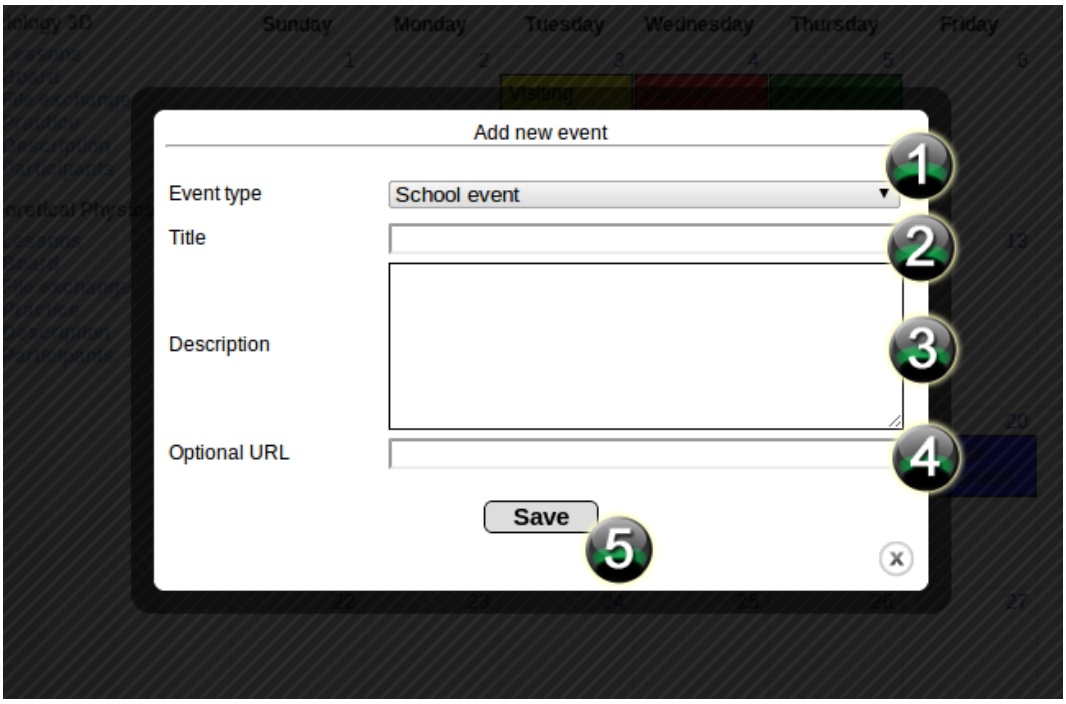


Figure C.5. Pop-up box allowing to add a new event.

Edition & deletion

There exist a possibility to edit existing event. In order to do so, just click on the colored rectangle representing desired event. Then appropriate pop-up box should appear, displaying selected event. To receive possibility of edition, `Edit` button (1) has to be clicked. This will invoke a pop-up box similar to the one related to adding event, allowing user to apply planned changes. Since then, the process of edition looks similar to the one responsible for adding events.

So as to solve the situation, when the event is canceled, there exist an option to remove it from calendar. To accomplish this task, it is enough to select the event meant for deletion and click the `Delete` button (2) that appears along with pop-up box.

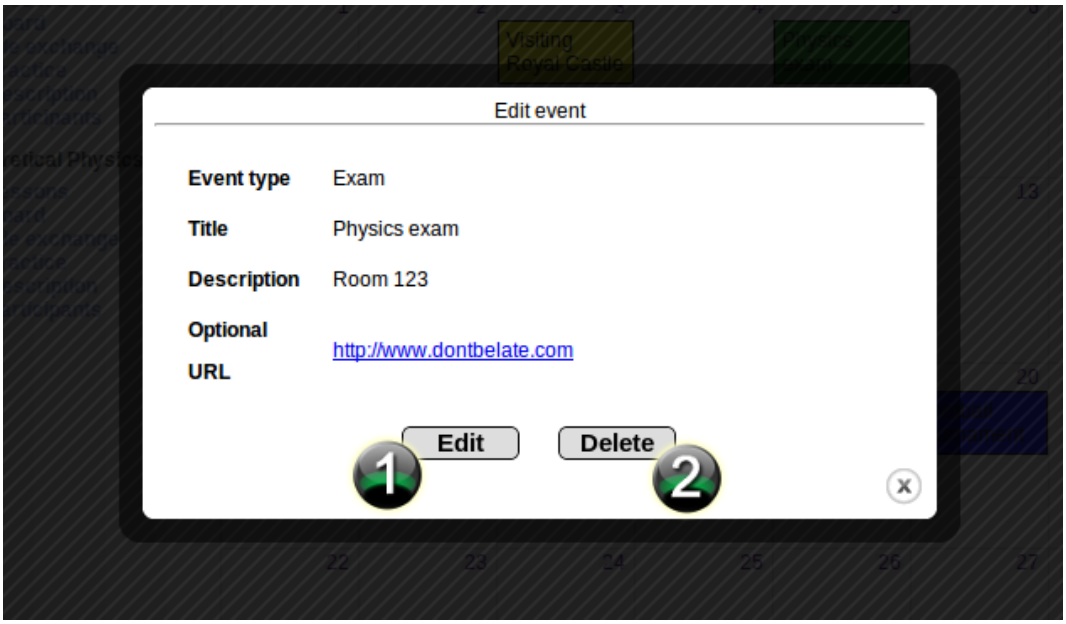


Figure C.6. Pop-up box allowing to edit an existing event.

C.3. Timetable

Overview

As the timetable is editable only from the administration panel, CampusNet users are allowed only to browse it. In case of any doubts or problems regarding his/her schedule, one should contact a suitable person from school's administrative staff (who possesses rights to access the administration panel and modify the database).

C.4. E-register

This module of the platform is divided into two subparts – marks (1) and attendance (2). Each of them are self-explanatory, thus we will assume, that user desiring to use certain function has already entered adequate section. Moreover, any operations of data modification within this module are allowed only to teachers – students and parents may only browse it.

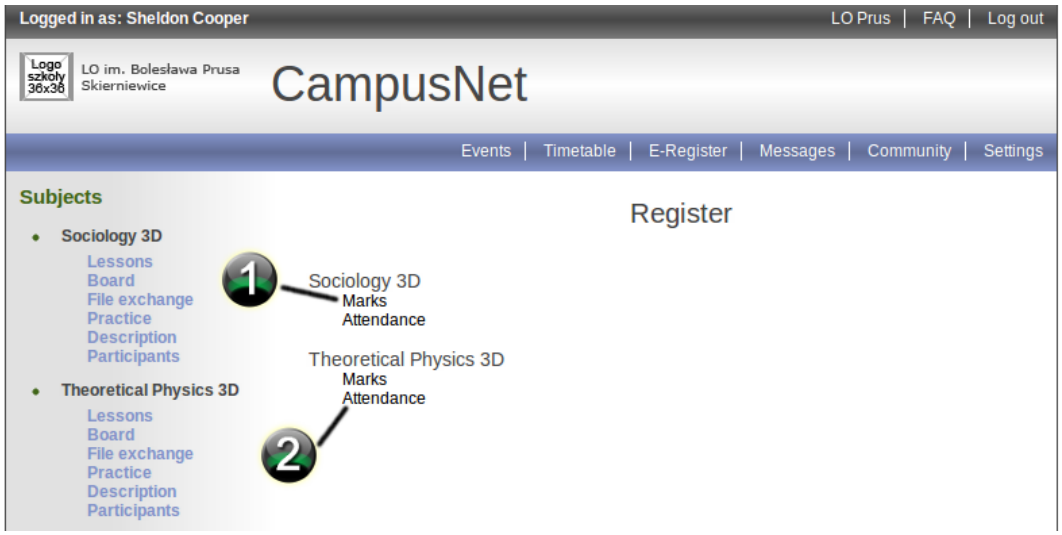


Figure C.7. E-register initial page.

C.4.1. Marks

Overview

The marks subpart of e-register allows teacher to manage students' grades obtained during given course. The buttons `Add column` (1), `Save changes` (2) and `Print` (3) enable teacher to create new columns for marks (representing an event being evaluated by him/her, e.g. test, homework etc.), save recent changes and print out the e-register (after generating a PDF file), respectively. `X` buttons (4) cause the appropriate column deletion. Column marked as (5) represent students' weighted average grade, while (6) stands for semester grade.

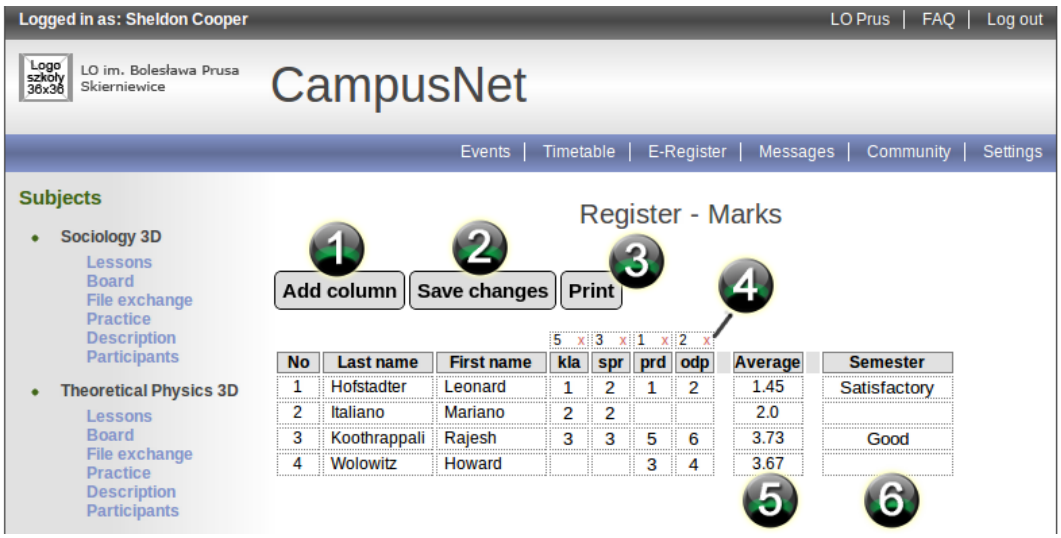


Figure C.8. E-register marks general view.

Adding new column

Clicking the `Add column` button causes a pop-up box to be displayed on the screen. In order to complete the addition successfully, it is necessary to fill in `Column name` (1) and `Weight` ((2), at most two-digit integer) fields and use the `Add` (3) button. If one changes his/her mind and desires to cancel this process, `X` (4) closes the pop-up box.

After completion of the task, the teacher is able to input marks for every student in newly created column.

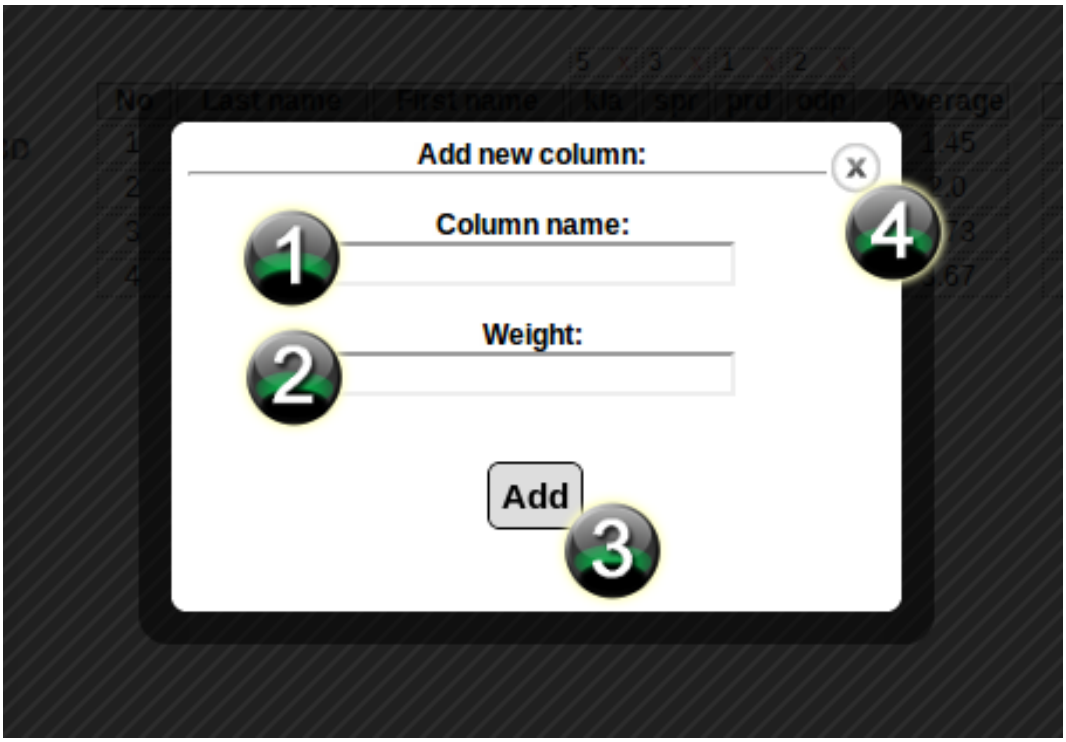


Figure C.9. Pop-up box allowing to add new mark column.

Deleting column

Using this function makes a confirmation box (1) to appear on the screen, allowing to cancel (2) the operation or to accept it (3). One should remember that deleting certain column erases also all marks stored within it.

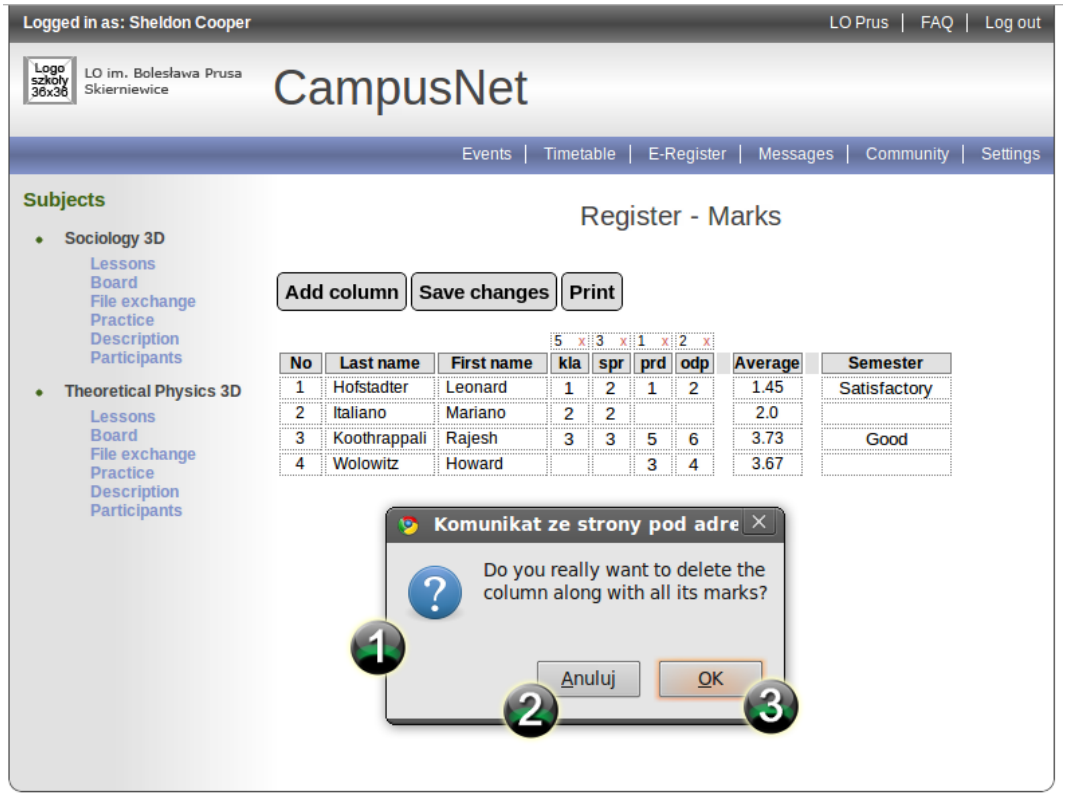


Figure C.10. Pop-up box asking the user to confirm his/her decision.

Adding & deleting marks

Having created some mark columns, teacher is allowed to input students grade. Marks may be placed in appropriate cells, e.g. (3) – there exists no difference between putting a mark into an empty or occupied cell. However, one shall remember that in the second case, older grade will be removed.

Deletion of a certain mark may be achieved by insertion of lowercase `x` (1) character instead of it. Recent changes have to be confirmed by clicking the `Save changes` button (2), otherwise all data will be lost on next page reloading.

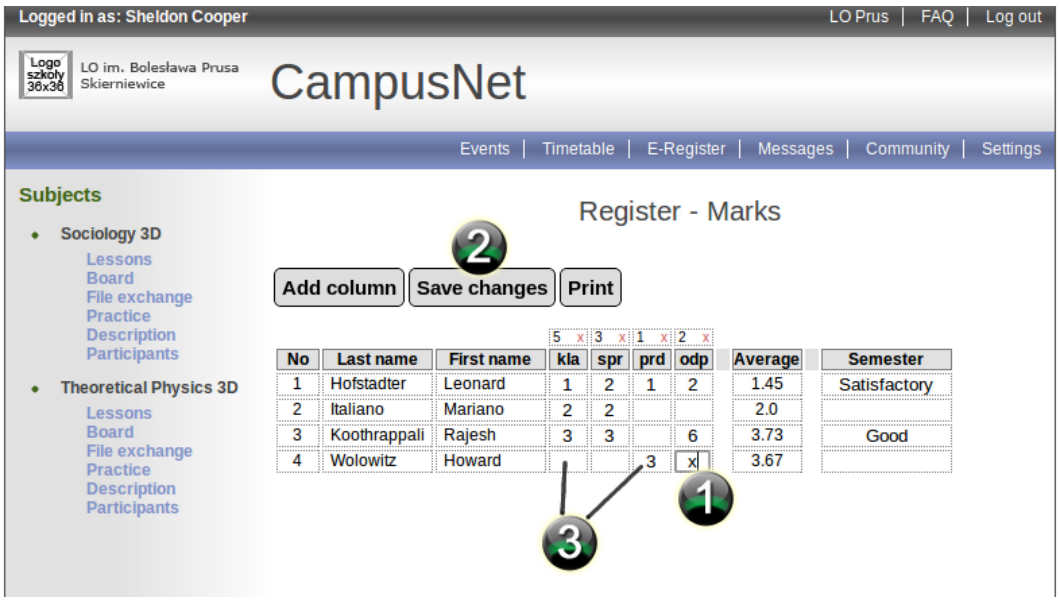


Figure C.11. E-register marks - mark deleting example.

Invalid marks

There exists some restrictions on possible input marks, only values from 1 to 6 (with optional + or − signs) are allowed in case of the partial ones. Semester grades possess a text format, and their value differ along with currently used language version.

In case of an incorrect input (1), an approach to click the `Save` button (2) causes a suitable message to be displayed (3). Moreover, the corresponding cell is being highlighted to facilitate error's localization.

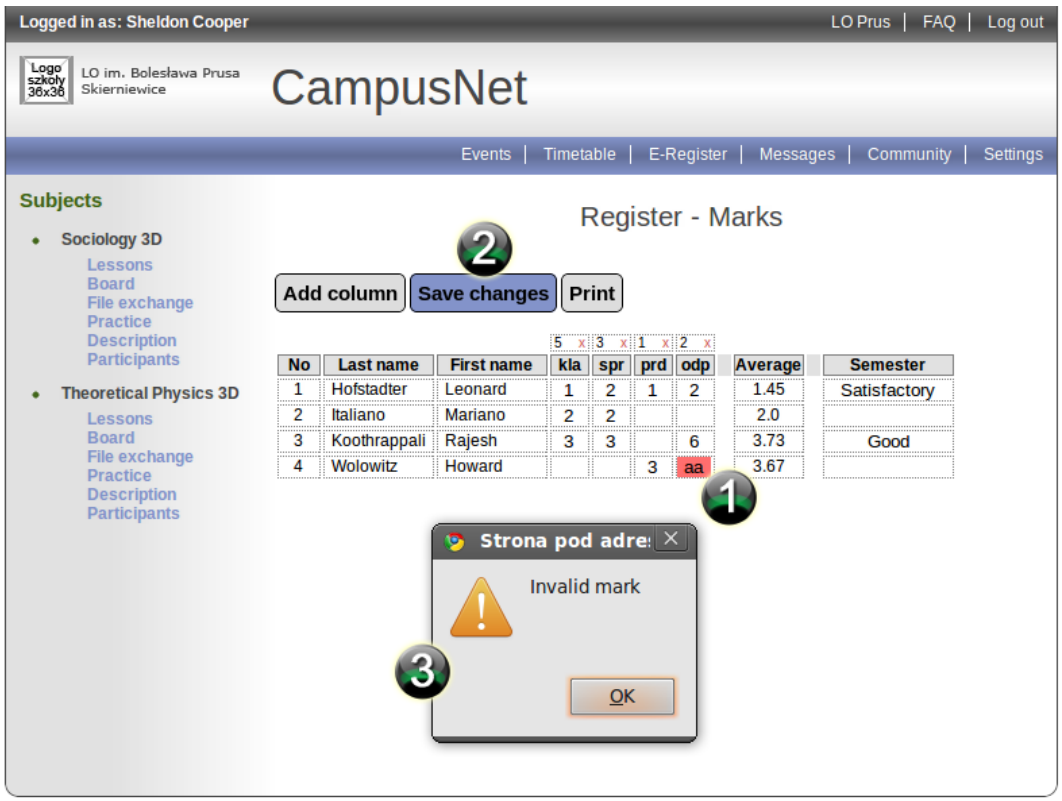


Figure C.12. Pop-up box informing about incorrect input.

Printing out

Every teacher is allowed to export the e-register marks to a PDF file, e.g. for the purpose of later printout. The resulting file possesses the following layout:

- subject's name (1),
- teacher's full name (2),
- list of students (3),
- list of students' marks(4).

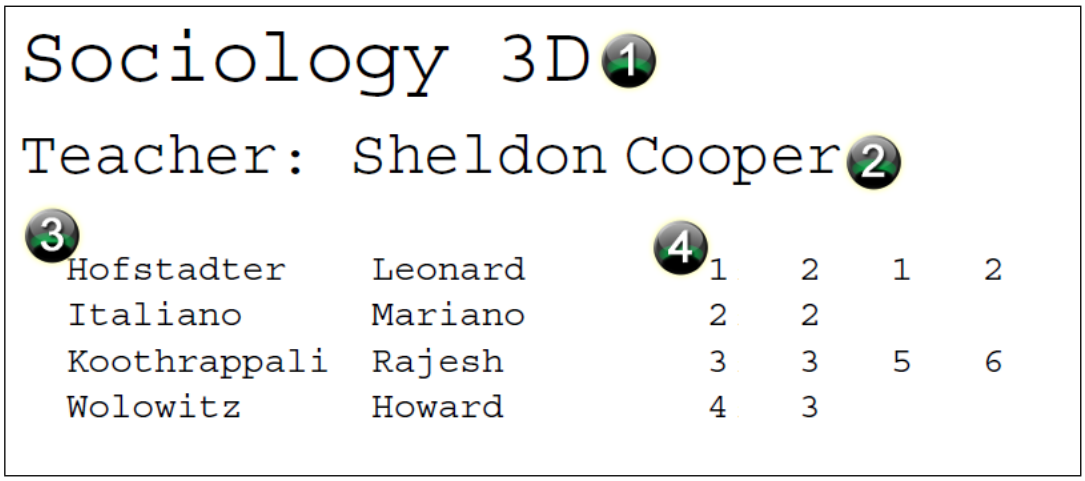


Figure C.13. Exemplary PDF output file containing marks' information.

C.4.2. Attendance

Overview

The attendances subpart of e-register allows teacher (students and parents may only browse this module) to manage students' attendance performance during given course. Hovering the date input field (1) helps to switch between different days. Clicking on one of the + buttons (2) empowers teacher to modify values withing given column.

One should realise, that a column represent one-hour lesson (we assumed that student will attend to at most 8 lessons a day). Hence, if the teacher has a fourth lesson with a certain class on a given day, he/she should click the + button above column labeled as `4`. By default this action will set all students' attendances to the value `present` (which is the most probable case), also allowing further manual modifications. In case of any doubts, see Fig. C.16.

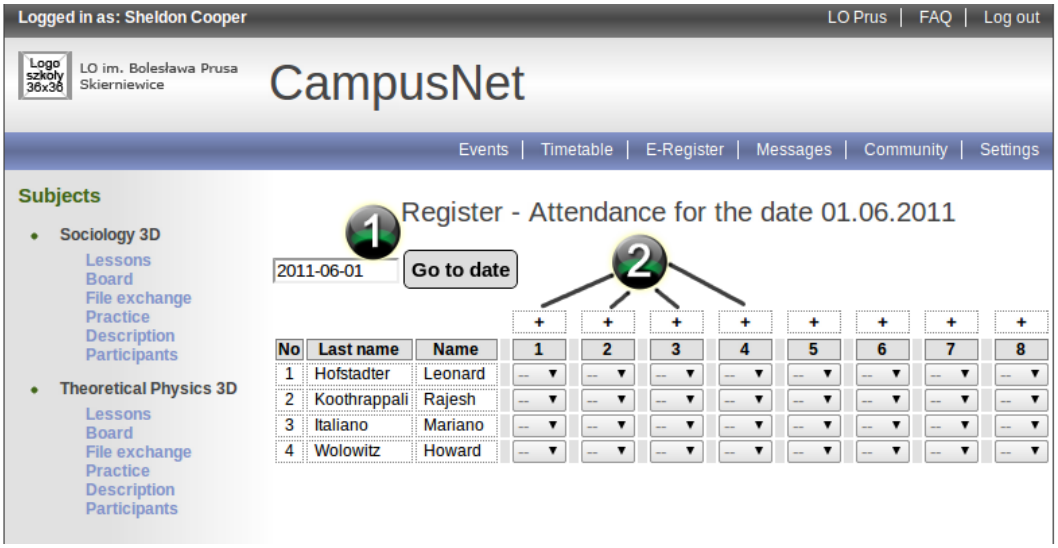


Figure C.14. E-register attendances general view.

Change date

After selecting the date input field a calendar pop-up box (1) shall appear on the screen. In order to choose desired date it is enough to click on the cell representing the day.

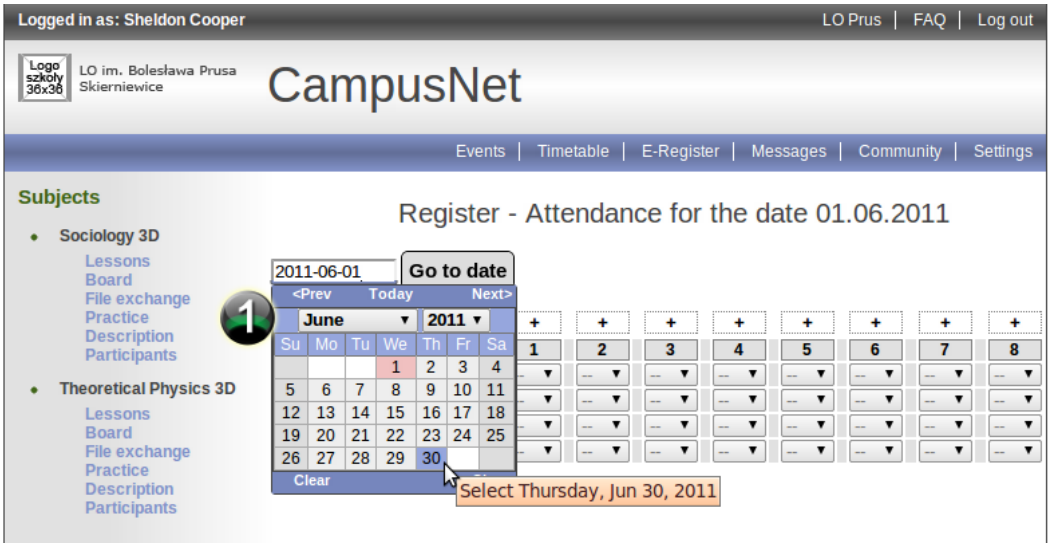


Figure C.15. Pop-up box allowing date selection.

Editing attendances

This feature gives teacher the opportunity to modify attendance value fields (e.g. (2)) according to students' performance. Clicking on one of the – buttons (1) causes all fields within a column to be restored to their initial state (At the Fig. C.16 columns labeled as 6, 7 and 8 are in the initial state).

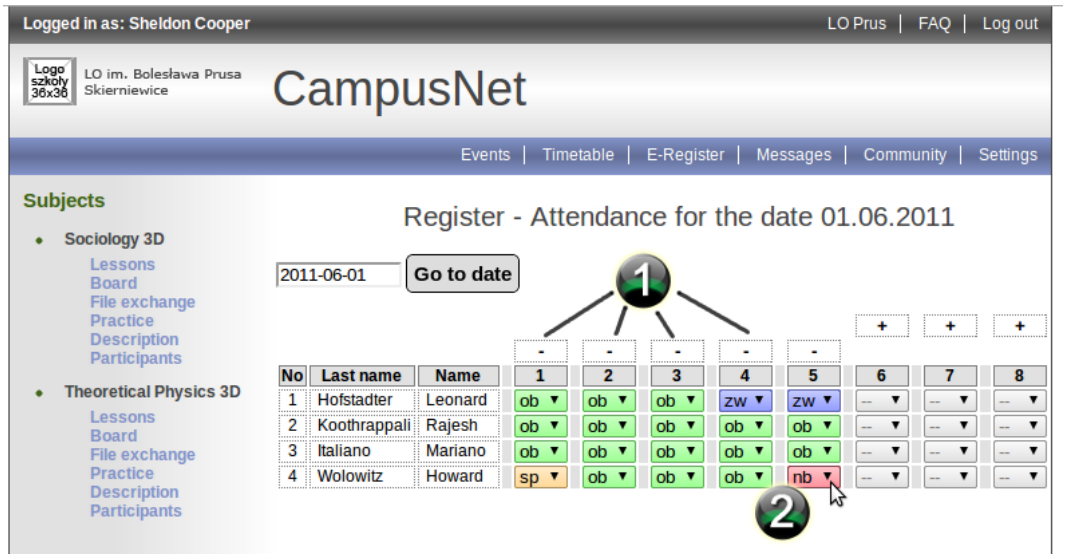


Figure C.16. Exemplary edition of teacher panel attendances module.

C.5. Settings

Overview

Settings module allows each user to view and edit personal information. The layout of this page consists of a profile photo (1), user's full name (2) and basic information residing in the database (3). `Change password` button (4) enables user to customize his/her password.

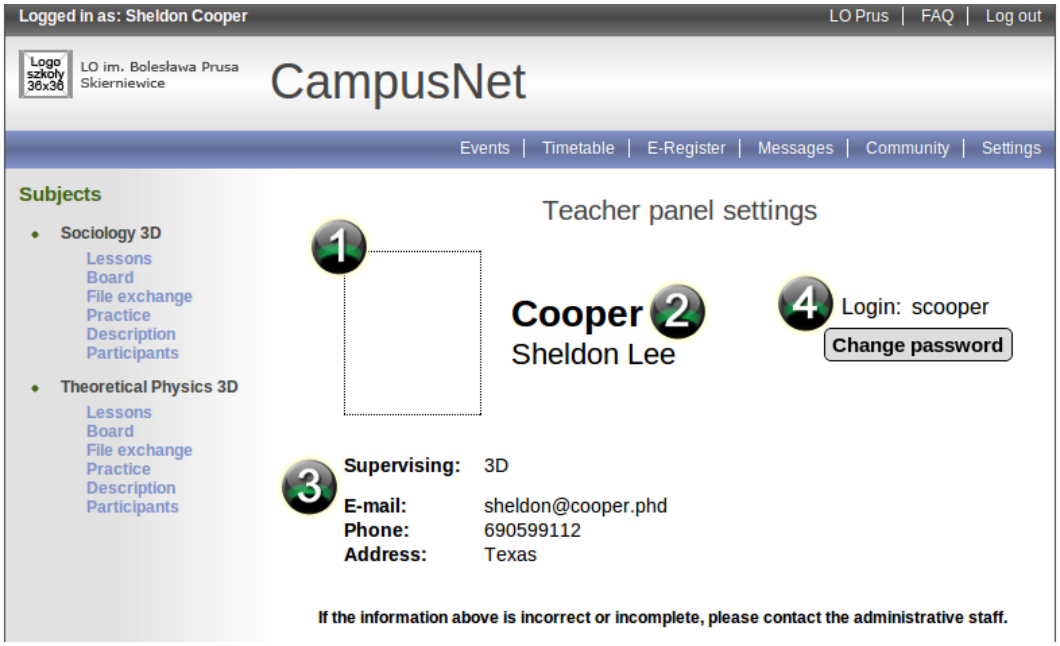


Figure C.17. Settings section general view.

Changing password

Enabling this feature causes a short form to be displayed on the screen. After filling in the necessary information – `Old password` (1), `New password` (2) and its repetition (3), the process may be completed by clicking on the `Change password` button (4).

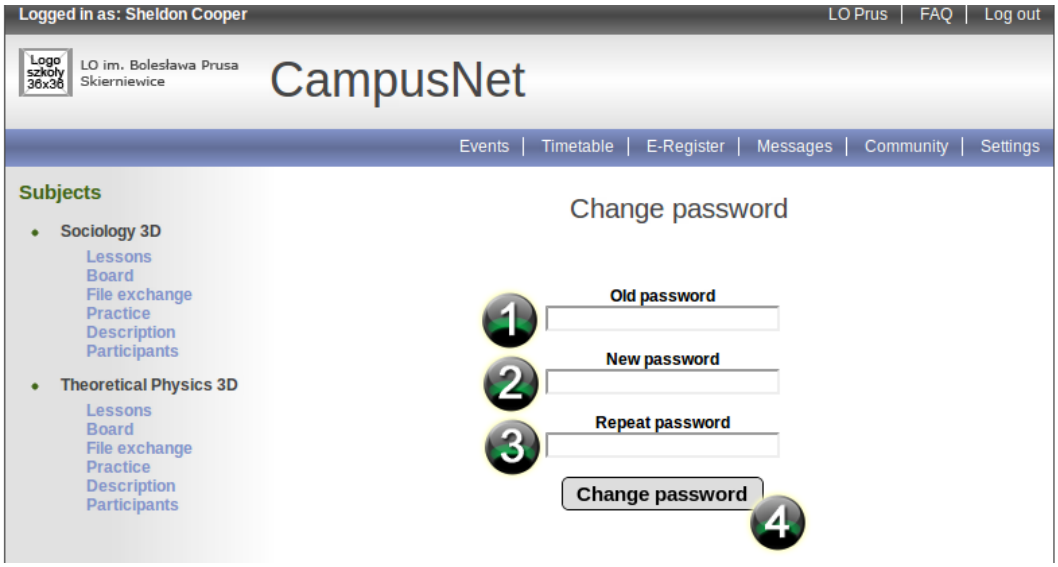


Figure C.18. Password edition page.

Incorrect password input

In case of incorrect fulfilment of the aforementioned form with incorrect data, a suitable message (1) will inform user about the error. So as to repeat the process, one should click on the `Back to settings` button (2) and retry.

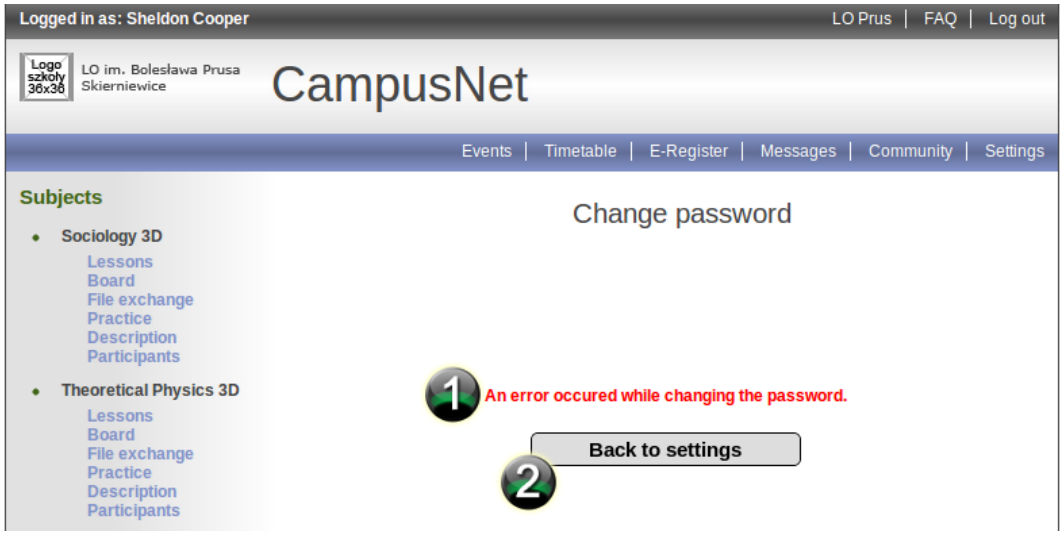


Figure C.19. Error message due to unsuccessful password change process

C.6. Community

Overview

The main idea behind this feature is to enable all users to view individual profile of every school's community member. People profiles are categorized according to their role within the school (1) and represented in form of a list. In case of students and parents, the additional class-related division has been introduced (2). `1C`, `2A` etc. items represent class identifiers. So as to proceed to the deeper level of the list, it is necessary to click on desired hyperlink.

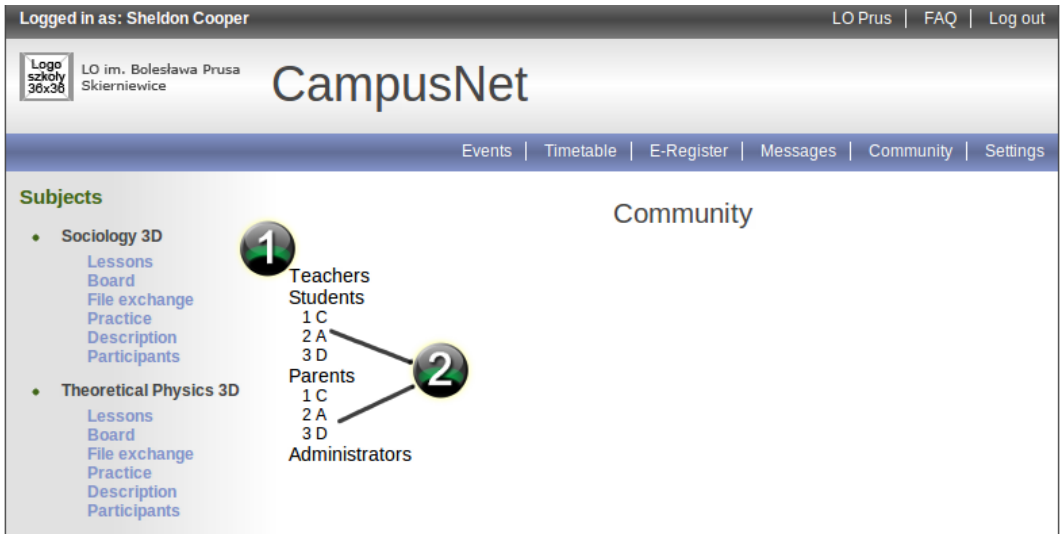


Figure C.20. Security section general view.

Viewing profile

Personal profile is the lowest level of community members list. Its layout is composed of personal photography (1), one's full name (2) and an extract of relevant information (3), format of which depends on the type of the user.

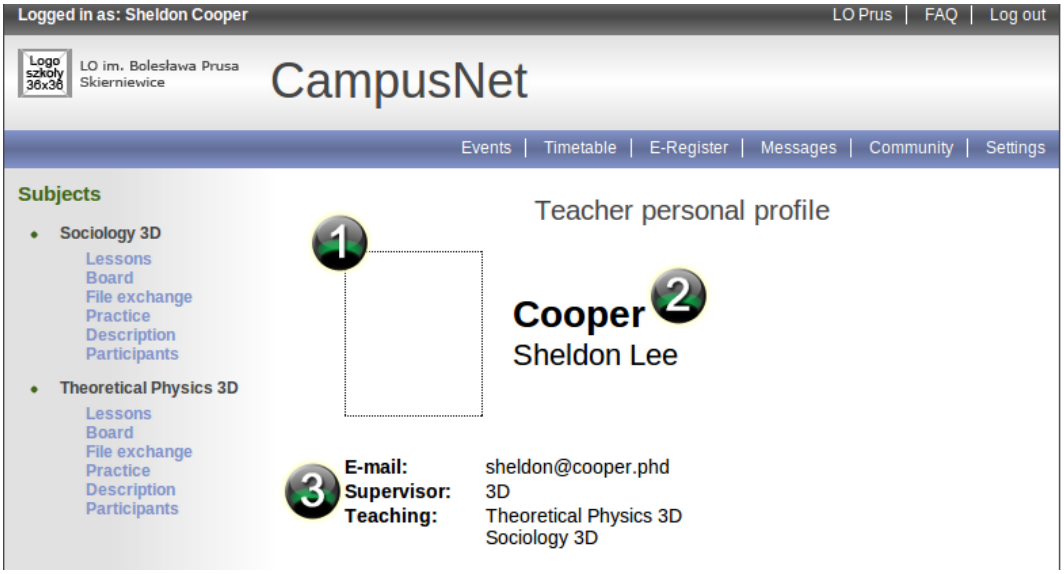


Figure C.21. Teacher's personal profile.

C.7. Messages

Overview

Selecting the `Messages` hyperlink from personal menu redirects user (regardless of the type) to the `Received` box. `All/unread` options (1) allow to select the messages (show all or unread only) to displayed on the screen. `New message` (3), `Delete message` (4), `Show received` (5) and `Show sent` (6) links are self-explanatory. All received/sent messages are displayed within the area of (7).

The checkbox (2) allows one to select all items without the necessity of single-checking each of them. Finally, the pagination menu (8) empowers the user to browse through pages in case of having a large amount of messages in the box.

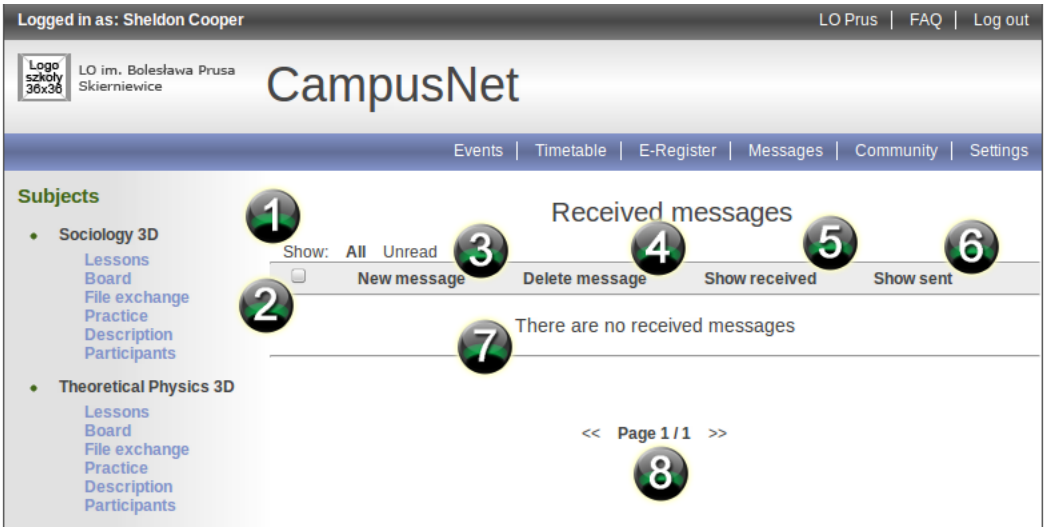


Figure C.22. Inbox general view.

Sending new message

The form consists of three different input fields, namely the recipient field (1), subject field (2) and content field (3). The first one requires user to input at least three characters (being a part of first or last name of the recipient) – then search process commences, browsing the database and returning matching results. It is possible to send a message to multiple recipients at one time. So as to do this, it is enough just to start typing another query within the recipient field (1). Every person, to whom the message will be sent, is represented in that field as a single token. After filling in the remaining input areas, one shall confirm the sending by clicking on `Send` button (4).

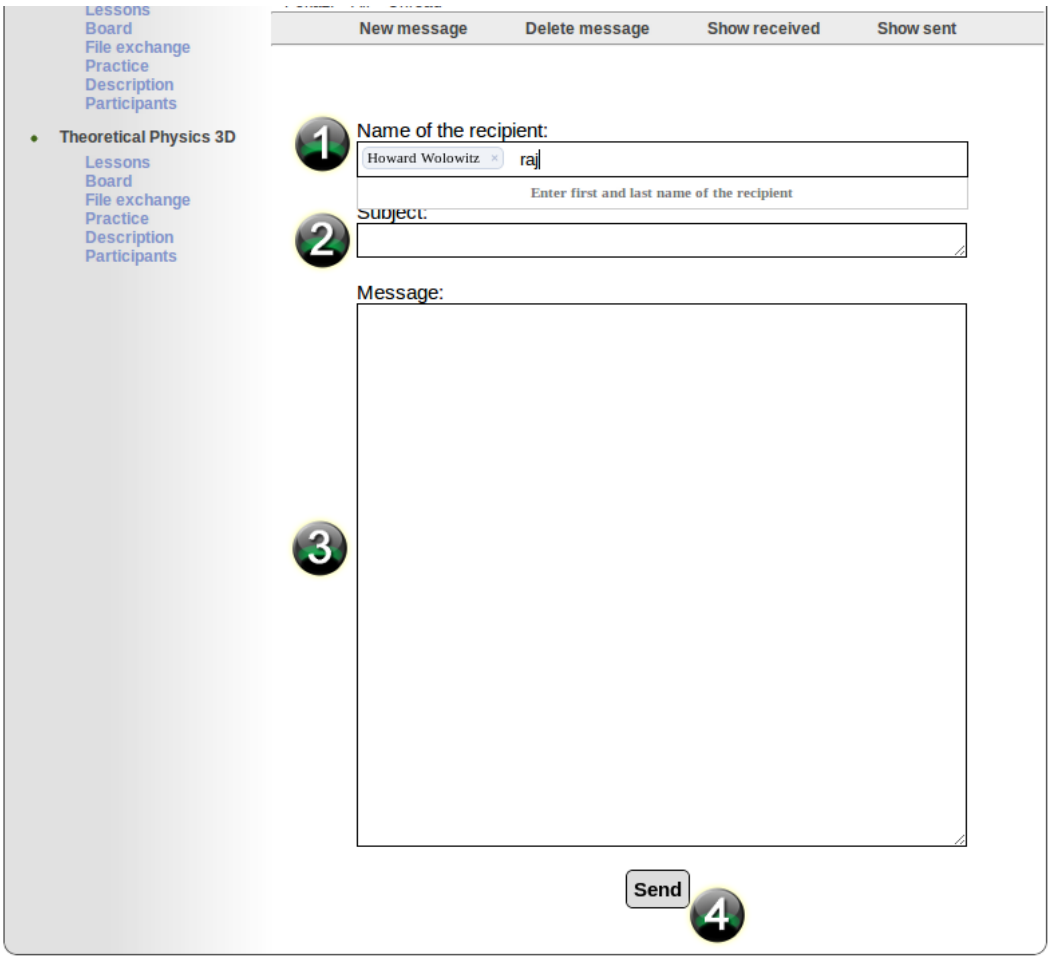


Figure C.23. Form allowing to send a new message.

Sent & received

The message module allows user to keep two separate boxes, standing for sent and received messages. One may observe below an exemplary view of sent box. Checkboxes (1) present the functioning of `Select all` checkbox. (2) and (3) help to picture the representation of unread (2) and read (3) messages – they differ in font weight.

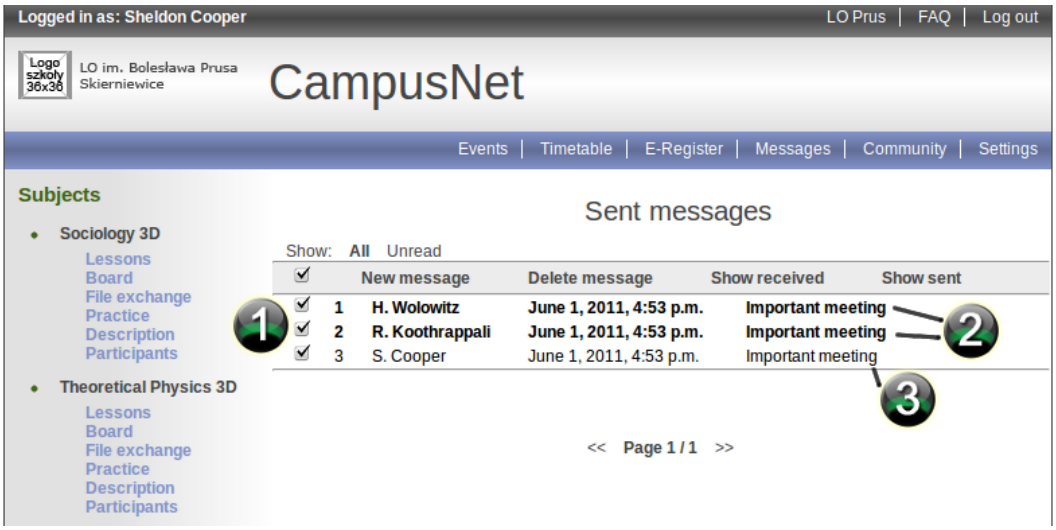


Figure C.24. Sent messages general view.

Viewing message

As can be expected, the module provides possibility to see every message in detail. To proceed to this view it is enough to click on arbitrary component of a message represented in received/sent box. Next, the proper item will be displayed, presenting its author, date (1) and content (2).

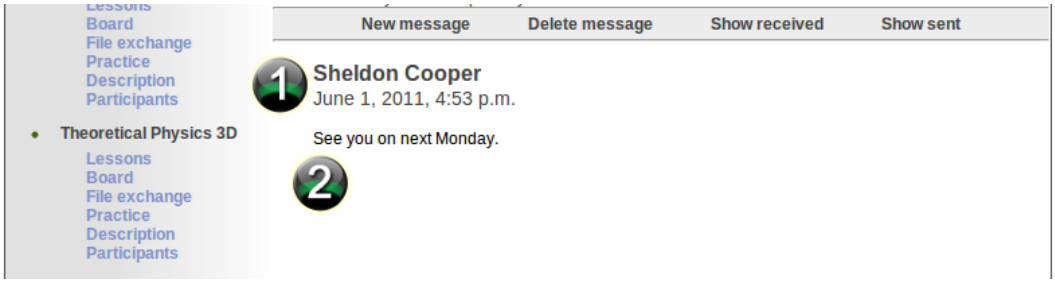


Figure C.25. Viewing an exemplary message.

C.8. Lessons

Overview

Lessons module offers to user an opportunity to monitor the history of a certain course he/she is participating in. Only teachers are allowed to modify the content of this section.

In order to introduce new lesson event, it is necessary to click on `Add lesson button` (1). (2) represents a single lesson entry, containing its topic, description and given homework. One (provided he/she is a teacher) may delete existing entry with use of `X` button (3).

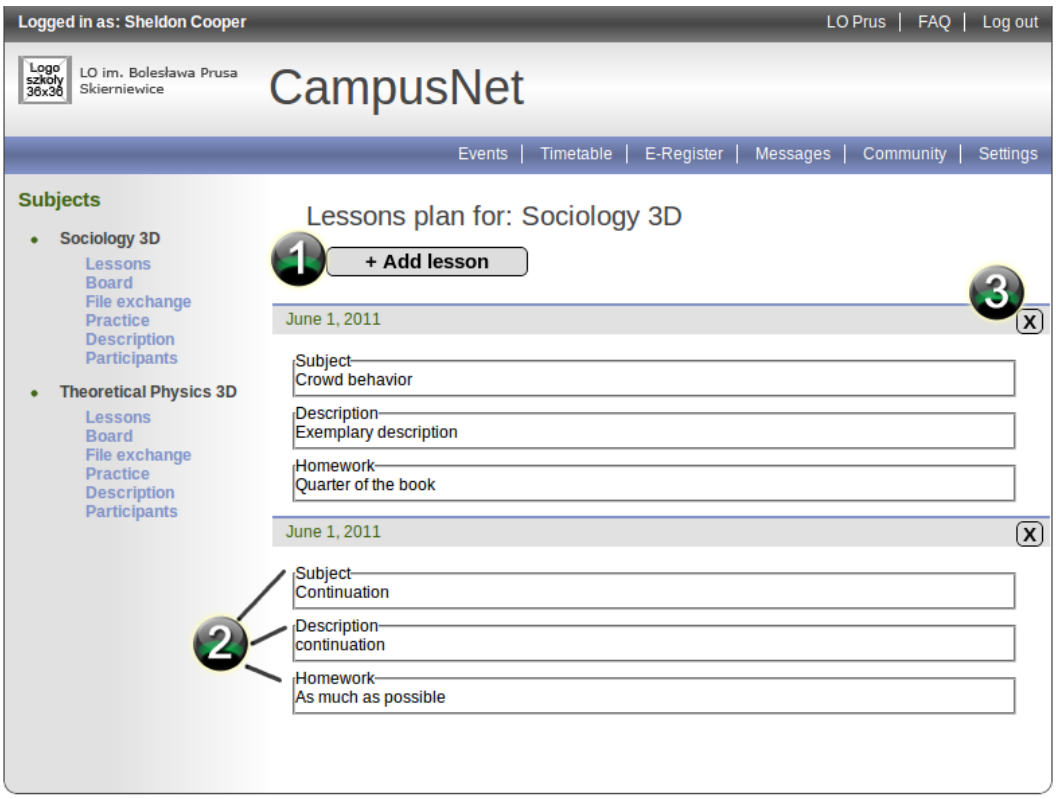


Figure C.26. Lessons section general view.

Adding new lesson

So as to add new lesson event, it is required to fill in the date (1) and subject (2) input fields. One may also attach the description (3) and optional homework (4). Process of addition is completed after clicking on the `Save` button (5).

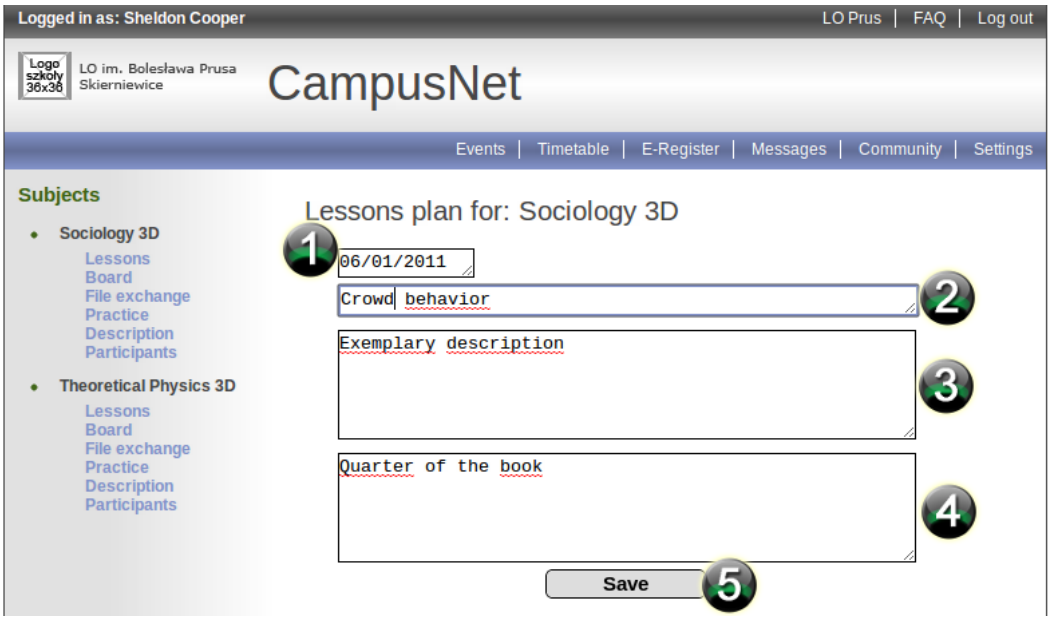


Figure C.27. Form allowing to add new lesson event.

C.9. Board

Overview

The idea behind this module is to allow all course's participants (teachers and students, parents do not possess access to this section) to exchange information related to the course efficiently. Subject's board consists of a set of people's posts, organized as a list (1). Input field below (2) empowers users to create new entries, while `Submit` button (3) completes the process of aforementioned creation. Every user is permitted to delete his/her own post, which may be achieved by clicking on the `Delete` button (5). Pagination menu (4) helps users to browse through the collection of posts.

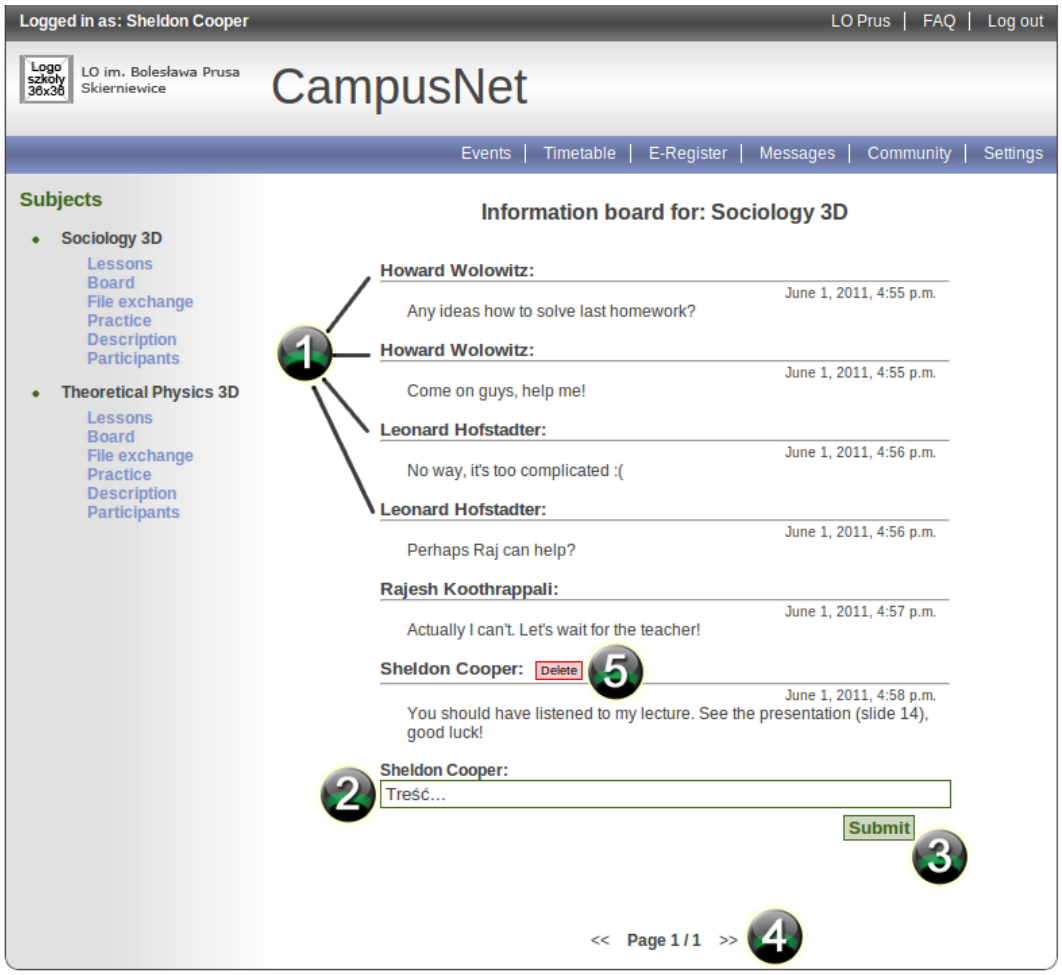


Figure C.28. Subject's board general view.

C.10. Subject's description

Overview

Subject's description, available from subject menu, is a brief information regarding the content of certain course (2). Both students and their parents may view the descriptions, while teachers are also empowered to modify them. Last option may be reached by using the `Edit` button (1).

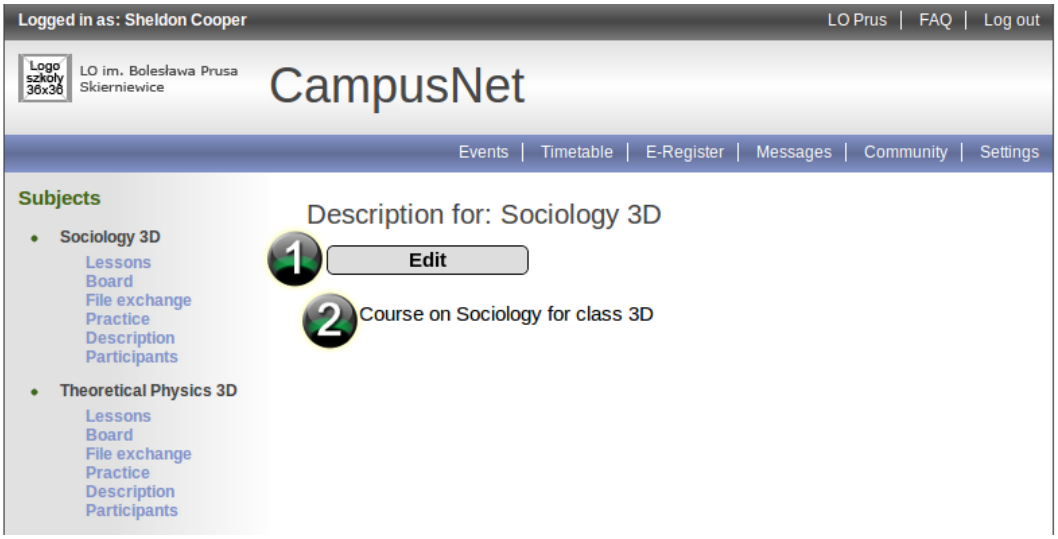


Figure C.29. Subject's description general view.

Editing description

Within this section, the teacher shall introduce a brief description of every course he/she is tutoring. In order to do so, it is enough to enter desired information within a text field (1) and complete the process by clicking on `Save` button (2).

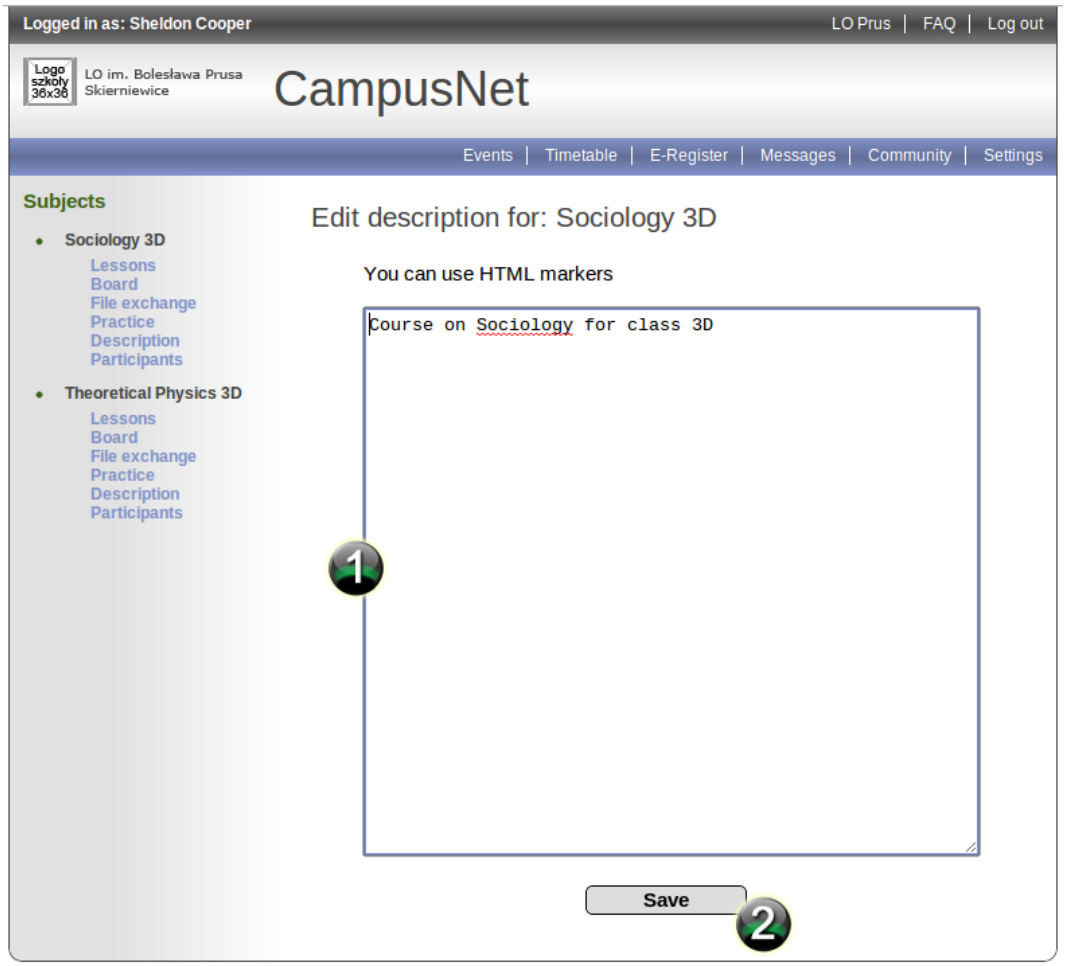


Figure C.30. Form allowing to edit subject's description.

C.11. Paricipants

Overview

The list of participants, available to teachers, students and their parents, ensures the easiness of finding people attending the same course. This module is organized as a list consisting of a teacher tutoring the course (1) and all participating students (2). Parents are given the opportunity to browse through children-related lists.

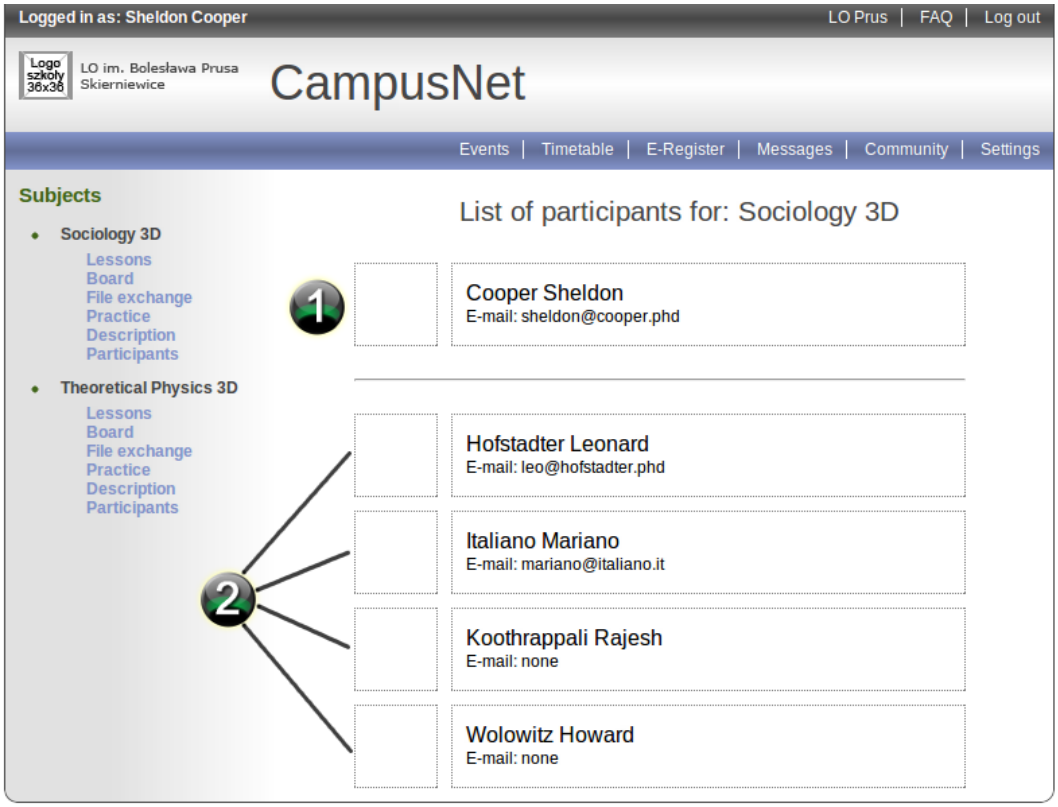


Figure C.31. Paricipants' list general view.

D. Testing manual

Chapter 4 contains the description of software testing performed by authors of this thesis. Still, we suppose that one may be interested in redoing the testing on his/her own or perhaps creating some additional tests. Consequently, we present a brief guide, explaining how to setup and conduct a successful test on the CampusNet platform.

D.1. Setup

As it was mentioned in sections 4.1 and 4.2, in order to perform automatic testing, installation of Selenium environment is required. There exist three obligatory and one optional step on the way to complete this task successfully.

Step 1

- ◇ software: Selenium server,
- ◇ version: 2.0b3,
- ◇ from: Selenium Project website,
- ◇ url: http://code.google.com/p/selenium/downloads/list

First of all, the Selenium server should be downloaded, e.g. from the URL mentioned above. We recommend choosing the standalone version (`selenium-server-standalone-2.0b3.jar`). Note that this file shall be placed (along with testing sources) into `/functional_tests/` subdirectory of `/campusnet_project/` folder[1].

Step 2

- ◇ software: Java Runtime Environment,
- ◇ version: 6 (32-bit),
- ◇ from: Linux repositories,

Due to the fact, that Selenium server is a Java application, installation of Java Runtime Environment is compulsory:

```
1 sudo apt-get install sun-java6-jre
```

Listing D.1. *Installation of Java Runtime Environment.*

Step 3

- ◇ software: Selenium IDE,
- ◇ version: 0.2,
- ◇ from: Mozilla Firefox website,

[1] In followance with the convention introduced in Appendix A.

◇ url: https://addons.mozilla.org/firefox/downloads/

Next, Selenium IDE may be optionally installed. Implementing this tool allows one to create new tests and modify existing ones. In fact, Selenium IDE we are installing is a Mozilla Firefox plug-in.

Step 4

◇ software: Selenium libraries for Python,
◇ from: Linux repositories,

Finally, automatic testing requires also Selenium libraries for Python to be attached:

```
1 sudo easy_install selenium
```

Listing D.2. *Installation of Selenium libraries for Python.*

Having completed the aforementioned steps, one is able to run the tests.

D.2. Running functional testing

Before starting functional testing, it is necessary to enter the `.../campusnet_project/functional_tests/` directory and run the bash script:

```
1 cd .../campusnet_project/functional_tests/
2 ./run
```

Listing D.3. *Running the bash script.*

Then, the tests may be run with the following command:

```
1 python tests.py
```

Listing D.4. *Running Selenium test.*

After this, Python test suite shall connect to the Selenium server, which automatically executes predefined tests. A complete log documenting performed actions is saved in the `output.log` file.

D.3. Running regressive testing

While running this type of testing is extremely simple, one shall remember about the necessity of creating a test database before the execution[2]. This may be achieved by the following sequence of commands:

```
1 mysql -u root -p
2 [here appears the prompt for root password to the database: 'root_pwd']
3 mysql > CREATE DATABASE campusnet_test CHARACTER SET utf8;
4 mysql > GRANT ALL ON campusnet_test.* TO campusnet_user@'localhost'
5     IDENTIFIED BY 'campusnet_user_pass';
6 mysql > exit;
```

Listing D.5. *Creating a test database.*

Then, one should navigate to the project's main directory and execute the following command:

```
1 cd .../django_project/campusnet
2 python manage.py test
```

Listing D.6. *Running regressive testing.*

The information regarding success or failure will be shown within the console window.

[2] For more information, see Sec. 4.3

E. Database visualization

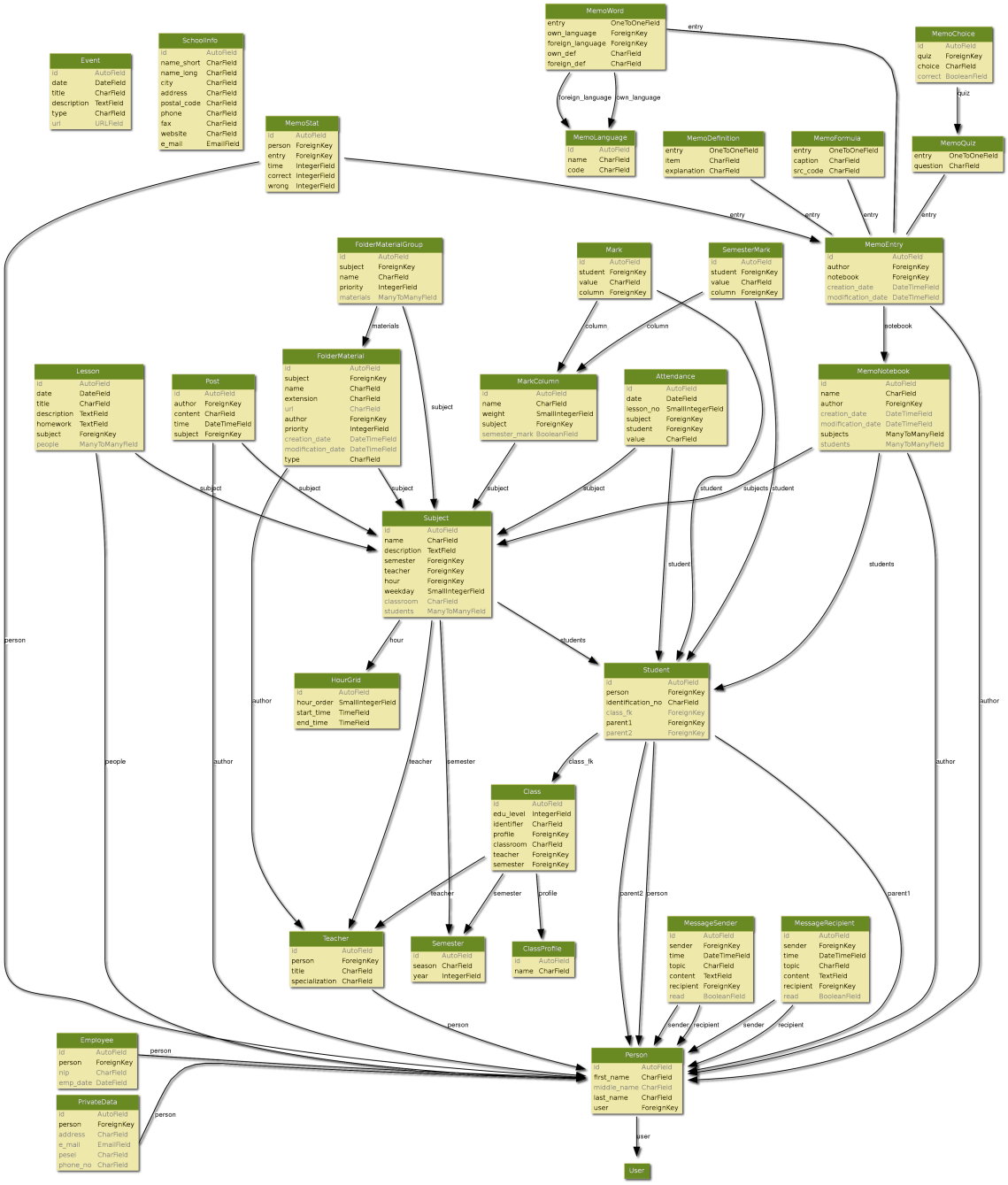


Figure E.1. Visualization of final version of CampusNet database.

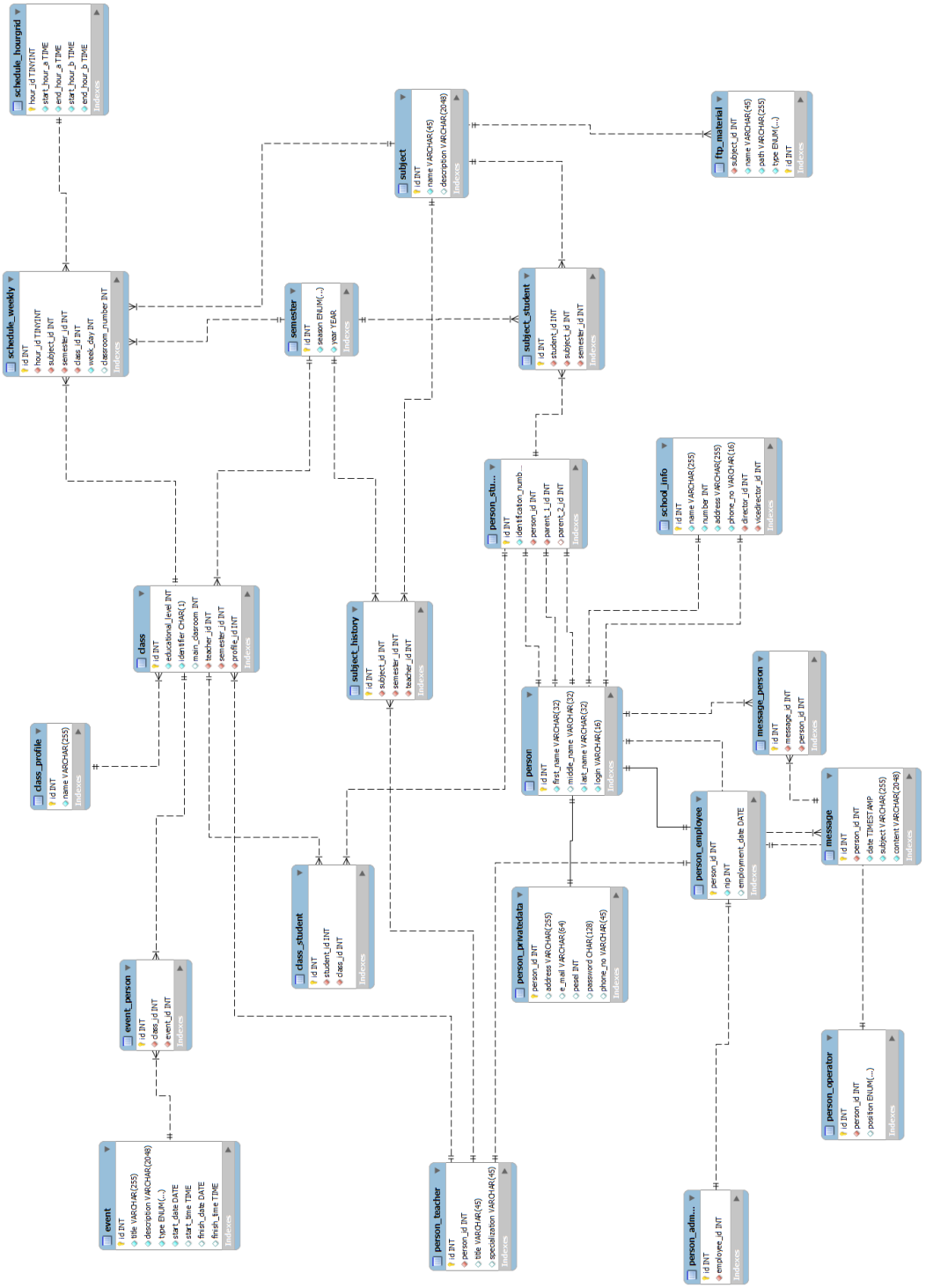


Figure E.2. Visualization of an early version of CampusNet database.

Bibliography

[1] M. Pilgram, *Dive Into Python 3*, Apress, Springer, 2009
[2] J. Forcier, W. Chun, P. Bissex, *Python Web Development with Django*, AddisonWesley, 2008
[3] M. Eernisse, *Build Your Own AJAX Web Applications*, Sitepoint, 2006
[4] K. Markov, V. Velychko, L.F.d.M. Lopez, J. Casellanos, *New Trends in Information Technologies*, Ithea, 2010
[5] I. Sommerville, *Software Engineering - eighth edition*, Addison Wesley 2007
[6] R. Elmasri, S.B. Navathe, *Fundamentals of Database Systems*, The Benjamin / Cummings Publishing Comp., 2004
[7] H. Partl:*German LaTeX*,TUGboat Vol. 9, No. 1 ('88)
[8] Wikipedia Free Encyclopedia `http://www.wikipedia.org`
[9] *The Django Book*, `http://djangobook.com/`
[10] *jQuery documentation*, `http://docs.jquery.com/Main_Page/`
[11] *Django documentation*, `https://docs.djangoproject.com/en/1.2/`
[12] *AJAX documentation*, `http://api.jquery.com/category/ajax/`
[13] *The Trac Project*, `http://trac.edgewall.org/`
[14] *Mercurial SCM*, `http://mercurial.selenic.com/`
[15] *MySQL*, `http://www.mysql.com/`
[16] *Language codes*, `http://www.i18nguy.com/`
[17] *Moodle*, `http://moodle.org/`
[18] *SchoolTool*, `http://schooltool.org/`
[19] *Prymus.info*, `http://prymus.info/`
[20] *Librus*, `http://www.librus.pl/`